

Phillip B. Ströbel

Von der Pythia zu Python

Einführung in die Programmierung für die Geschichtsforschung



November 2025

ver. 2025.h

Inhaltsverzeichnis

1	Einleitung – Programmierung für die Geschichtswissenschaft	7
1.1	Von der Pythia zu Python	7
1.2	Wieso soll ich als Historiker:in das Programmieren erlernen?	8
1.3	Grosse Sprachmodelle zum Coden	9
1.4	Zum Nutzen dieses Skripts	9
1.5	Nachschlagewerke	10
2	Variablen und Datentypen	13
2.1	Informationen speichern	14
2.2	Variablen	15
2.2.1	Variablen definieren	15
2.2.2	Namensräume	17
2.2.3	Variablen aufrufen	17
2.2.4	Mutationen von Variablen	19
2.2.5	Video zu Variablen	20
2.3	Datentypen	20
2.3.1	Integer	20
2.3.2	Float	22
2.3.3	String	23
2.3.4	Boolescher Wert	25
2.3.5	Video zu Datentypen	26
2.4	Quiz zu Variablen und Datentypen	26
3	Datenstrukturen und Loops	27
3.1	Ordnung, Ordnung, Ordnung	28
3.2	Listen	28
3.2.1	Video zu Listen	32
3.3	Dictionaries	32
3.3.1	Video zu Dictionaries	34
3.4	Sets	34
3.4.1	Set-Operationen	35
3.4.2	Video zu Sets	36
3.5	Tupel	36
3.5.1	Video zu Tupel	37
3.6	Kurze Zusammenfassung zu Datenstrukturen	37

3.7	Loops	37
3.7.1	for-Loops	38
3.7.2	while-Loops	41
3.7.3	Video zu Loops	42
3.8	Quiz zu Datenstrukturen und Loops	42
4	Reguläre Ausdrücke zur Textanalyse	43
4.1	Reguläre Ausdrücke	44
4.1.1	Automatentheorie und reguläre Ausdrücke	44
4.1.2	Reguläre Ausdrücke in Python	47
4.1.3	Video zu regulären Ausdrücken	54
4.2	Quiz zu regulären Ausdrücken	54
5	If-Else-Statements und Entscheidungslogik	55
5.1	Wenn ... dann	56
5.1.1	XML	56
5.2	Operatoren und Ausdrücke	63
5.2.1	Dateien laden	64
5.3	If-Else-Statements	67
5.3.1	Video zu If-Else-Statements	70
5.4	Vergleiche und Boolesche Operatoren	70
5.4.1	Video zu Vergleichen und Booleschen Operatoren	72
5.5	Quiz zu If-Else-Statements und Entscheidungslogik	72
6	Funktionen	73
6.1	Der Repetition entfliehen	73
6.1.1	Datei-Encodings	74
6.2	Funktionen in Python – Definition & Aufruf	77
6.2.1	Funktionsdefinition (def), Signatur, Aufruf	77
6.2.2	Argumentarten kompakt	79
6.2.3	Rekursivität	80
6.2.4	Gute Praxis, kompakt	82
6.2.5	Video zur Einführung in die Funktionen	82
6.3	Quiz zu Funktionen	82
7	Klassen und objektorientierte Programmierung	83
7.1	Zwei Paradigmen	84
7.2	Was sind Klassen und Objekte?	84
7.2.1	Einführungsvideo zu Klassen	85
7.3	Konstruktor <code>__init__</code> und Instanziierung	85
7.3.1	Video zu Konstruktor <code>__init__</code> und Instanziierung	85
7.3.2	Mutierbarkeit von Objekten	86
7.3.3	Video zu Mutierbarkeit	86
7.4	Attribute und Methoden	86
7.4.1	Video zu Attributen und Methoden	87
7.5	Quiz I zu Klassen und OOP	87
7.6	Vererbung und Subklassen	87

7.6.1	Video zu Vererbung und Subklassen	90
7.7	Polymorphie und <i>Method Overriding</i>	90
7.7.1	Video zu Polymorphie und <i>Method Overriding</i>	91
7.8	Speziellen Methoden (“dunder”)	91
7.8.1	Video zu speziellen Methoden	91
7.9	Quiz II zu Klassen und OOP	91
8	Fehlerbehandlung und robuste Programme	93
8.1	Fehlertypen verstehen	94
8.2	Fehler abfangen mit try/except	95
8.3	Video zu Fehlertypen und Fehler abfangen	99
8.4	Quiz zu Fehlerbehandlung und robusten Programmen	99
9	Ethische Reflexion und Verantwortung	101
9.1	Algorithmischer Bias und Quellenkritik	102
9.2	Verantwortlicher Umgang mit Datenquellen	102
10	Textanalyse historischer Daten mit NLTK, CLTK und spaCy	103
10.1	Textvorverarbeitung und Tokenisierung	105
10.1.1	Video zu Textvorbereitung und Tokenisierung	107
10.2	Stoppwörter und Wortfrequenzanalyse	107
10.2.1	Video zu Stoppwörtern und Wortfrequenzanalyse	110
10.3	Named Entity Recognition (NER)	110
10.3.1	Video zu NER	110
10.4	Quiz zur Textanalyse historischer Daten	111
11	Literaturverzeichnis und Glossar	113

Einleitung – Programmierung für die Geschichtswissenschaft

En France aussi, un pronostic s'impose en ce qui concerne l'histoire quantitative telle qu'elle sera pratiquée dans les années 1980: dans ce domaine au moins, **l'historien de demain sera programmeur ou ne sera plus.**

Emmanuel Le Roy Ladurie (1968)

DISCLAIMER I: Dieses Kapitel befindet sich noch im Ausbau und wird während des Kurses noch erweitert.

DISCLAIMER II: Bei Fehler (sowohl Typos wie auch inhaltlicher Natur): Bitte Mail an mich.

1.1 Von der Pythia zu Python

Die Pythia ist im antiken Griechenland allgemein bekannt als immer wieder gern gefragte Weissagerin im Orakel von Delphi. Von ihr holten sich die Griechen Rat, wenn wichtige Entscheidungen anstanden. Ihre Prophezeiungen waren indes jeweils unverständlich und mussten von weiteren anwesenden Priestern interpretiert werden. Die so übermittelten Botschaften waren aber im Normalfall keine 1:1-Anleitung für die Fragesteller:innen, was sie denn tun müssen. Ganz im Gegenteil: die Weissagungen waren (bewusst?) mehrdeutig und oft als Rätsel verpackt. Diese mussten die Fragesteller:innen adäquat lösen, um den Kern der Aussage Pythias zu verstehen und dementsprechend richtig zu handeln. Wenn man aber weiss, wie man eine Frage richtig stellt, dann sind die Chancen grösser, dass man eine interpretierbare Antwort erhält.

Auch die Programmiersprache Python funktioniert so ähnlich: Wenn man weiss, wie man sie bedient, dann erhält man die Antworten, die man braucht. Aber wieso sollte man sich als Historiker:in überhaupt mit dem Programmieren abgeben?

Emmanuel Le Roy Ladurie bezieht zum Programmieren bereits 1968 ganz klar Stellung und sagt, dass die Historiker:innen von morgen Programmierer:innen sind, oder sie existieren dann nicht mehr. Er blickte wohl auf einen Zeithorizont von gut 30 Jahren. Zwar hat sich seine Behauptung nicht bestätigt, aber es ist unumstritten, dass digitale Methoden vermehrt Einzug in die Geisteswissenschaften erhalten haben.

An der Front sind hierbei Forscher:innen im Bereich der sogenannten *Digital Humanities*. Sie wenden Algorithmen, Computerprogramme und Künstliche Intelligenz (KI) auf Texte, Bilder und Objektsammlungen (man denke an Münzen, Inschriften oder Manuskripte) an. Die daraus entstehenden Erkenntnisse sind von grosser Bedeutung: bei der aktuellen Digitalisierungswelle ist es für einen einzelnen Menschen unmöglich, für eine Arbeit alle Texte, alle Bilder oder alle Objekte gleichzeitig in den Blick zu nehmen. Der Trend geht vom sogenannten Close Reading zum Distant Reading (siehe Moretti (2013)), und dann wieder zum Close Reading zurück (um eine Analogie der Textanalyse zu bemühen. Das gilt aber auch für sämtliche anderen historischen Objekte). Dadurch, dass man aber zuerst das “Big-Picture” erhält, lässt sich um einiges präziser arbeiten.

In den letzten Jahren sind die Digital Humanities stetig gewachsen, was sich nicht zuletzt durch die grosse Anzahl an Publikationen und die immer besser besuchten Konferenzen in diesem Bereich zeigt. So existieren zum Beispiel die Online-Publikationen *Digital Humanities Quarterly*¹ und das *Journal of Open Humanities Data*,² oder die Konferenzen *Digital Humanities* der *Alliance of Digital Humanities Organizations*³ und diejenige der *Digital Humanities im deutschsprachigen Raum* (DHd).⁴ Daneben gibt es noch eine ganze Reihe an spezialisierten Publikationen: Im Bereich der Altertumswissenschaften finden wir die *Digital Classics Online*⁵, für das Mittelalter das Magazin *Digital Philology*.⁶ oder ganz generell, das *Journal of Digital History*.⁷ Für einen ausführlichen Überblick siehe <https://dhjournals.github.io/list>. Der Trend ist aber klar: Die digitalen Methoden gewinnen immer mehr an Bedeutung.

1.2 Wieso soll ich als Historiker:in das Programmieren erlernen?

Wie im vorhergehenden Abschnitt bereits erwähnt, wird in der Geschichte immer mehr Forschung mit digitalen Methoden vorangetrieben. Die ebenfalls aktuell stattfindenden Bemühungen der Gallerien, Bibliotheken, Archive und Museen (auf Englisch GLAM-Sektor, für *Galleries, Libraries, Archives and Museums*) im Bereich der Digitalisierung sorgen für immer mehr verfügbare, jedoch kaum mehr überblickbare Datenmengen. Um diese bewältigen zu können, kommt man nicht mehr um den

¹Siehe <https://dhq.digitalhumanities.org>.

²Siehe <https://dhq.digitalhumanities.org>

³Siehe <https://adho.org>.

⁴Siehe <https://digitalhumanities.de/>.

⁵Siehe <https://journals.ub.uni-heidelberg.de/index.php/dco>.

⁶Siehe <https://www.press.jhu.edu/journals/digital-philology-journal-media-val-cultures>.

⁷Siehe <https://journalofdigitalhistory.org>.

Computer herum. Zudem eignen sich Programmiersprachen auch dazu, historische Ansichten und Prozesse in einer vereinfachten Sprache zu modellieren, was zu neuen Erkenntnissen führen kann.

Das Paradigma des programmierenden Historikers mag sich vielleicht nicht so durchgesetzt haben, wie von Le Roy Ladurie prophezeit, aber es hielt sich hartnäckig, sodass sogar eine eigene Plattform, nämlich der *Programming Historian* entstand.⁸ Auf dieser Webseite veröffentlichen die Macher anfänger-gerechte und peer-reviewte Tutorials. Das in diesem Kurs Erlernte soll den Studierenden dabei helfen, die Notebooks so zu adaptieren, dass sie sie für ihre Zwecke nutzen können. Dafür ist ein grundlegendes Verständnis von Python unumgänglich.

Zuguterletzt möchte ich nicht unerwähnt lassen, dass man als klassisch arbeitende:r Historiker:in durch das Erlernen einer Programmiersprache neue Möglichkeiten schafft, Forschungsfragen zu beantworten. Obwohl die traditionellen Methoden nicht so schnell verdrängt werden, können digitale Ansätze eine wertvolle Ergänzung dazu sein.

1.3 Grosse Sprachmodelle zum Coden

Wir sollten den Elefanten im Raum direkt ansprechen: grosse Sprachmodelle wie ChatGPT (OpenAI), Gemini (Google), Claude (Anthropic), Perplexity (Perplexity AI) und dergleichen produzieren in Windeseile funktionierenden Code. In vielen Fällen macht dieser Code auch genau das, was die Benutzer:innen im Sinn hatten. Aber wie bei der Ausgabe von natürlicher Sprache kann es vorkommen, dass die Sprachmodelle halluzinieren oder fehlerhaften Code produzieren. Das hängt von verschiedenen Faktoren ab: welche Version des Sprachmodells habe ich benutzt? Oder auch: auf welchen Daten wurden die Sprachmodelle trainiert? Auch die Programmiersprachen entwickeln sich weiter. Bestimmte Prozeduren, spezifische Libraries und deren inhärente Methoden können sich verändern. Und obwohl die allermeisten grossen Sprachmodelle Zugang zum Internet haben, schaffen es die aktuellsten Informationen unter Umständen nicht in gewünschten Code. Das Resultat: fehlerhafte Skripte, die es zu **debuggen** —also korrigieren—gilt. Die Fähigkeit zu debuggen erfordert ein **q** Verständnis der Programmierung und der zentralen Konzepte. Genau dieses wollen wir uns mit diesem Skript aneignen.

Dabei möchte ich nicht vom Benutzen der grossen Sprachmodelle abraten. Vielmehr geht es darum, dass man den Code, den diese Modelle zurückliefern, kritisch begutachtet. Es gibt neben den Sprachmodellen noch dezidierte Modelle, die das Coding unterstützen, zum Beispiel *GitHub Copilot*.⁹

1.4 Zum Nutzen dieses Skripts

Dieses Skript ist in erster Linie als Begleitmaterial des im Herbstsemester 2025 zum ersten Mal stattfindenden Kurses “ Geschichte in der Praxis: Von der Pythia zu Python – Einführung in die Programmierung für die Geschichtswissenschaft.“ Es

⁸Siehe <https://programminghistorian.org/>. Ob diese Entwicklung nur auf Le Roy Ladurie zurückzuführen ist, lässt sich bezweifeln, jedoch ist die Verbindung einfach herzustellen.

⁹Siehe <https://github.com/features/copilot>.

führt jeweils in die wesentlichen Konzepte ein. Nach mindestens einem Unterkapitel folgt in der Regel ein Video, welches die beschriebenen Aspekte nochmals illustriert. Die Videos sind für alle, auch nicht UZH-Angehörige, zugänglich. Daneben gibt es nach jedem Kapitel (oder grösseren Teilkapitel) ein kleines Quiz mit maximal zehn Fragen. Nach je zwei bis drei Kapiteln folgt eine Übung auf OLAT. Die letzteren beiden sind ausserhalb des UZH-Netzes nicht verfügbar. Jedoch ist der Kurs so aufgebaut, dass man sich dessen Inhalte auch im Selbststudium aneignen kann.

Zu Beginn jedes Kapitels werden jeweils die Lernziele erklärt. Als Nächstes folgt der Inhalt, manchmal ergänzt durch Exkurse oder Kompakt-Abschnitte.

Lernziele Kapitel 1: Einleitung – Programmierung für die Geschichtswissenschaft

Die Lernziele sind in blauen Boxen.

Exkurs

Die Exkurse sind in grünen Boxen.

Kompakt

Die Kompakt-Abschnitte sind in roten Boxen.

Ich habe bewusst viele Beispiele inkludiert (siehe Code-Beispiel 1.1). Sie sollten die Funktionsweise des Besprochenen illustrieren. Am besten schreibt man die Code-Beispiele kurz ab, so festigt man die Konzepte bereits zum ersten Mal.

Code-Beispiel 1.1 Beispiel eines Code-Abschnittes

```
>>> print('Hello World!')  
... Hello World!
```

Nach jedem Kapitel gibt es zudem ein Jupyter-Notebook, das die besprochenen Konzepte nochmals aufgreift und in der Regel andere Beispiele bemüht, als jene des Kapitels. Am Ende eines jeden Notebooks gibt es zudem Aufgaben. Es empfiehlt sich, neben dem Notebook selbst noch diese Aufgaben zu erledigen. Werden die beherrscht, wird auch die Prüfung ein Kinderspiel. Diese Notebooks befinden sich hier: <https://pstroe.github.io/von-der-pythia-zu-python>.

1.5 Nachschlagewerke

Als generelle Nachschlagewerke können empfohlen werden:

- **Offizielle Python-Dokumentation:** Die offizielle Python-Dokumentation¹⁰ bietet stets die aktuellsten Informationen zu allerlei Fragen. Sie ist aber klar an Programmierer gerichtet.
- **W3Schools:** W3Schools¹¹ bietet nicht nur für Python, sondern auch für andere Programmier- und Auszeichnungssprachen (wie XML und HTML) ein

¹⁰Siehe <https://docs.python.org/3>.

¹¹Siehe <https://www.w3schools.com/python>.

reichhaltiges Nachschlagewerk. Zudem bietet es die Möglichkeit, die dort präsentierten Code-Snippets direkt auszuprobieren und anzupassen.

Variablen und Datentypen

[...] historians don't think in terms of independent and dependent **variables**. We assume the interdependency of variables as we trace their interconnections through time. Sorting them into separate categories just isn't very useful to us.

John Lewis Gaddis (2002)

Lernziele Kapitel 2: Variablen und Datentypen

- 1. Verstehen** Die Studierenden können den Begriff der *Variablen* in Python erläutern und beschreiben, wie Variablen als “Schubladen” fungieren, in denen (historische) Werte (z. B. Jahreszahlen, Bezeichnungen) gespeichert werden.
- 2. Analysieren** Die Studierenden können die vier grundlegenden Datentypen (*Integer*, *Float*, *String*, *Boolean*) unterscheiden und jeweils an einem historischen Beispiel deren Zweck und Einsatzgrenzen erklären.
- 3. Anwenden** Die Studierenden können in Python eigene Variablen anlegen, historische Daten mit passenden Datentypen kodieren und ein kurzes Skript schreiben, das diese Variablen ausgibt und je nach Typ unterschiedlich verarbeitet.
- 4. Bewerten** Die Studierenden können potenzielle Typkonflikte (z. B. Addition von Strings vs. Integers) identifizieren und einfache Strategien zur Typkonvertierung (`int()`, `str()`) anwenden, um ihre historischen Daten korrekt zu verarbeiten.
- 5. Erzeugen** Die Studierenden können ein kleines Datenmodell für eine historische Quelle entwerfen, in dem sie für jede Eigenschaft (Datum, Autor, Textlänge, Forschungsstatus) geeignete Variablen und Datentypen wählen.

2.1 Informationen speichern

Der Begriff der “Variablen”, den auch [Gaddis \(2002\)](#) im obigen Zitat verwendet, leitet sich vom lateinischen *variabilis* ab, was so viel wie ‘veränderlich’ bedeutet. In der Mathematik und der Statistik sind sie Platzhalter für messbare Werte in einer spezifischen Merkmalsdimension. Nehmen wir zum Beispiel an, dass wir Personen beschreiben wollen, so können wir das anhand von bestimmten Merkmalen wie Alter, Geschlecht, Grösse und Gewicht tun. Wenn wir nun für eine Gruppe von 20 Personen diese Merkmale messen, ordnen wir für jede Person in dieser Gruppe für jede dieser Variablen einen bestimmten Wert zu. Der Name impliziert schon, dass der Wert einer Variablen von Person zu Person unterschiedlich, also variabel sein kann. So wiegt Person A beispielsweise 87,4kg während Person B bloss 69,1kg wiegt. Person A wird dem Geschlecht ‘männlich’ zugeordnet, während Person B ‘weiblich’ ist. So speichern wir für die ganze Gruppe, welche unsere *Population* darstellt, diese Informationen in den entsprechenden Variablen und erzeugen so einen Datensatz.

Wir bemerken bereits beim Sammeln der Daten, dass sich Datentypen der Variablen unterscheiden. Beim Geschlecht handelt es sich um eine sogenannte **kategoriale** Variable. Solche Variablentypen nehmen einen (oder teils auch mehrere) von vordefinierten Kategorien an. Für die Merkmalsdimension “Geschlecht” können das zwei (männlich, weiblich) oder je nach Definition von “Geschlecht” auch mehr sein. Den kategorialen Variablen stehen die **metrischen** Variablen gegenüber. Diese nehmen einen Zahlenwert an, der wiederum unterschiedlich dargestellt werden kann. Beispielsweise wird das Alter oft mit einer ganzen Zahl dargestellt. Person A könnte dementsprechend angeben, 31 Jahre alt zu sein, während Person B ihr Alter mit 25 angibt. Das Gewicht hingegen wird mit einer rationalen Zahl beschrieben. Je nach Art der Variablen lassen sich unterschiedliche Berechnungen und Statistiken anstellen.

Halten wir fest: Eine Variable ist ein Gefäss, das entlang einer bestimmten Merkmalsdefinition für ein Objekt in einem Datensatz einen bestimmten Wert (kategorial oder metrisch) annimmt. Welchen Wert stellen Variablen nun für Historiker:innen dar? Diese Frage lässt sich nicht pauschal beantworten, da dies von verschiedenen Faktoren wie den Forschungsfragen, den zu verwendenden Daten und der Modellierbarkeit des Problems abhängt. Würde jemand versuchen wollen, die Überzeugungskraft von Ciceros Reden quantitativ zu messen und dafür als Variablen die Länge der Sätze, Anzahl Augenkontakte zwischen Cicero und dem Publikum und Ciceros Lautstärke definieren, wäre dieses Projekt zum Scheitern verurteilt, da die erste Variable nichts über die Überzeugungskraft aussagt und die letzten beiden ihren Teil dazu beitragen könnten, aber schlicht nicht zu erheben oder bekannt sind. Im Gegensatz dazu nehmen wir nun Münzfunde als Datenbasis. Dem Objekttyp Münzhort liegen inhärent unter anderem folgende Variablen zugrunde: Datum von, Datum bis, Nennwert (Denarius, Aureus, etc.), Münzstätte, Fundort, etc. Genau dies hat bereits [Lockyear \(1997\)](#) für Münzhorte aus Rumänien gemacht und dabei Aussagen zum Auftreten bestimmter Münzen in Dakien machen oder gewisse metallurgisch mit Münzhorten aus Italien vergleichen können.

Um nochmals auf das Eingangszitat zurückzukommen: Das möchte ich an dieser Stelle etwas entkräften. Gerade mit Gaddis Aussage, dass Historiker:innen stets von

der Unabhängigkeit von Variablen ausgehen, wird die Klassifikation in **abhängige** und **unabhängige** Variablen, die ihm gemäss nicht vorgenommen wird, ja eben doch gemacht. Dass es Sinn machen kann, die Variablen noch in zwei Unterklassen zu teilen, zumindest was quantitative Analysen angeht, soll folgendes Beispiel verdeutlichen: Gehen wir wiederum auf unseren Datensatz mit den Personen ein, so liesse sich die Hypothese aufstellen, dass männliche Personen in der Regel schwerer sind als weibliche Personen. In diesem Szenario wäre das Gewicht die abhängige Variable, während dem Geschlecht die Rolle der unabhängigen Variablen zufällt. Anders ausgedrückt: Wir machen das Gewicht am Wert des Geschlechts fest (es soll, unserer These zufolge, vom Geschlecht abhängen). Auf die Münzhorten übertragen: Man könnte sich hier überlegen, ob die metallurgische Zusammensetzung der Münze vom Fundort abhängt. Oder aber: je näher der Fundort (unabhängige Variable) an einer Heeresstrasse liegt, desto grösser ist die Münzhorte (abhängige Variable).

Wir haben nun gesehen, was für Informationen wir *wie* speichern können und wie diese Informationen voneinander abhängig sein können. Im Folgenden werden wir lernen, wie wir Variablen in Python definieren, wie wir erste, einfache Mutationen daran vornehmen und wie wir mit ihnen rechnen. Danach machen wir uns Gedanken zu den verschiedenen Datentypen und wie wir sie für die Geschichtsforschung einsetzen können.

2.2 Variablen

Wie im vorherigen Abschnitt bereits erwähnt, können wir Variablen als Gefässe—oder auch Schubladen—betrachten, die wir mit Inhalt füllen. Das durch die Variablen zu beschreibende Objekt würde so eine Kommode darstellen, und wenn wir jetzt Schublade(n) öffnen—denn es können mehrere gleichzeitig geöffnet sein—und uns die darin vorliegenden Werte ansehen—also die Variable auslesen—ermöglicht uns dies, Aussagen über das Objekt zu treffen. Und genau diese Schritte wollen wir uns jetzt in Python anschauen. Wir werden Variablen definieren, inspizieren, miteinander verrechnen und manipulieren. Wir bedienen uns dafür eines Datensatzes aus der Antike: *Iron Age and Roman Coins from Wales* von der University of Wales. Das Datenset ist frei auf *Archaeology Data Service* verfügbar.¹ Wir werden es auch für zukünftige Demonstrationen hier nutzen, weshalb es ratsam ist, sich die Daten herunterzuladen.

2.2.1 Variablen definieren

Variablen in Python müssen bestimmte Bedingungen erfüllen. So könne sie aus einer (fast) beliebigen Reihenfolge von Zeichen bestehen, wobei die Zeichen das Alphabet (Gross- wie Kleinbuchstaben), Zahlen, das Minuszeichen `-` und den Unterstrich `_` umfassen. Angenommen, wir haben nun eine solche Münze vor uns, wie sie in Abbildung 2.1 dargestellt ist, dann könnten wir, sofern wir es genau wissen, ihr Prägejahr wie im Code-Beispiel 2.1 gezeigt erfassen:

Code-Beispiel 2.1 Beispiel zum Definieren des Jahrs der Münze aus Abbildung 2.1

¹Siehe <https://doi.org/10.5284/1000263>.



(a) Vorderseite



(b) Rückseite

Abbildung 2.1 Münze des Nero (As) aus dem Jahr 65 n. Chr. (RIC I Nero 416).

General period	Metal	Weight	Denomination	Date from	BCorAD1	Date to	BCorAD2	Reverse type	Catalogue reference
4th Century AD Roman	Copper Alloy		Nummus	337	AD	340	AD	Uncertain	Uncertain
3rd Century AD Roman	Copper Alloy		Radiate	260	AD	269	AD	Uncertain	Uncertain
3rd Century AD Roman	Billon		Radiate	251	AD	253	AD	Uncertain	Uncertain
Augustus & 1st Century AD Roman	Copper Alloy		Dupondius	43	AD	64	AD	CERES AVGVSTA SC	as RIC I Claudius 110
Augustus & 1st Century AD Roman	Copper Alloy		Dupondius	43	AD	64	AD	CERES AVGVSTA SC	as RIC I Claudius 110
Augustus & 1st Century AD Roman	Copper Alloy		As	43	AD	64	AD	Minerva SC	as RIC I Claudius 100

Tabelle 2.1 Die ersten sechs Zeilen und einige ausgewählte Spalten des Münzdatensets aus Wales.

```
>>> date_from = 65
```

Wir entnehmen diesen Wert dem Datensatz. Zum besseren Verständnis schauen wir uns einen Teil der Tabelle 2.1 (die originale Tabelle hat in Wirklichkeit noch viel mehr Spalten), in welcher die Daten erfasst sind, einmal genauer an (zumindest die ersten Zeilen). Für die meisten Münzen lässt sich das genaue Prägedatum nicht bestimmen, weshalb wir in der Tabelle die zwei Merkmaldimensionen **Date from** und **Date to** finden. Diese beiden Werte geben also ein Zeitfenster an, das den Entstehungszeitraum der Münze beschreibt und das Numismatiker:innen eruiert haben. Für die Münze des Nero im Wales-Datensatz lautet das **Date to** ebenfalls 65.

Für die Münze des Nero können wir noch weitere Variablen definieren. Wir finden im Datensatz Angaben zum Gewicht, Durchmesser, Material, Wert, Zustand usw. Das Code-Beispiel 2.2 zeigt weitere Möglichkeiten, die Variablen zu definieren:

Code-Beispiel 2.2 Beispiel zum Definieren weiterer Variablen der Münze aus Abbildung 2.1

```
>>> date_to = 65
>>> weight = 7.86
>>> bc_or_ad = "AD"
>>> reverse_type = "PONTIF MAX TR POT IMP P P SC Apollo"
```

Wir sehen, dass wir für gewisse Variablen die Werte zwischen zwei Anführungszeichen setzen "...". Das ginge auch mit einfachen Anführungszeichen '...'. Das tun wir, um Python zu sagen, dass es sich hierbei um Text oder programmiersprachlich einen **String** handelt. Für die Merkmaldimensionen **date_from** und **date_to** haben wir jeweils eine sogenannte **Integer** verwendet und für die Dimension **weight** eine **Float**. Wir kommen im Kapitel 2.3 noch genauer auf die Datentypen zu sprechen.

Zurück zum Definieren der Variablen: Wir sehen, dass wir pro Zeile jeweils eine Variable definieren. Generell dient ein Zeilenumbruch in Python dazu, dem **Code-Interpreter** mitzuteilen, dass eine neue Anweisung folgt. Beim Code-Beispiel 2.2 speichert der Interpreter also zuerst den Wert 65 in der Variable **date_to** und danach

den Wert `7.86` in der Variable `weight`. Er arbeitet sich im Anschluss Zeile für Zeile weiter durch den Code, bis es keine weiteren Anweisungen mehr gibt.

Wir sehen auch, dass wir den Variablen sinnvolle Namen gegeben haben. Theoretisch hätten wir die Variable `date_to` auch einfach `my_funny_variable` oder `asdfg` nennen können. Es empfiehlt sich jedoch, die Variablen so zu benennen, dass man leicht erraten kann, welche Art von Wert sich darin befindet. Das heisst, dass wir uns, wenn immer möglich, der natürlichen Sprache bedienen. Wir sollten aber darauf achten, dass die Variablennamen kurz sind. Eine Variable wie `text_that_we_find_on_the_obverse_of_the_coin` ist zwar denkbar, macht den Code aber unleserlich.

Zuguterletzt sei noch erwähnt, dass gewisse Variablennamen nicht möglich sind. Beispielsweise kann eine Variable nicht mit einer Zahl beginnen. Wenn wir nun aus irgendeinem Grund für mehrere Münzen das Gewicht festlegen und das im Code explizit machen wollen würden, können wir die Variablen nicht als `1_weight`, `2_weight`, usw. definieren, sondern müssen die Zahl nach hinten stellen: `weight_1` und `weight_2`.

2.2.2 Namensräume

Wir haben nun für unsere Münze mehrere Variablen definiert. Diese “leben” in einem sogenannten **Namensraum** —wir könnten auch “Speicher” sagen, und zwar solange, **Q** bis das Programm fertig ausgeführt ist—also bei der letzten Zeile angelangt ist—oder, was treffender für unseren Umgang mit Jupyter Notebooks ist, bis das Jupyter Notebook beendet wird oder dessen **Kernel** neu gestartet wird. Wenn wir Variablen **Q** wie in den Code-Beispielen definieren, so landen diese im **lokalen Namensraum**. **Q** Dieser liegt hierarchisch gesehen und dem **globalen Namensraum** welcher wiederum **Q** unter dem sogenannten **Built-in** Namensraum liegt. Wir werden in Kapitel 7 noch **Q** mals auf die Namensräume zurückkommen. Für den Moment geben wir uns damit zufrieden, dass die von uns definierten Variablen (und später auch Funktionen, siehe Kapitel 6) im lokalen Namensraum gespeichert werden und abrufbar sind.

2.2.3 Variablen aufrufen

Der Sinn und Zweck der Definition von Variablen ist deren Wiederverwendung im weiteren Code. Sie stehen somit für die weitere Verarbeitung zur Verfügung. Die Abbildung 2.2 zeigt auf, wie die Variablen im lokalen Namensraum auf ihre Werte zeigen. In unserem Fall zeigen alle Variablen auf einen eigenen Wert. Wollen wir nun einen solchen Wert aufrufen, indem wir ihn zum Beispiel als Output einer Jupyter Notebook-Zelle ausgeben, können wir das mit einer Built-in-Funktion namens `print` machen. `print` lebt im Built-in Namensraum von Python. Darin sind Variablen und vor allem Funktionen vordefiniert, die wir ständig nutzen können. Dazu zählt eben auch, Output entweder auf die Konsole (wenn man sich in einem Terminal befindet) oder in einem Jupyter Notebook “auszudrucken”.

Nachdem wir also beispielsweise die Variable `date_from` definiert haben, können wir sie uns wieder ausgeben lassen, wie das Code-Beispiel 2.3 illustriert:

Code-Beispiel 2.3 Beispiel zum Ausgeben einer Variablen

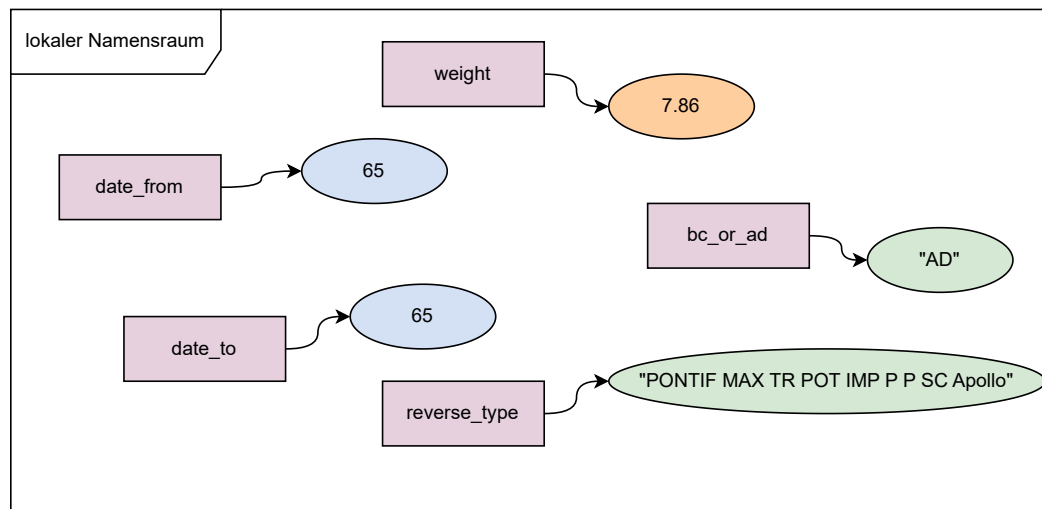


Abbildung 2.2 Beispiele der Variablen-Definition im lokalen Namensraum für die Münze des Nero.

```
>>> print(date_to)
65
```

Das geht auch mit Variablen, in welcher wir einen String abgespeichert haben (Code-Beispiel 2.4):

Code-Beispiel 2.4 Beispiel zum Ausgeben einer String-Variablen

```
>>> print(reverse_type)
PONTIF MAX TR POT IMP P P SC Apollo
```

Funktionen werden mit ihrem Funktions-Namen aufgerufen—in unserem Fall `print`, auf welchen (ohne Abstand) sogenannte **Argumente** —eingeschlossen von runden Klammern—folgen. Abbildung 2.3 zeigt, wie die Print-Funktion, die im Built-in-Namensraum Bestand hat, über den globalen Namensraum in den lokalen Namensraum vordringt und in beiden Namensräumen nach der Variablen, die wir der Funktion als Argument mitgegeben haben, sucht. Wird sie fündig, führt sie die entsprechende Aktion, für welche sie definiert worden ist, aus. Sollten wir eine Variable ausgeben wollen, welche wir nicht definiert haben, erhalten wir eine Fehlermeldung, wie das Code-Beispiel 2.5 zeigt. Zu Fehlern und deren Behandlung wird Kapitel 8 noch weitere Erläuterungen bieten, während Funktionen allgemein in Kapitel 6 noch vertiefter behandelt werden.² Für die Ausgabe der Variable `revers_type` ist noch anzumerken, dass die Ausgabe ohne die Anführungszeichen erfolgt. Diese werden für Strings nur für die Definition benötigt.

Code-Beispiel 2.5 Beispiel zum Ausgeben einer String-Variablen

```
>>> print(diameter)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
```

²Bis dahin werden wir jedoch einige Built-in-Funktionen benutzen. Eine Übersicht über alle solchen Funktionen findet sich auf <https://docs.python.org/3/library/functions.html>. Es gibt insgesamt 71 Built-in-Funktionen. Es ist nützlich, sich die 30 gängigsten davon zu merken. Diese werden im Verlaufe der folgenden Kapitel laufend eingeführt.

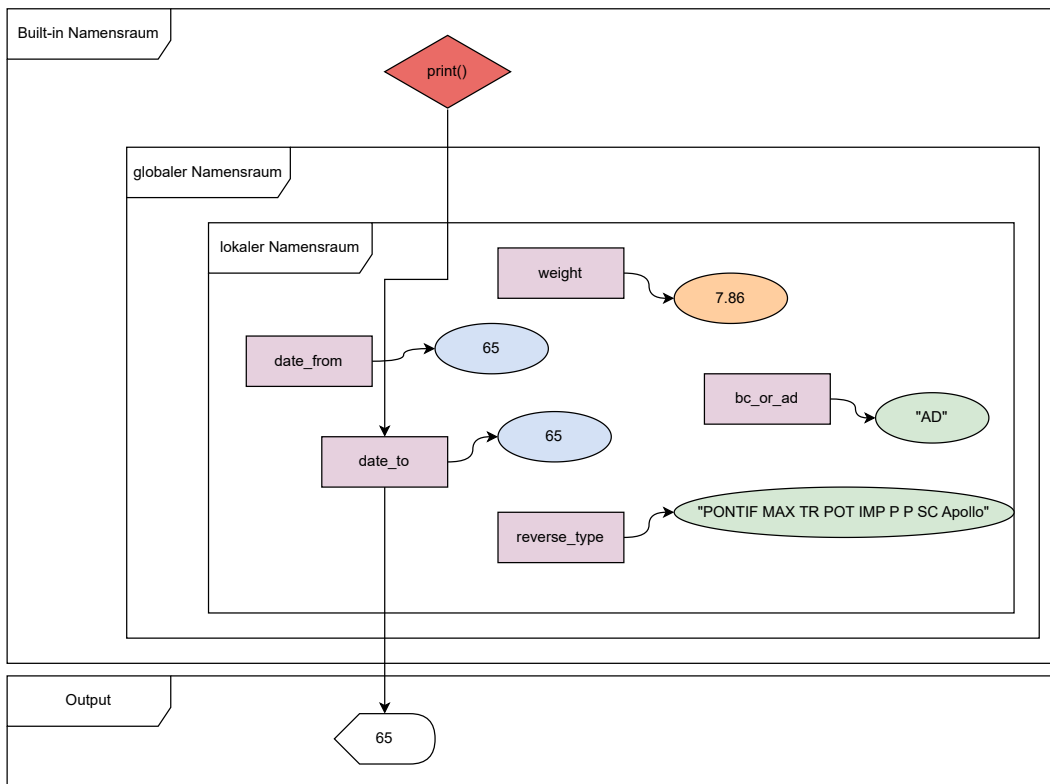


Abbildung 2.3 Beispiel der print-Funktion und ihrer Aktion im Code.

```
NameError: name 'diameter' is not defined
```

2.2.4 Mutationen von Variablen

Bei näherer Betrachtung unserer im lokalen Namensraum definierten Variablen fällt uns auf—wie in Abbildung 2.2 dargestellt—, dass die Variablen `date_from` und `date_to` denselben Wert aufweisen. Anstatt also für beide diese Variablen den Wert 65 zu definieren, könnten wir zuerst den Wert für die Variable `date_from` setzen und diese dann mit `date_to` gleichsetzen. Im Code drücken wir das so aus wie in Beispiel 2.6:

Code-Beispiel 2.6 Beispiel zum Gleichsetzen von Variablen mit dazugehörigen Print-Statements

```
>>> date_from = 65
>>> date_to = date_from
>>> print(date_from)
65
>>> print(date_to)
65
```

Im lokalen Namensraum ändert sich die Definition der Variablen wie in Abbildung 2.4 gezeigt. Das heißt, dass nun beide Variablen auf den exakt gleichen Wert zeigen. Wenn wir nun aber zum Beispiel den Wert von `date_from` in 64 umdefinieren, so ändert sich die Definition von `date_to` nicht. Diese Variable zeigt in diesem Fall immer noch auf 65. Die erste Variable wurde hingegen **überschrieben**.

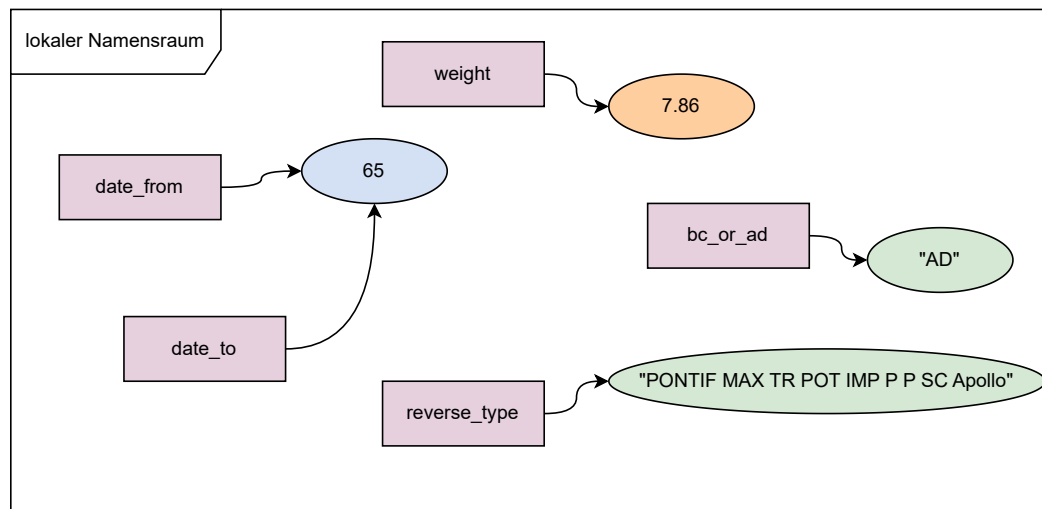


Abbildung 2.4 Beispiel der Gleichsetzung von zwei Variablen im lokalen Namensraum.

Ihr alter Wert—also 65—ist nicht mehr abruf- oder rekonstruierbar. Ganz genau genommen wird bei der “Umdefinition” der Variable `date_from` die ursprüngliche Variable im Speicher gelöscht und komplett neu angelegt. Dies geschah deshalb, weil Variablen, die als Integer definiert sind, zu den nicht mutierbaren Variablen gehören

❏ (wir widmen uns dem Thema der [Mutierbarkeit](#) im Unterkapitel 7.3.2).

2.2.5 Video zu Variablen

📺 Schaue dir jetzt das [Video 2.2.5](#) zu den Variablen an und bearbeite dann das nächste Unterkapitel.

2.3 Datentypen

Wir wollen uns nun den Datentypen widmen. Im vorherigen Unterkapitel haben wir gesehen, dass wir bereits Variablen von den Typen Integer, Float und String definiert haben. Daneben gibt es noch die Boolesche Variablen. In der Folge behandeln wir diese vier wichtigsten Datentypen und ihre Operationen.

2.3.1 Integer

Wir sind dem Datentypen Integer bereits in der Form der Variablen `date_from` und `date_to` begegnet. Integers sind ganze Zahlen, definiert über den natürlichen Zahlen, inklusive der 0 und den negativen Zahlen. Was man selbstverständlich mit Zahlen machen kann, ist Rechnen. Das macht auch das “Kerngeschäft” von Python aus, denn im Grunde kann man es als überpotenten Rechner verstehen. Schauen wir uns ein paar Rechenbeispiele anhand eines Münzfundes in *Augusta Raurica* an (siehe [Martin \(1977\)](#) für weitere Details).

1937 fand man bei Ausgrabungen bei der Insula 17 einige Silbermützen, die sich wie folgt kategorisieren liessen:

- 1 Halbvictoriat

- 6 Denare (röm. Republik)
- 2 Quinare (röm. Republik)
- 1 Denar (augusteisch)
- 2 Quinare (augusteisch)

Für Python ist es ein Leichtes, nun die Summe zu berechnen. Das Code-Beispiel 2.7 zeigt, wie wir das mit Variablen bewerkstelligen:

Code-Beispiel 2.7 Beispiel zum Addieren von Variablen

```
>>> halbvictoriat = 1
>>> rep_denar = 6
>>> rep_quinar = 1
>>> aug_denar = 1
>>> aug_quinar = 2
>>> summe = halbvictoriat + rep_denar + rep_quinar + aug_denar + aug_quinar
>>> print(summe)
11
```

Wir haben das Ergebnis—also elf Münzen in der Gesamtheit—in der Variable `summe` abgespeichert. Wenn wir nun lediglich an der Anzahl Denare interessiert sind, könnten wir auch nur deren Werte zusammenrechnen. Zu Illustrationszwecken möchten wir aber die Subtraktion sehen, wie das Code-Beispiel 2.8 veranschaulicht:

Code-Beispiel 2.8 Beispiel zum Subtrahieren von Variablen

```
>>> diff = summe - halbvictoriat - rep_quinar - aug_quinar
>>> print(diff)
7
```

Wir haben also sieben Denare in diesem Münzfund. Wie viel aber macht der Anteil Denare am ganzen Fund aus? Das Code-Beispiel 2.9 macht deutlich, wie man zu diesem Ergebnis kommt:

Code-Beispiel 2.9 Beispiel zum Berechnen von Prozenten

```
>>> anteil_denare = diff / summe * 100
>>> print(anteil_denare)
63.63636363636363
```

Wir sehen hier zweierlei: zum einen die beiden Operatoren `/` für die Division und `*` für die Multiplikation, zum anderen erhalten wir als Resultat, das wir als “63.64 Prozent” bezeichnen würden, keine Integer zurück erhalten, sondern eine Float. Die Variablen-Typen können wir mit einer weiteren Built-In-Funktion namens `type()` in Erfahrung bringen (siehe Code-Beispiel 2.10):

Code-Beispiel 2.10 Beispiel Anzeigen von Variablen-Typen

```
>>> print(type(summe))
<class 'int'>
>>> print(type(anteil_denare))
<class 'float'>
```

```
>>> print(type(reverse_type))
<class 'str'>
```

Bevor wir uns dem Beispiel zuwenden: Es ist zu beachten, dass wir hier die zwei Funktionen `print()` und `type()` ineinandergeschachtelt haben. Python evaluiert diese Funktionen von innen nach aussen. Zuerst wendet es also die `type()`-Funktion auf die Variable `summe` an, anschliessend die `print()`-Funktion auf das Resultat der `type()`-Funktion. Auch das vertiefen wir noch im Kapitel 6.

Wir sehen im Code-Beispiel ausserdem, dass die unterschiedlichen Variablen, die wir definiert haben, zu unterschiedlichen **Klassen** gehören (zu Klassen kommen wir in Kapitel 7). Warum erhalten wir nun auf einmal einen Float? Das liegt an der Division. Die Division gibt als Resultat stets eine Float aus. Aus der Division $4/2$ folgt somit das Resultat 2.0. Generell gilt zu merken: Sobald in irgendeiner Rechnung eine Float involviert ist, wird das Resultat ebenfalls vom Datentyp Float sein. Möchten wir also den Anteil der Denare als Integer darstellen, können wir wiederum auf eine Built-In-Funktion zurückgreifen, wie das Code-Beispiel 2.12 zeigt:

Code-Beispiel 2.11 Beispiel zur Konversion von Variablen

```
>>> print(int(anteil_denare))
63
>>> print(round(anteil_denare))
64
>>> print(float(halbvictoriat))
1.0
>>> print(str(halbvictoriat))
```

Die Funktion `int()` verwandelt jede Float in eine Integer. Wichtig dabei zu beachten ist, dass sie immer abrundet. So rundet sie 8.1 auf 8 ab genau so, wie sie es mit 8.9 tun würde. Um diesem Verhalten entgegenzuwirken gibt es die Funktion `round()`. Diese rundet so, wie wir es in der Schule gelernt haben: beispielsweise also 8.49 auf 8 und 8.5 auf 9. Zuguterletzt findet sich in Code-Beispiel 2.12 auch noch die Funktion `str()`, die aus sämtlichen Datentypen einen String produziert.

2.3.2 Float

Der Datentyp Float ist uns ebenfalls schon begegnet. Floats sind Gleitkommazahlen. Man kann sie entweder direkt als Float definieren oder aber sie entstehen durch Berechnungen wie der Division. Sie sind besonders wichtig für die statistischen Auswertungen. Wir können beispielsweise den durchschnittlichen Gewichtsverlust von Münzen berechnen. Im Datensatz der University of Wales finden sich fünf Denarien, die in Neros Zeit fallen und eine Gewichtsangabe haben. Zurzeit Neros soll der Denar noch ein Gewicht von 3.4g gehabt haben. Dieses Standardgewicht und die Angaben im Datensatz halten wir in Variablen fest und berechnen dann den durchschnittlichen Gewichtsverlust:

Code-Beispiel 2.12 Beispiel zum Rechnen mit Floats, inklusive negativem Bereich

```
>>> orig_coin = 3.4
>>> coin_1 = 3.08
>>> coin_2 = 2.68
```

```
>>> coin_3 = 2.87
>>> coin_4 = 3.15
>>> coin_5 = 3.09
>>> verlust = ((coin_1 - orig_coin) + (coin_2 - orig_coin) + (coin_3 -
orig_coin) + (coin_4 - orig_coin) + (coin_5 - orig_coin))/5
>>> print(verlust)
-0.4259999999999999
```

Wir bemerken, dass dieselben Rechenregeln herrschen, die wir bereits aus der Grundschule kennen: Punkt vor Strich und Klammern vor allen anderen Termen. Im obigen Beispiel werden so zuerst die Differenzen vom Originalgewicht mit den gemessenen Gewichten berechnet und dann diese Differenz durch fünf dividiert (da wir ja insgesamt fünf Münzen haben). Der Wert -0.426 (gerundet) gibt den durchschnittlichen Gewichtsverlust in Gramm an. Dieser kann durch unterschiedliche Ursachen bedingt sein. Das Minus vor der Float gehört jedoch zur Zahl dazu. So stellt Python Zahlen im negativen Bereich dar.

Mit der Funktion `float()` lassen sich im Übrigen auch Strings zu Floats transformieren. Liegen uns beispielsweise Daten in Textform, also Strings vor, und wollen wir damit aber beginnen zu rechnen, braucht es eine entsprechende Umwandlung. Code-Beispiel 2.13 veranschaulicht dies:

Code-Beispiel 2.13 Beispiel zum Transformieren von Strings zu Floats

```
>>> orig_coin_string = "3.4"
>>> print(float(orig_coin_string))
3.4
>>> print(type(float(orig_coin_string)))
<class 'float'>
```

2.3.3 String

Wenn wir uns dem Datentyp String zuwenden, würde man meinen, dass alles, was für Integers und Floats gilt, auf die Strings nicht zutrifft. Das ist aber nur die halbe Wahrheit, denn Python erlaubt uns auch das “Rechnen” mit Strings. Zumindest die Operatoren `+` und `*` können wir auch auf Strings anwenden und sie funktionieren so, wie man das etwa erwarten würde. Die Anwendung illustriert das Code-Beispiel 2.14:

Code-Beispiel 2.14 Beispiel zur Anwendung von `+` und `*` für Strings

```
>>> fundort = "Augst"
>>> objekt = "Denar"
>>> print(fundort + " " + objekt)
Augst Denar
>>> print('-' * 5)
-----
>>> print("Fundort: " + fundort + "\n" + '-' * 5 + "\n" + "Objekt: " + objekt)
Fundort: Augst
-----
Objekt: Denar
```

Dieses Beispiel ist schon ein wenig komplexer. Wir definieren zuerst zwei Variablen: `fundort` und `objekt`. Mit dem darauf folgenden `print()` [konkatenerieren](#) wir drei Strings, wobei " " (da gibt es einen Abstand zwischen den Anführungszeichen) sicherstellt, dass zwischen Fundort und Objekt ein Leerzeichen eingefügt wird.

Um eine Linie auszugeben, wäre es mühsam, fünf oder noch mehr `--`-Zeichen zu schreiben. Einfacher geht das, indem man ein solches Zeichen mit der gewünschten Anzahl multipliziert. Schliesslich sehen wir in der letzten Code-Zeile, wie Fundort und Objekt zusammen ausgegeben werden. Davor schreiben wir jedoch eine Bezeichnung der Variablen und setzen eine Trennlinie dazwischen. Beim Zeichen `\n` handelt es sich um eine codierte Form des Zeilenumbruchs. Es gibt noch weitere solche Zeichen, die man [Escape-Sequenzen](#) nennt. Diese vertiefen wir im Kapitel 4 zu regulären Ausdrücken.

Ganz nebenbei zeigt uns das Code-Beispiel auch, dass Strings sowohl zwischen einfachen `'...'` als auch doppelten Anführungszeichen `"..."` stehen können. Ganz wichtig: Es gibt auch den sogenannten leeren String, den wir `''` oder `""` schreiben. Dieser hat die Länge 0. Wir müssen die Länge von Strings übrigens nicht mühsam selbst auszählen. Dazu gibt es die Built-In-Funktion `len()`. Diese zählt in einem String alle vorkommenden Zeichen, inklusive aller Leerzeichen. Das Code-Beispiel 2.15 verdeutlicht dies:

Code-Beispiel 2.15 Beispiel zum Bestimmen der Länge von Strings mit `len()`

```
>>> print(len(reverse_type))
35
>>> print(len(fundort))
5
```

Diese Längenangaben werden übrigens in Integers ausgegeben, womit wir wiederum rechnen können.

Das Ausgeben von Strings, welches das Konkatenieren von einigen Variablen erfordert, kann unter Umständen mühsam werden. Dafür gibt es in Python die sogenannten f-Strings. Mit diesen ist es möglich, Variablen in den String einzubauen, die dann vom Python-Interpreter mit dem entsprechenden Wert ersetzt werden. Dabei werden zum Beispiel Floats, die man nicht mit Strings konkatenieren kann, automatisch in Strings umgewandelt. Um das zu verdeutlichen: Code wie im Beispiel 2.16 gezeigt, resultiert in einer Fehlermeldung:

Code-Beispiel 2.16 Beispiel einer Fehlermeldung beim Versuch, Strings mit einer Integer zu konkatenieren

```
>>> print("Mein " + objekt + " wiegt " + coin_1 + " Gramm.")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "float") to str
```

Den Fehlermeldungen widmen wir uns noch im Detail. Für den Moment reicht uns, dass ein sogenannter `TypeError` erkannt worden ist. Wir interpretieren den so, dass die zwei Datentypen String und Float nicht miteinander konkateniert werden können. Wenn wir nun einen f-String verwenden, können wir dieses Problem umgehen, wie das Code-Beispiel 2.17 beweist:

Code-Beispiel 2.17 Beispiel eines f-Strings

```
>>> print(f"Mein {objekt} wiegt {coin_1} Gramm.")
Mein Denar wiegt 3.08 Gramm.
```

Zuerst ist es wichtig, das dem String vorangestellte `f` zu erkennen. Dies weist den Interpreter darauf hin, dass nun ein formatierter String folgt. Die von uns zuvor definierten Variablen `objekt` und `coin_1` finden wir im obigen Code in geschweiften Klammern `{...}`. Das ist für den Python-Interpreter das Zeichen, dass er in den Namensräumen nach den entsprechenden Variablen Ausschau hält, sie in Strings umwandelt und in den String einfügt. So könnten wir auch das Code-Beispiel 2.14 viel simpler niederschreiben:

Code-Beispiel 2.18 Beispiel des angepassten Code-Beispiels 2.14

```
>>> print(f"Fundort: {fundort}\n{'-'*5}\nObjekt: {objekt}")
Fundort: Augst
-----
Objekt: Denar
```

Wir sehen im Code-Beispiel 2.18, dass auch ganze Ausdrücke in den geschweiften Klammern stehen dürfen (hier `'-'*5`). Diese werden vom Interpreter ebenfalls zuerst evaluiert und, sofern das Resultat einen String darstellt oder sich in einen solchen verwandeln lässt, in den String eingefügt.

2.3.4 Boolescher Wert

Wir widmen uns jetzt noch dem letzten Datentypen, nämlich den [booleschen Werten](#). Sie haben ihre Bezeichnung vom englischen Mathematiker George Boole (1815–1864) [q](#) geerbt und sind zentral in der booleschen Algebra. In der Informatik werden boolesche Variablen verwendet, um Wahrheitswerte abzubilden, und können dementsprechend nur zwei Werte annehmen, nämlich *wahr* und *falsch* oder in Python ausgedrückt: `True` und `False`. Wir werden vor allem im Kapitel 5 zur Entscheidungslogik nochmals auf die booleschen Werte eingehen.

Vorweg nehmen wir aber einige Generalia, die für Boolesche Werte gelten. Zuerst zur Definition: In Python können boolesche Werte ganz einfach in Variablen gepackt werden (siehe Code-Beispiel 2.19):

Code-Beispiel 2.19 Beispiel für die Definition von booleschen Variablen.

```
>>> fundort_bekannt = True
>>> print(fundort_bekannt)
True
>>> gewicht_bekannt = False
>>> print(gewicht_bekannt)
False
```

Die Built-in-Funktion `bool()` evaluiert für jedes Argument dessen Wahrheitswert. Es gilt folgendes Verhalten zu beachten (siehe Code-Beispiel 2.20:

Code-Beispiel 2.20 Verhalten von `bool()`.

```
>>> bool(fundort)
```

```
True
>>> bool(coin_1)
True
>>> bool('')
False
>>> bool(0)
False
>>> bool(1)
True
>>> bool(0.0)
False
```

Wir sehen, dass `bool()` für Werte oder Variablen zu `True` evaluiert, sofern es sich nicht um 0, 0.0 oder einen leeren String handelt.

2.3.5 Video zu Datentypen

 Schaue dir jetzt das [Video 2.3.5](#) zu den Datentypen an und bearbeite dann das Quiz in OLAT.

2.4 Quiz zu Variablen und Datentypen

☒ Löse das [Quiz auf OLAT](#) und bearbeite dann das nächste Kapitel.

Datenstrukturen und Loops

Ordnung ist das halbe Leben, heisst es; und wenn man das Glück des Ordentlichen und die traurige Lage manches Unordentlichen ansieht, so dürfte man wohl sagen, Ordnung ist das ganze Leben.

Johann Gottfried Seume (1826)

Lernziele Kapitel 3: Datenstrukturen und Loops

- 1. Verstehen** Die Studierenden können die Konzepte von *List*, *Dictionary*, *Set* und *Tupel* in Python erklären und die grundlegenden Unterschiede (z. B. geordnete vs. ungeordnete Sammlung, Duplikate) benennen.
- 2. Analysieren** Die Studierenden können entscheiden, welche Datenstruktur (*List*, *Dictionary*, *Set*, *Tupel*) für gegebene historische Fragestellungen (z. B. Sammlung aller Fundorte vs. Erhalt aller Vorkommen) angemessen ist.
- 3. Anwenden** Die Studierenden können mit `for`- und `while`-Schleifen über Datenstrukturen iterieren, um historische Datensätze (z. B. Liste von Jahreszahlen oder Set von Ortsnamen) systematisch zu verarbeiten.
- 4. Bewerten** Die Studierenden können die Laufzeit- und Speicherimplikationen von *List*-Operationen gegenüber *Set*-Operationen einschätzen und begründen, wann welche Struktur effizienter ist.
- 5. Erzeugen** Die Studierenden können ein kleines AnalyseProgramm entwerfen, das mit Hilfe von *Lists*, *Dictionary*, *Sets* und `for`-Schleifen historische Daten konsolidiert (z. B. Duplikate entfernt, Häufigkeiten gezählt) und die Ergebnisse ausgibt.



Abbildung 3.1 Königsliste aus dem Grab von Sethos I. in Auftrag gegebenen Tempel in Abydos (Quelle: [WikimediaCommons](#) (2025)).

3.1 Ordnung, Ordnung, Ordnung ...

“Ordnung ist das halbe Leben” fängt der deutsche Schriftsteller und Dichter Johann Gottfried Seume (1763–1810) sein Werk “Von der Ordnung” an, nur um im gleichen Satz noch zu korrigieren, dass Ordnung wohl doch das ganze Leben sei. Für diesen Kurs ist die Ordnung auf zwei Ebenen zentral: Auf der Ebene des Programm-Codes—also für Python—gibt die Ordnung die Ausführung vor: Was ist mit welchem Objekt wann und wie zu tun? Auf der Ebene der Geschichtsforschung spielt die Ordnung eine Rolle bei der Einordnung der Ereignisse. Schon Platon hat das erkannt, denn in seinen *Nomoi* heisst es: “Alles nämlich, was im Staate nach Ordnung und Gesetz geschieht, bewirkt jegliches Gute; das meiste Ordnungswidrige und schlecht Angeordnete dagegen hebt wieder anderes, was wohl angeordnet war, auf.” Ordnung bringt also Gutes, Unordnung Chaos. Und so wäre für uns wohl auch die Geschichte umso schwerer verständlich, wenn sie Geschichtsforschende nicht “in Ordnung” bringen würde.

Diese Ordnung wurde beispielsweise schon im Altertum gepflegt. So sind uns aus dem alten Ägypten die Pharaonenlisten in einem von Sethos I. in Auftrag gegebenen Tempel erhalten (siehe Abbildung 3.1. Sie beinhaltet eine Genealogie von der 1. bis zur 18. Dynastie (mit einigen Lücken und den nicht abgedeckten Zwischenzeiten).

Ein weiteres Beispiel einer Liste finden wir in den Siegerlisten der Olympischen Spiele. Als Illustration sehen wir hier einen Papyrus aus dem 5. Jhdt. v. Chr., der für zwei Zeiträume die Gewinner aus verschiedenen Disziplinen beinhaltet. In der chronologischen Aneinanderreihung von Namen haben uns die Autoren der Pharaonenliste und dieses Papyrus also eine gewisse Ordnung hinterlassen.

Python bietet einige Möglichkeiten, eine solche Ordnung herzustellen. Die in diesem Kapitel vorgestellten Datenstrukturen zeigen, wie Informationen “ordentlich” abgespeichert werden, damit man sie später wieder aufrufen und abfragen kann. Da man Informationen oft seriell verarbeitet, kommt nun auch das Konzept der **q Loops** (dt. *Schleifen*) ins Spiel. Diese Kontrollmechanismen erlauben das Iterieren durch Datenstrukturen. Zusammen mit der Entscheidungslogik aus dem Kapitel 5 haben wir dann bereits das Grundgerüst beisammen, das es uns erlaubt, mächtige Programme zu schreiben.

3.2 Listen

- q** Die wohl geordnetste Datenstruktur in Python ist die [Liste](#) . In einer Liste können jegliche Datentypen abgespeichert werden. So kann eine Liste zum Beispiel aus



Abbildung 3.2 Papyrus Oxyrynchus 222. Er beinhaltet eine Liste der Sieger der Olympischen Spiele von 480 v. Chr. bis 468 v. Chr. und 456 v. Chr. bis 448 v. Chr. (Quelle: WikimediaCommons (2024))

Coin No	Weight	Copy	Denomination	Issuer	Date from	BCorAD1	Date to	BCorAD2	Reverse type
4	6.5	Copy	Dupondius	Antoninus Pius	138	AD	161	AD	Uncertain
1	7.86	As		Nero	65	AD	65	AD	PONTIF MAX TR POT IMP P P SC Apollo
2	7.69		Dupondius	Nero	67	AD	67	AD	SECVRITAS AVGVSTI SC
2	9.25		Dupondius	Tiberius Caesar (Augustus)	9	AD	14	AD	ROM ET AVG
4	6.49	Copy	As	Agrippa (Gaius)	43	AD	64	AD	Neptune SC
5	3.78	Copy	As	Agrippa (Gaius)	43	AD	64	AD	Neptune SC
6	5.41	Copy	Dupondius	Claudius I	43	AD	64	AD	CERES AVGVSTA SC
7	10	Copy	Dupondius	Antonia (Claudius)	43	AD	64	AD	TI CLAVDIVS CAESAR AVG P M TR P IMP SC
8	4.05	Copy	Dupondius	Antonia (Claudius)	43	AD	64	AD	TI CLAVDIVS CAESAR AVG P M TR P IMP SC
9	8.48	As		Claudius I	41	AD	50	AD	LIBERTAS AVGVSTA SC
11	6.82	Copy	As	Claudius I	43	AD	64	AD	Minerva SC
12	6.5	Copy	As	Claudius I	43	AD	64	AD	Minerva SC

Tabelle 3.1 Auszug aus dem Münzdatenset der University of Wales mit einigen ausgelassenen Spalten.

Strings bestehen oder wiederum aus Listen selbst. Es lassen sich so beliebig komplexe (auch verschachtelte) Datenstrukturen bauen. Wir wollen weiterhin mit dem Datensatz der Münzen arbeiten, den wir in Kapitel 2 eingeführt haben. Die Tabelle 3.1 zeigt einen Auszug aus dem Datenset.

Wir wollen uns hier die Gewichte genauer ansehen. Wir können die tabellarische Information ganz einfach als Liste wiedergeben, wie Code-Beispiel 3.1 zeigt:

Code-Beispiel 3.1 Beispiel zur Abbildung der Münzgewichte als Liste

```
>>> weights = [6.5, 7.86, 7.69, 9.25, 6.49, 3.78, 5.41, 10, 4.05, 8.48, 6.82,
```

6.5]

Wir sehen, dass wir eine Liste zwischen eckigen Klammern [...] definieren. Die Elemente werden durch Kommata abgetrennt. Eine Liste muss nicht zwingend aus identischen Datentypen bestehen. Wir können zum Beispiel die Informationen zu einer Münze—nehmen wir jene mit der “Coin No” 4—in einer Liste abspeichern (Code-Beispiel 3.2):

Code-Beispiel 3.2 Beispiel zur Abbildung der Münzgewichte als Liste

```
>>> coin_4 = [4, 6.49, "As", "Agrippa (Gaius)", 43, "AD", 64, "AD", "ROM ET  
AVG"]
```

Die Variable `coin_4` enthält somit alle wichtigen Informationen zu einer Münze in einer geordneten Liste. Als Nächstes kann man sich fragen, wie man auf einzelne Elemente in einer Liste zugreifen kann. Listen sind in Python—aber auch in anderen Programmiersprachen—*indexiert*. Das heisst konkret, dass jedes Element in einer

q Liste einen **Index** erhält. Das bedeutet gleichzeitig, dass es sich bei Listen um geordnete Datenstrukturen handelt (im Gegensatz zu Dictionaries und Sets (siehe unten)). In Python beginnt der Index bei 0. Wollen wir also die “Coin No” unseres `coin_4` ausgeben lassen, müssen wir das erste Element in dieser Variable mit dem Index 0 ansteuern. Wollen wir das “Weight” in Erfahrung bringen, brauchen wir den Index 1 und für den “Issuer” den Index 3. Dabei wird der Index in eckige Klammern [...] direkt an die Variable angefügt. Das Code-Beispiel 3.3 verdeutlicht dies:

Code-Beispiel 3.3 Beispiel zum Listenzugriff mittels Index

```
>>> print(coin_4[0])  
4  
>>> print(coin_4[1])  
6.49  
>>> print(coin_4[3])  
Agrippa (Gaius)
```

Es gibt noch weitere Zugriffsmöglichkeiten: Beispielsweise lässt sich mit der Angabe eines Bereichs, den wir mit einem Doppelpunkt abtrennen, die Liste “slicen”. Ein solcher Slice beinhaltet dann alle Elemente vom Startpunkt bis zum Endpunkt (ACHTUNG: ohne das Element, das durch den Endpunkt angegeben ist). Das Code-Beispiel 3.4 zeigt einen Slice vom “Weight” bis zum “Reverse Type”:

Code-Beispiel 3.4 Beispiel für einen Slice

```
>>> print(coin_4[1:8])  
[6.49, 'As', 'Agrippa (Gaius)', 43, 'AD', 64, 'AD']
```

Wir sehen, dass die Elemente 1 bis (aber ohne) 8 als neue, verkürzte Liste zurückgegeben werden.

Wie für Strings gibt es im Übrigen die Möglichkeit, mit der Built-In-Methode `len()` die Anzahl der Elemente in einer Liste zu bestimmen. Für den `coin_4` und seinen Slice aus dem Code-Beispiel 3.4 sieht das so aus:

Code-Beispiel 3.5 Beispiele für das Bestimmen der Länge von Listen

```
>>> print(len(coin_4))
```

```
9
>>> print(len(coin_4[1:8]))
7
```

Die Indexierung der Liste erlaubt es uns, Elemente in der Liste anzupassen. Wir können Werte gegeneinander austauschen, entfernen oder neue hinzufügen. Das Austauschen geschieht mit einer Substituierung, das Entfernen mit der Built-In-Funktion `pop()` und das Hinzufügen mit der Built-In-Funktion `append()`. Hier folgt eine Illustration dieser Prozesse:

Code-Beispiel 3.6 Beispiele für das Manipulieren von Listen

```
>>> coin_4[1] = 5.3
>>> print(coin_4)
[4, 5.3, 'As', 'Agrippa (Gaius)', 43, 'AD', 64, 'AD', 'ROM ET AVG']
>>> coin_4.pop(0)
4
>>> print(coin_4)
[5.3, 'As', 'Agrippa (Gaius)', 43, 'AD', 64, 'AD', 'ROM ET AVG']
>>> coin_4.append(4)
>>> print(coin_4)
[5.3, 'As', 'Agrippa (Gaius)', 43, 'AD', 64, 'AD', 'ROM ET AVG', 4]
```

Das Code-Beispiel 3.6 macht deutlich, wie wir Listen manipulieren können. Als Erstes könnten wir zum Beispiel feststellen, dass wir die Münze falsch gewogen haben. Das tatsächliche Gewicht beträgt 5.3 Gramm. Da das Gewicht für die Variable `coin_4` auf Position 1 liegt, definieren wir diesen Wert mittels `=` neu (ganz ähnlich wie wenn wir eine Variable definieren würden).

Als Nächstes könnten wir uns sagen, dass wir die Information der “Coin No” gar nicht brauchen. Wir sagen mit dem Argument, das wir zwischen den Klammern von `pop()` definieren, welches Element an welchem Index wir entfernt haben wollen. Dieses Element wird zurückgeliefert, damit wir wissen, was entfernt worden ist. Die veränderte Liste ist jetzt also so modifiziert, dass sie um ein Element kürzer ist. Sollten wir merken, dass wir die Information trotzdem noch gebrauchen könnten, lässt sich mit der `append()`-Funktion das Element wieder an das Ende der Liste hinzufügen (`append()` ergänzt die Liste immer vom Ende her).

Listen können auch verschachtelt werden. Wollen wir zum Beispiel die Münzen aus der Tabelle 2.1 in einer Liste abbilden, könnten wir das so tun:

Code-Beispiel 3.7 Beispiele für verschachtelte Listen. Wir nehmen hier wieder die ursprünglichen Werte von `coin_4` an.

```
>>> coin_4 = [4, 6.49, "As", "Agrippa (Gaius)", 43, "AD", 64, "AD", "ROM ET AVG"]
>>> coin_6 = [6, 5.41, "Dupondius", "Claudius I", 43, "AD", 64, "AD", "CERES AVGVSTA SC"]
>>> coin_list = [coin_4, coin_6]
>>> print(coin_list)
[[5.3, 'As', 'Agrippa (Gaius)', 43, 'AD', 64, 'AD', 'ROM ET AVG', 4], [6, 5.41, 'Dupondius', 'Claudius I', 43, 'AD', 64, 'AD', 'CERES AVGVSTA SC']]
>>> print(coin_list[1])
```

```
[6, 5.41, "Dupondius", "Claudius I", 43, "AD", 64, "AD", "CERES AVGVSTA SC"]
>>> print(len(coin_list))
2
```

Im Code-Beispiel 3.7 sehen wir, wie zwei separate Listen, von welchen jede eine andere Münze beschreibt, in einer Liste vereint werden. Diese hat dann die Länge 2.

Es wird aber auch schnell klar, dass Listen unübersichtlich werden können, wenn jedes Feld etwas anderes bedeutet. Zudem kann es passieren, dass man bei der Konstruktion der Liste mal zwei Felder vertauscht. Somit würden wir nicht in jeder Liste die gleiche Information am gleichen Index finden, was die automatische Analyse der Daten erschwert oder gar verunmöglicht. Wir schauen uns deshalb nun eine Datenstruktur an, welche sich besser kontrollieren lässt.

3.2.1 Video zu Listen

 Schaue dir jetzt das [Video 3.2.1](#) zu den Listen an und bearbeite dann das nächste Unterkapitel.

3.3 Dictionaries

- q Ein **Dictionary** (dt. *Wörterbuch*) ist ab Python 3.7 ebenfalls eine geordnete Datenstruktur. Die Reihenfolge, wie Einträge in einem Dictionary abgelegt werden, bleibt also dieselbe. Jedoch können wir nicht direkt mit Indices auf die Elemente in einem Dictionary zugreifen. In einem Dictionary geschieht der Zugriff mit einem
- q sogenannten **Key** (Schlüssel). Ist der Key, der entweder aus einem String oder einer Integer besteht, im Dictionary vorhanden, dann wird der entsprechende **Value**
- q (Wert) zurückgeliefert. Der Value kann sämtliche Datentypen umfassen, auch wieder aus Dictionaries selbst. Keys müssen innerhalb des Dictionaries einzigartig sein. Speichert man zweimal hintereinander Values mit dem gleichen Key ab, wird der vorherige Wert überschrieben.

Ein Dictionary besteht also aus Key-Value-Paaren. Wir definieren Dictionaries zwischen geschweiften Klammern `{...}`. Da wir im vorigen Abschnitt 3.2 bereits zwei Münzen definiert haben, können wir diese nun einfach in ein Dictionary speichern:

Code-Beispiel 3.8 Beispiel für das Erstellen eines Dictionaries.

```
>>> coin_dataset = {"coin_4": coin_4, "coin_6": coin_6}
>>> len(coin_dataset)
2
>>> coin_dataset["coin_6"]
[6, 5.41, 'Dupondius', 'Claudius I', 43, 'AD', 64, 'AD', 'CERES AVGVSTA SC']
>>> len(coin_dataset["coin_6"])
9
```

Wir setzen das Dictionary also zusammen, indem wir für jeden Eintrag einen Key definieren. Hier könnten wir beispielsweise den Variablennamen nehmen, der

aus `coin_` und der “Coin No” besteht.¹ Nach dem Doppelpunkt folgt dann der Wert. Hier sind das die bereits bestehenden Münzeinträge in Form von Listen. Wir haben unserem `coin_dataset` zwei Münzen hinzugefügt. Die Bestimmung der Länge des Dictionaries bestätigt dies. Wenn wir nun gezielt auf den `coin_6` zugreifen wollen, machen wir dies ebenfalls mit eckigen Klammern (wie für die Indices bei Listen), verwenden aber anstatt eines Indexes den Key. Das bewirkt die Wiedergabe des entsprechenden Wertes, der dann die gewöhnliche Länge 9 hat. Beachte den Vorteil: Speichern wir die Münzen in Listen ab, müssen wir genau wissen, an welcher Position welche Münze abgespeichert ist. Wenn wir ein Dictionary verwenden, dann können wir direkt auf eine Münze zugreifen, indem wir sie mit dem richtigen Key ansteuern.

Wir könnten die Datenstruktur für die Münze noch zusätzlich verbessern. Das sei hier am `coin_6` illustriert:

Code-Beispiel 3.9 Beispiel für das Erstellen eines Dictionaries.

```
>>> coin_dataset_new = {"coin_6": {"coin_no": 6, "weight": 5.41, "denomination": "Dupondius", "issuer": "Claudius I", "date_from": 43, "bcorad1": "AD", "date_to": 64, "bcorad2": "AD", "reverse_type": "CERES AVGVSTA SC"}}
>>> len(coin_dataset_new)
1
>>> len(coin_dataset_new["coin_6"])
9
>>> coin_dataset_new["coin_6"]["weight"]
5.41
```

Das Code-Beispiel 3.9 zeigt, wie als Value von `coin_6` ein neues Dictionary angelegt wird. Hierin werden wiederum neue Key-Value-Paare definiert, wobei solche Paare durch Kommata getrennt werden. Wir sehen, dass dieses neue Datenset im Moment nur ein Item umfasst. Die Länge des Datensatzes mit dem Key `coin_6` bleibt aber bei 9 (wie bei der Liste). Hingegen können wir direkt auf das Gewicht zugreifen, indem wir `coin_dataset_new` zuerst sagen, dass wir den Datensatz zum `coin_6` benötigen, und anschliessend mit dem Key `weight` auf das Gewicht zugreifen werden. Der erste Key sperrt in diesem Sinne den ersten Datensatz auf und der Zweite die Details.

Dictionaries sind beliebig erweiterbar. Beispielsweise können wir nun auch noch den `coin_4` zu `coin_dataset_new` hinzufügen:

Code-Beispiel 3.10 Beispiel für die Erweiterung eines Dictionaries.

```
>>> coin_dataset_new["coin_4"] = {"coin_no": 4, "weight": 6.49, "denomination": "As", "issuer": "Agrippa (Gaius)", "date_from": 43, "bcorad1": "AD", "date_to": 64, "bcorad2": "AD", "reverse_type": "Neptuen SC"}
>>> print(len(coin_dataset_new))
2
>>> print(coin_dataset_new)
{'coin_6': {'coin_no': 4, 'weight': 5.41, 'denomination': 'Dupondius', 'issuer': 'Claudius I', 'date_from': 43, 'bcorad1': 'AD', 'date_to': 64, 'bcorad2': 'AD', 'reverse_type': 'CERES AVGVSTA SC'}, 'coin_4': {'coin_no': 4, 'weight':
```

¹Beachte: Wir finden in der Tabelle 2.1 gewisse “Coin No” doppelt. Da Keys in Dictionaries einzigartig sein müssen, eignet sich die “Coin No” im realen Leben wohl nicht. Für die Anschauungszwecke hier reicht das aber.

```
6.49, 'denomination': 'As', 'issuer': 'Agrippa (Gaius)', 'date_from': 43, '
bcorad1': 'AD', 'date_to': 64, 'bcorad2': 'AD', 'reverse_type': 'Neptuen SC']}]}
```

Die Länge des neuen Dictionaries ist jetzt 2 und beinhaltet nun auch den `coin_4` mit denselben Keys wie `coin_6`. So haben wir das Münzdatenset schon viel besser strukturiert.

Zwei wichtige Built-In-Funktionen der Dictionary-Klasse sind `keys()` und `values()`. Sie zeigen uns die Keys, respektive die Values, eines Dictionaries in der Form einer Liste an:

Code-Beispiel 3.11 Beispiel für die Erweiterung eines Dictionaries.

```
>>> print(coin_dataset_new.keys())
dict_keys(['coin_6', 'coin_4'])
>>> print(coin_dataset_new.values())
dict_values([{'coin_no': 4, 'weight': 5.41, 'denomination': 'Dupondius', '
issuer': 'Claudius I', 'date_from': 43, 'bcorad1': 'AD', 'date_to': 64, '
bcorad2': 'AD', 'reverse_type': 'CERES AVGVSTA SC'}, {'coin_no': 4, 'weight':
6.49, 'denomination': 'As', 'issuer': 'Agrippa (Gaius)', 'date_from': 43, '
bcorad1': 'AD', 'date_to': 64, 'bcorad2': 'AD', 'reverse_type': 'Neptuen SC'
}])
>>> print(coin_dataset_new["coin_6"].keys())
dict_keys(['coin_no', 'weight', 'denomination', 'issuer', 'date_from', '
bcorad1', 'date_to', 'bcorad2', 'reverse_type'])
>>> print(coin_dataset_new["coin_6"].values())
dict_values([4, 5.41, 'Dupondius', 'Claudius I', 43, 'AD', 64, 'AD', 'CERES
AVGVSTA SC'])
```

Wir bemerken, dass im Code-Beispiel 3.12 der letzte Aufruf `print(coin_dataset_new["coin_6"].values())` die ursprüngliche Liste wiedergibt. Das Extrahieren von Keys und Values wird noch wichtig, wenn wir uns den Loops zuwenden (siehe Abschnitt 3.7).

3.3.1 Video zu Dictionaries

 Schaue dir jetzt das [Video 3.3.1](#) zu den Dictionaries an und bearbeite dann das nächste Unterkapitel.

3.4 Sets

- q Ein **Set** ist im Gegensatz zu Listen und Dictionaries ungeordnet. Das heisst, wir können uns nicht auf die Reihenfolge verlassen. Dementsprechend sind Sets ungeeignet, wenn die Position in einer Datenstruktur eine Aussage über das Objekt (wie zum Beispiel das Gewicht) macht. Ausserdem sind Sets nicht indexiert. Das Einzige, was sie mit Listen und Dictionaries gemein haben, ist ihre Veränderbarkeit.

Nichtsdestotrotz können sie sich als nützlich erweisen. Wenn wir beispielsweise eine Liste von Fundorten von Münzen haben und wissen möchten, wie viele einzigartige Fundorte es gibt, können wir diesen Wert mittels der Built-In-Funktion `set()` schnell ermitteln:

Code-Beispiel 3.12 Beispiel für die Erweiterung eines Dictionaries.

```
>>> places = ["Augst", "Aventicum", "Augst", "Vindonissa"]
>>> print(len(places))
4
>>> unique = set(places)
>>> print(unique)
{'Augst', 'Aventicum', 'Vindonissa'}
>>> print(len(unique))
3
```

Die unter der Variable `places` abgespeicherte Liste wird mittels `set()` in ein Set umgewandelt. Hierbei werden doppelte Einträge entfernt, weswegen nur die einzigartigen Items übrig bleiben.

3.4.1 Set-Operationen

In Python sind die aus der Mengenlehre bekannten Set-Operationen—Vereinigung, Schnittmenge, und Differenz—möglich (siehe Abbildung 3.3).

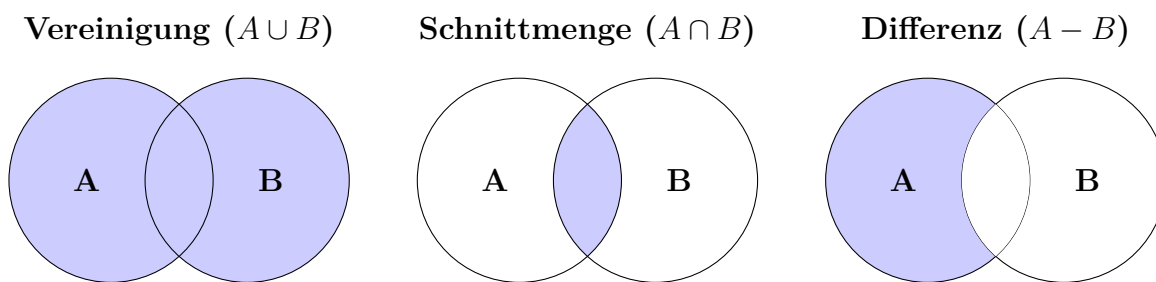


Abbildung 3.3 Visualisierung der Set-Operationen in Python

Das Code-Beispiel 3.13 zeigt, wie Vereinigung, Schnittmenge und Differenz in Python definiert sind:

Code-Beispiel 3.13 Beispiel für Set-Operationen Vereinigung, Schnittmenge und Differenz.

```
>>> dig_2022 = {"Augst", "Vindonissa", "Aventicum", "Curia"}
>>> dig_2023 = {"Aventicum", "Colonia Agrippina", "Vindonissa", "Turicum"}
>>> all_locations = dig_2022.union(dig_2023)
>>> print("Alle Fundorte:", all_locations)
Alle Fundorte: {'Augst', 'Aventicum', 'Colonia Agrippina', 'Curia', 'Vindonissa', 'Turicum'}
>>> locations_in_common = dig_2022.intersection(dig_2023)
>>> print("Gemeinsame Fundorte:", locations_in_common)
Gemeinsame Fundorte: {'Aventicum', 'Vindonissa'}
>>> only_2022 = dig_2022.difference(dig_2023)
>>> print("Nur 2022:", only_2022)
Nur 2022: {'Augst', 'Curia'}
```

Die Set-Operationen im Code-Beispiel 3.13 verdeutlichen, wie die verschiedenen Mengen aus Abbildung 3.3 gebildet werden. Bei jeder Operation fungiert jeweils eine Variable als Objekt und die andere als Argument. Die Reihenfolge spielt

hierbei keine Rolle, mit Ausnahme der Berechnung der Differenz. Hätten wir beispielsweise `dig_2023.difference(dig_2022)` definiert, so erhielten wir die Ausgrabungsorte, an welchen nur im Jahr 2023 gegraben wurde: `{'Colonia Agrippina', 'Turicum'}`.

Auch zu Sets kann man Elemente hinzufügen. Jedoch funktioniert die Built-In-Methode `append()` nur für Listen. Für Sets benutzen wir die Methode `add()`, wie das Code-Beispiel 3.14 zeigt:

Code-Beispiel 3.14 Beispiel für Set-Operationen Vereinigung, Schnittmenge und Differenz.

```
>>> dig_2022.add("Colonia Agrippina")
>>> print(dig_2022)
{'Augst', 'Aventicum', 'Colonia Agrippina', 'Curia', 'Vindonissa'}
>>> dig_2022.add("Augst")
>>> print(dig_2022)
{'Augst', 'Aventicum', 'Colonia Agrippina', 'Curia', 'Vindonissa'}
```

Wir sehen, dass das Hinzufügen von “Colonia Agrippina” das Set um genau dieses Element erweitert. Jedoch wird es nicht wie bei einer Liste an das Ende des Sets (das es so *per se* gar nicht gibt) hinzugefügt, sondern taucht einfach irgendwo im Set auf. Dass das Set hier alphabetisch geordnet ist, ist purer Zufall. Ein weiteres Hinzufügen einer Grabungsstätte mit `dig_2022.add('Pergamon')` resultiert in folgender Ausgabe: `{'Augst', 'Aventicum', 'Pergamon', 'Colonia Agrippina', 'Curia', 'Vindonissa'}`. Dies verdeutlicht noch einmal, dass Sets nicht geordnet sind.

3.4.2 Video zu Sets

 Schaue dir jetzt das [Video 3.4.2](#) zu den Sets an und bearbeite dann das nächste Unterkapitel.

3.5 Tupel

Von einer ungeordneten Datenstruktur gehen wir zurück zu einer geordneten: dem **Tupel**. Ein Tupel ist jedoch, im Vergleich zu Listen, Dictionaries und Sets, unveränderbar. Wir können also nicht einfach Elemente entfernen oder hinzufügen oder diese abändern. Statt in einer Liste oder in einem Dictionary könnten wir zum Beispiel die Werte des `coin_6` einem Tupel erfassen. Wir benutzen für die Konstruktion von Tupeln die runden Klammern `(...)` und verwenden für die Trennung von Elementen Kommata:

Code-Beispiel 3.15 Beispiel für Erstellung eines Tupels.

```
>>> coin_6_tupel = (6, 5.41, "Dupondius", "Claudius I", 43, "AD", 64, "AD", "CERES AVGVSTA SC")
>>> print(coin_6_tupel[1])
5.41
```

Das Code-Beispiel 3.15 veranschaulicht, wie ein Tupel erzeugt wird. Die Reihenfolge der Elemente sowie deren Werte sind unveränderbar. Tupel sind indexiert. Gleich wie bei Listen können wir so spezifische Elemente ansteuern. Ebenfalls gleich verhält sich das Tupel bezüglich der Elemente, die es aufnehmen kann: Es kann selbst wiederum alle Datentypen enthalten.

3.5.1 Video zu Tupel

 Schaue dir jetzt das [Video 3.5.1](#) zu den Tupeln an und bearbeite dann das nächste Unterkapitel.

3.6 Kurze Zusammenfassung zu Datenstrukturen

Wir haben gesehen, dass wir Daten unterschiedlich repräsentieren können. Es gibt aber auch gerade “nur” vier Datenstrukturen, die man kennen muss, um in Python zu programmieren. Aus diesen Grundarten lassen sich dann beliebig komplexe Datenstrukturen bauen, um zum Beispiel historische Artefakte zu beschreiben. Wichtig ist, dass wir uns darüber Gedanken machen, was mit den Datenmodellen passieren soll. Von weiteren Verarbeitungsschritten hängt es nämlich ab, ob wir eine Münze nun als Dictionary oder als Liste von Listen repräsentieren. Tabelle 3.2 gibt nochmals einen Überblick über alle Datenstrukturen und deren Grundeigenschaften. Es empfiehlt sich, diese Tabelle bei der Erstellung von Strukturen vor Augen zu haben.

Datenstruktur	geordnet	veränderbar	indexiert
Liste	✓	✓	✓
Dictionary	✓	✓	✗
Set	✗	✓	✗
Tupel	✓	✗	✓

Tabelle 3.2 Eigenschaften zentraler Python-Datenstrukturen

3.7 Loops

Wenn wir systematisch über Datenstrukturen iterieren möchten, können wir dafür sogenannte **Loops** —zu Deutsch *Schleifen*—verwenden. Grundsätzlich können wir  über alle Datenstrukturen, die indexiert sind. Diese Eigenschaft liegt zuerst einmal Listen, Tupeln, aber auch den Strings (siehe Abschnitt 2.3.3) zugrunde.² Aber auch über Dictionaries und Sets kann man mit Loops iterieren, wenn man sie davor in eine indexierte Form bringt. Man kann beispielsweise die Built-In-Funktion `list()` verwenden, um aus einem Objekt, das keine Liste ist, eine solche zu erzeugen:

Code-Beispiel 3.16 Beispiel für die Umwandlung von Objekten in eine Liste.

```
>>> list(dig_2022)
```

²Strings sind wie Listen indexiert: Auch hier startet der Index bei 0. Wenn wir beispielsweise den dritten Buchstaben im String 'Augustus' ausgeben wollen, den wir in der Variablen `name` abgespeichert haben, würden wir das mit `print(name[2])` tun.

```
['Vindonissa', 'Curia', 'Aventicum', 'Augst']
>>> coin_6_tupel = (6, 5.41, "Dupondius", "Claudius I", 43, "AD", 64, "AD", "
CERES AVGVSTA SC")
[6, 5.41, 'Dupondius', 'Claudius I', 43, 'AD', 64, 'AD', 'CERES AVGVSTA SC']
>>> list(coin_6_tupel[2])
['D', 'u', 'p', 'o', 'n', 'd', 'i', 'u', 's']
```

Wie wir bereits in Abschnitt 3.5 aus einer Liste von Ortschaften ein Set gemacht haben, können wir mit `list()` aus einigen Objekten eine Liste erzeugen (siehe Code-Beispiel 3.16). Beachte, dass die Umwandlung eines Sets in eine Liste in einer zufälligen Anordnung der Elemente resultiert. Auch ein Tupel, das an sich schon indexiert ist, können wir in eine Liste umwandeln. Zuguterletzt lässt sich auch aus einem String eine Liste erzeugen. Hierbei wird jeder einzelne Buchstabe als einzelnes Element wiedergegeben. Wir wenden uns nun zuerst dem `for`-Loop zu.

3.7.1 `for`-Loops

Mittels eines `for`-Loops lässt sich einfach über eine Sequenz iterieren. Wir illustrieren das zuerst anhand des Strings, den wir schon länger kennen. Um einen `for`-Loop zu erzeugen, brauchen wir das **Keyword** `for`, das Python signalisiert, dass jetzt ein Loop startet, und `in`, das Python sagt, über welche Sequenz es iterieren soll. Der Doppelpunkt signalisiert, dass nun die Anweisung folgt, was bei der Iteration über die Sequenz mit jedem Element geschehen soll. Diese Aktion wird typischerweise auf einer neuen Zeile geschrieben, wobei diese entweder um vier Leerschläge oder einen Tabulator eingerückt ist.³

Code-Beispiel 3.17 Beispiel für einen `for`-Loop über einen String.

```
>>> for character in coin_6_tupel[2]:
...     print(character)
...
D
u
p
o
n
d
i
u
s
```

Wir sehen im Code-Beispiel 3.17, wie wir aus einem Tupel zuerst den String zur “Denomination” extrahieren und dann über diesen iterieren. Nach dem `for`-Statement folgt die Variable, mit welcher wir die Elemente, über die wir iterieren, ansprechen (in diesem Fall `character`). Eingerückt und auf einer neuen Zeile finden wir dann das Statement, das für jedes Element angewendet werden soll.⁴ In diesem

³Welche Einrückungsmethode man verwendet, ist egal. Jedoch ist es wichtig, dass man über den gesamten Programm-Code hinweg konstant bleibt.

⁴Die drei Punkte sind zu ignorieren. Diese treten nur in Erscheinung, wenn man Python über das Terminal bedient.

Fall möchten wir lediglich jeden Buchstaben in der Denomination “Dupondius” auf die Konsole ausgeben. Das heisst, dass die Variable `character` bei jeder Iteration einen anderen Wert (zuerst D, dann u, usw.) annimmt, bis die Sequenz zu Ende ist. Insofern behält die Variable `character` nach dem Durchiterieren den Wert `s`.⁵ Die Variable (und darum auch der Wert) ist weiterhin im lokalen Namensraum verfügbar (siehe Abschnitte 2.2.2 und 2.2.3).

Wir wollen uns noch weitere `for`-Loops anschauen, die über Listen, Sets, und Tupel iterieren. Die Loops im Code-Beispiel 3.18 illustrieren anhand der schon definierten Objekte die Funktionsweise.

Code-Beispiel 3.18 Beispiel für `for`-Loops über eine Liste, ein Set, und ein Tupel.

```
>>> for item in coin_6:
...     print(f"Diese Position hat den Wert: {item}")
...
Diese Position hat den Wert: 6
Diese Position hat den Wert: 5.41
Diese Position hat den Wert: Dupondius
Diese Position hat den Wert: Claudius I
Diese Position hat den Wert: 43
Diese Position hat den Wert: AD
Diese Position hat den Wert: 64
Diese Position hat den Wert: AD
Diese Position hat den Wert: CERES AVGVSTA SC
>>> for place in list(dig_2022):
...     print(f"Eine Ausgrabung fand in {place} statt")
...
Eine Ausgrabung fand in Vindonissa statt.
Eine Ausgrabung fand in Curia statt.
Eine Ausgrabung fand in Aventicum statt.
Eine Ausgrabung fand in Augst statt.
>>> for item in coin_6_tupel:
...     print(type(item))
...
<class 'int'>
<class 'float'>
<class 'str'>
<class 'str'>
<class 'int'>
<class 'str'>
<class 'int'>
<class 'str'>
<class 'str'>
```

Wir sehen, dass wir beim Iterieren über die Liste nichts weiter tun müssen, als der Loop-Syntax zu folgen. So haben wir die Möglichkeit, die Werte in der Liste in einem formatierten String (siehe Abschnitt 2.3.3 und insbesondere Code-Beispiel 2.17) wiederzugeben. Um das Gleiche mit einem Set zu machen—wir nehmen eines zu Grabungsorten—, müssen wir das Set zuerst in eine Liste umwandeln (und dabei

⁵Das ist leicht mit einem weiteren `print(character)` zu überprüfen.

die Zufälligkeit beachten). Wenn uns die Datentypen der Werte in einem Tupel interessieren, können wir an jeder Position mit der Built-In-Funktion `type()` den Datentyp ausgeben lassen.

Um über Dictionaries zu iterieren, bedarf es eines kleinen Tricks. Hierbei rufen wir die `items()`-Methode eines Dictionaries auf. Diese Funktion wandelt das Dictionary in eine Liste um, in welcher die Key-Value-Paare als Tupel gespeichert sind. Wir haben bereits erörtert, dass Listen und Tupel indexiert sind. Somit können wir über das Dictionary iterieren. Speziell hierbei ist jedoch, dass eine Variable nicht ausreicht. Wir allozieren zwei Variablen, wovon die erste für den Schlüssel steht und die zweite für den Wert:

Code-Beispiel 3.19 Beispiel für Iteration über ein Dictionary.

```
>>> print(coin_dataset_new.items())
dict_items([('coin_6', {'coin_no': 6, 'weight': 5.41, 'denomination': 'Dupondius', 'issuer': 'Claudius I', 'date_from': 43, 'bcorad1': 'AD', 'date_to': 64, 'bcorad2': 'AD', 'reverse_type': 'CERES AVGVSTA SC'})])
>>> for key, value in coin_dataset_new.items():
...     print(key)
...     print(value)
...     <class 'dict'>
...
coin_6
{'coin_no': 6, 'weight': 5.41, 'denomination': 'Dupondius', 'issuer': 'Claudius I', 'date_from': 43, 'bcorad1': 'AD', 'date_to': 64, 'bcorad2': 'AD', 'reverse_type': 'CERES AVGVSTA SC'}
```

Wir sehen im Code-Beispiel 3.19 als erstes, was `items()` bewirkt: Der Inhalt des Dictionaries wird in einem `dict_items`-Objekt gespeichert, das aus einer Liste aus Tupeln (hier nur einem Tupel) besteht. Der Variablen-Namen `key` erhält hier beim Iterieren über die Liste von Tupeln immer das erste Element im Tupel (in diesem Fall `coin_6`) und `value` wird das zweite Element (in diesem Fall das Dictionary) zugewiesen. Das Herausgeben von `key` und `value` bestätigt dies. Wie greifen wir nun aber auf die Werte der Münze zu? Hierfür müssen wir in einem verschachtelten Dictionary die `items()`-Methode zweimal anwenden, wie das im Code-Beispiel 3.20 ersichtlich wird:

Code-Beispiel 3.20 Beispiel für Iteration über ein verschachteltes Dictionary.

```
for key, value in coin_dataset_new.items():
...     for item, item_value in value.items():
...         print(f"Der Wert '{item}' ist {item_value}.")
...
Der Wert 'coin_no' ist 6.
Der Wert 'weight' ist 5.41.
Der Wert 'denomination' ist Dupondius.
Der Wert 'issuer' ist Claudius I.
Der Wert 'date_from' ist 43.
Der Wert 'bcorad1' ist AD.
Der Wert 'date_to' ist 64.
Der Wert 'bcorad2' ist AD.
```

```
Der Wert 'reverse_type' ist CERES AVGVSTA SC.
```

Wir haben eine zusätzliche `for`-Schleife, die über das innere Dictionary iteriert. Dieses sprechen wir mit `value` an (es ginge aber auch sonst eine anderslautende Variable). Da dieses innere Dictionary durch die `items()`-Methode wiederum in eine Liste von Key-Value-Paaren umgewandelt wird, brauchen wir hier zwei weitere Variablen. Wichtig ist dabei, dass diese nicht gleich lauten, wie jene im ersten `for`-Statement.

3.7.2 while-Loops

`for`-Loops iterieren in der Regel über die gesamte Sequenz, die auf `in` folgt. Ein `while`-Loop folgt einer komplett anderen Logik: Ein solcher Loop iteriert so lange über eine Sequenz, bis ein bestimmtes Kriterium erreicht ist. Kriterien definieren wir über entscheidungslogische Elemente, über welche Python verfügt. Darauf gehen wir in Kapitel 5 noch genauer ein. Für den Moment begnügen wir uns mit einem einfachen Variablenvergleich. Angenommen, wir möchten für unseren Münzdatensatz in Erfahrung Werte von exakt 10 Sesterzen—so wie sie im Datensatz erscheinen (also so wie sie “geordnet” sind)—extrahieren, dann könnte man diese mit einem `while`-Loop so machen, wie in Code-Beispiel 3.21 beschrieben:⁶

Code-Beispiel 3.21 Beispiel für einen `while`-Loop. Wir nehmen an, dass wir in einer Variablen `coins`, die wir als Liste definiert haben, sämtliche Münzen in separaten Dictionaries abgespeichert haben.

```
index = 0
sesterzen = []

while len(sesterzen) < 10:
...     if coins[index]['denomination'] == 'Sesterz':
...         sesterzen.append(coin_dataset[index])
...     index += 1
```

In diesem kleinen Beispiel passiert so einiges! Wir definieren zuerst eine Variable `[index]`, die uns hilft, durch das Münzdatenset, das ja in Form einer Liste vorliegt, durchzuiterieren. Als Nächstes definieren wir eine Variable, in welcher wir die ersten 10 Sesterzen speichern. Dann folgt die Initialisierung des `while`-Loops mit dem Keyword `while`. An diesen folgt direkt die Bedingung: `len(sesterzen) < 10`⁷ heisst, dass der Loop so lange über das Datenset iteriert, bis er 10 Sesterzen gefunden hat oder das Ende des Datensets erreicht hat.

Was folgt, ist eine weitere Bedingung, die dazu dient, den Wert der jeweiligen Münze herauszufinden. Wenn wir das erste Mal in die `while`-Schleife gehen, sind die Länge der Liste `sesterzen` und der Wert von `index` 0. Mit `coins[index]`—und bei der ersten Münze, die wir effektiv mit `coins[0]` ansprechen—fragen wir mit dem Key `'denomination'` nach dem Münzwert. Wenn dieser gleich dem String `'Sesterz'` ist, was wir mit den zwei Gleichheitszeichen `==` ausdrücken, dann wird mit `append()` die Liste um eben diesen Sesterzen erweitert.

⁶Beachte: Dieses Code-Beispiel kannst du nicht einfach mit einem der vorhergehenden Datenstrukturen bei dir ausführen.

⁷spricht: “Die Länge der Liste ‘Sesterzen’ ist kleiner als 10”

In jedem Fall setzen wir den `index` so lange mit `+=` um eins höher, bis die `len(sesterzen)` eben 10 entspricht, die Bedingung `len(sesterzen) < 10` intern den Wert `False` ausgeben würde. Das wäre hier der Fall, wenn die `len(sesterzen)` eben 10 entsprechen würde. Dann ist das Statement, das Python intern evaluiert—nämlich `10 < 10` falsch und der Loop wird beendet.

3.7.3 Video zu Loops

 Schaue dir jetzt das [Video 3.7.3](#) zu den Loops an und bearbeite dann das Quiz in OLAT.

3.8 Quiz zu Datenstrukturen und Loops

☒ Löse das [Quiz auf OLAT](#) und bearbeite dann das nächste Kapitel.

Reguläre Ausdrücke zur Textanalyse

Ein Muster besteht aus kleinsten zu isolierenden Einheiten, die gemäss der Wiederholungsvorschrift zu einem Ganzen, potentiell Unendlichen zusammengesetzt werden.

Kerstin Kraft (2001)

Lernziele Kapitel 4: Reguläre Ausdrücke zur Textanalyse

- 1. Verstehen** Die Studierenden können den Begriff der *regulären Ausdrücke* definieren und erläutern, wie sie zur Mustererkennung in Texten eingesetzt werden. Sie verstehen den Bezug zur **Automatentheorie** als theoretische Grundlage regulärer Sprachen.
- 2. Analysieren** Die Studierenden können unterschiedliche RegEx-Muster (z. B. Quantoren, Zeichenklassen, Anker) unterscheiden und deren Wirkung anhand historischer Textbeispiele erklären.
- 3. Anwenden** Die Studierenden können einfache und komplexe reguläre Ausdrücke in Python entwickeln, um historische Quellen (z. B. Datumsangaben, Personennamen oder Ortsangaben) systematisch zu extrahieren.
- 4. Bewerten** Die Studierenden können die Möglichkeiten und Grenzen von regulären Ausdrücken kritisch einschätzen, insbesondere im Hinblick auf **Ambiguität** und **Unschärfe** historischer Daten.
- 5. Erzeugen** Die Studierenden können eigene RegEx-basierte Suchroutinen entwerfen, die für konkrete Fragestellungen in der Geschichtsforschung (z. B. Extraktion von Münzdatierungen oder Fundorten) einsetzbar sind.

4.1 Reguläre Ausdrücke

- q **Reguläre Ausdrücke** (EN *regular expressions*, abgekürzt oft auch als *RegExp*, *regex*, oder *RegEx* zu finden) sind ein mächtiges Tool zur Verarbeitung von Strings. Immer wenn ihr in Dokumenten nach einem Wort sucht und es eventuell sogar durch ein anderes (oder mehrere Wörter) ersetzt, kommen im Hintergrund reguläre Ausdrücke zum Einsatz. Regulären Ausdrücken liegt die sogenannte **Automatentheorie** zugrunde.

Exkurs

4.1.1 Automatentheorie und reguläre Ausdrücke

Reguläre Ausdrücke basieren auf der Idee, dass Texte als Folge von Symbolen formalisiert werden können. Die Automatentheorie liefert dafür das Fundament. Endliche Automaten^a können genau diejenigen “Sprachen” erkennen, die durch reguläre Ausdrücke beschrieben werden. Für die Geschichtsforschung bedeutet dies, dass wir selbst komplexe Muster in Quellen formal modellieren und automatisiert auffinden können. Und gerade weil in der Geschichtsforschung Texte oftmals die Grundlage bilden, ist es von grossem Vorteil, wenn man versteht, wie Automaten und im Endeffekt dann eben auch die regulären Ausdrücke funktionieren. Im Folgenden werden die Basics für die Automatentheorie gelegt.

Zutaten

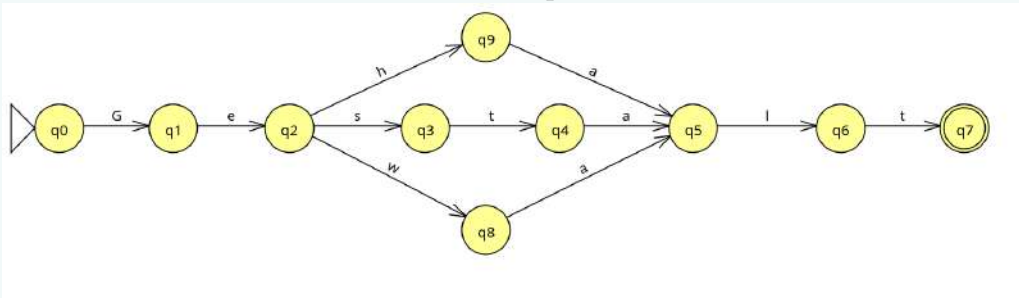
Für ein Verständnis der Automatentheorie müssen wir uns erst vergegenwärtigen, was die Funktion eines Automats ist: Wir können ihn als eine Art Maschine verstehen, die eine Folge von Zeichen einliest und entweder akzeptiert oder ablehnt. Wir sprechen dann von einem sogenannten *Akzeptor*. Daneben gibt es noch die *Transduktoren*, welche nicht nur Zeichen einlesen, sondern auch wieder ausgeben. Wir beschränken uns hier auf die Akzeptoren. Was brauchen wir nun also an Zutaten, um einen Automaten zu definieren?

Ein Automat besitzt

1. ein Alphabet Σ ,
2. eine endliche Menge an Zuständen Φ zwischen dem Startzustand und dem/den Endzustand/Endzuständen,
3. einen Startzustand S , wobei $S \in \Phi$,
4. einen oder mehrere Endzustände F , wobei $F \subseteq \Phi$,
5. und eine Menge an Übergängen δ zwischen den Zuständen, inklusive eines sogenannten “leeren Übergangs” ϵ oder auch λ . Die Übergänge sind als Funktion $\delta : \Phi \times \Sigma \rightarrow \Phi$

Ein Automat ist also definiert durch $\langle \Sigma, \Phi, S, F, \delta \rangle$. Entscheidend für einen Akzeptor ist nun also, aus welchen Elementen das Alphabet besteht und wie die Übergangsfunktionen definiert sind. Nehmen wir an, das Alphabet

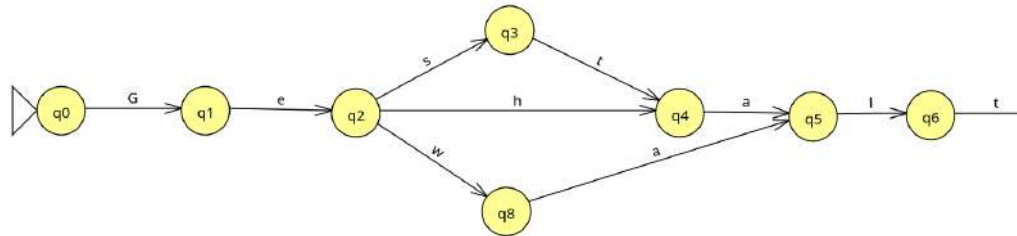
ist durch $\Sigma = \{a, b, c, \dots, z, A, B, C, \dots, \dots\}$, das heisst, alle Buchstaben des lateinischen Alphabets inklusive der Umlaute. Ein Subset aller Strings über Σ , zum Beispiel Σ_{Deutsch} , könnte sämtliche Wörter (oder eben Strings) der deutschen Sprache repräsentieren. Als solches wäre das Set $\{\text{Augustus}, \text{Box}, \text{Rinderkennzeichnungsfleischetikettierungsberwachungsaufgabenübertragungsgesetz}\}$ Teil von Σ_{Deutsch} . Eine ganz besondere Sprache ist “Sigma Stern”, oder Σ^* . Diese akzeptiert sämtliche Strings, die mit Σ gebildet werden können, also wiederum das Set $\{\text{Augustus}, \text{Box}, \text{Rinderkennzeichnungsfleischetikettierungsberwachungsaufgabenübertragungsgesetz}\}$, als auch $\{\text{asdljkl}, \text{iweuw}, \text{beimteutates}\}$. Der sogenannte **Kleene-Stern** $*$ sagt im Prinzip aus, dass jede Zeichenfolge, die mit dem entsprechenden Alphabet möglich ist, erlaubt ist, auch der leere String “”, oder die leere Sprache $\{\}$. Wir schauen uns nun mal ein konkretes Beispiel an:



Der Startzustand S ist mit einem Dreieck markiert, während der Endzustand F an einem doppelt gezogenen Kreis erkennbar ist. Zwischen allen Zuständen finden sich Kanten, die mit Buchstaben beschriftet sind. Diesem “Mini-Automat” liegt folgendes Alphabet zugrunde: $\Sigma_{\text{Mini-Automat}} = \{G, e, h, s, w, a, l, t\}$. Mit $\Sigma_{\text{Mini-Automat}}^*$ lassen sich Wörter wie *alt*, *halte*, oder auch *eGhwa* bilden. So, wie der obige Automat definiert ist, akzeptiert er aber lediglich die Sprache $\{\text{Gewalt}, \text{Gehalt}, \text{Gestalt}\}$. Wir haben hier einen *deterministischen endlichen Automaten* (kurz *DEA*) vorliegen. Warum deterministisch? Angenommen, uns liegt das Wort *Gewalt* vor und wir wollen wissen, ob es vom Automaten akzeptiert wird, dann beginnen wir mit dem Buchstaben *G*. Um vom Startzustand q_0 in den Zustand q_1 zu gelangen, muss der Automat ein solches *G* akzeptieren. Als Übergangsfunktion ausgedrückt: $\delta(q_0, G) = q_1$. Diese Funktion erhält als Parameter (oder deren Input) den aktuellen Zustand und das nächste Zeichen im String (hier sogar das erste). Wir gelangen dann zu q_1 , wenn der Automat ein *G* akzeptiert. Als nächstes Zeichen, das akzeptiert werden muss, müssten wir vom Zustand q_1 in den Zustand q_2 ein *e* in *Gewalt* finden (als Funktion: $\delta(q_1, e) = q_2$). Da dies tatsächlich der Fall ist, gelangen wir in den Zustand q_2 . Nun gibt es folgende Übergangsfunktionen: $\delta(q_2, h) = q_9$, $\delta(q_2, s) = q_3$ und $\delta(q_2, w) = q_8$. Im String folgt als Nächstes ein *w*. Da dies in den Übergangsfunktionen abgebildet ist, gelangen wir in den Zustand q_8 . So iterieren wir durch den String und prüfen Zeichen für Zeichen, ob wir in einen nächsten Zustand gelangen können. In diesem Beispiel erreichen wir den Endzustand deterministisch, da es—gegeben dem Input-String—von jedem Zustand nur eine Möglichkeit für den Automaten gibt, in einen Folgezustand zu gelangen. Endlich ist der Automat, weil wir eine endliche Menge an Endzuständen haben (in diesem Fall sogar

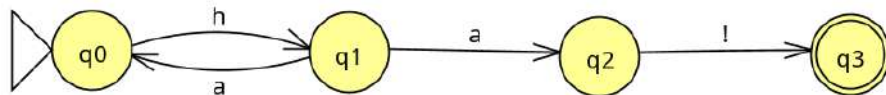
nur einen, nämlich q_7). Einen weiteren Fall von Nichtdeterminismus hätten wir im Übrigen vorliegen, wenn der Automat irgendwo eine ϵ -, respektive λ -Kante aufweisen würde.

Dieser Automat ist im Übrigen noch nicht so elegant, wie er sein könnte. Vergleichen wir den obigen Automaten mit einer etwas schlankeren Version:

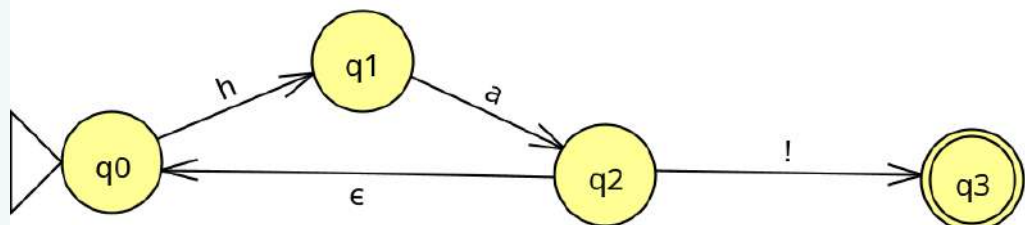


Wir sparen uns so einen Zustand (q_9). Für so einen kleinen Automaten mag dies irrelevant sein. Wächst jedoch die Sprache, die akzeptiert werden soll, so gibt es auch immer mehr Zustände, was schlussendlich einen Einfluss auf den Speicherplatz hat.

Zusammengefasst lässt sich sagen: Ein Automat ist deterministisch, wenn er keinen Zustand aufweist, von welchem mehrere Kanten mit demselben Symbol ausgehen, und wenn er keine leeren Kanten enthält. Für die Automaten-Theorie besonders wichtig ist der Fakt, dass es für jeden nicht-deterministischen Automaten einen deterministischen Automaten gibt. Schauen wir uns dafür zuerst einen nicht-deterministischen Automaten an:

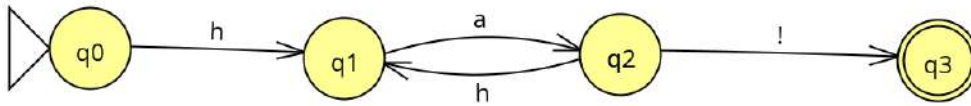


Dieser akzeptiert unter anderem die Sprache $\{ha!, haha!, hahahahahaha!\}$. Wir sehen, dass vom Zustand q_1 zwei Kanten mit dem Zeichen a ausgehen. Somit ist die Bedingung für den Determinismus nicht erfüllt. Ähnlich verhält es sich mit dem nächsten Beispiel:



Dieser Automat akzeptiert dieselbe Sprache, enthält aber anstatt zweier gleichlautender Kanten vom selben Zustand aus eine ϵ -Kante.

Wenn wir nun die Zustände und Übergangsfunktionen geringfügig anpassen, können wir aus diesen nicht-deterministischen Automaten einen deterministischen machen. Dieser sieht dann so aus:



Alle von jeglichen Zuständen ausgehenden Kanten sind nun eindeutig und die ϵ -Kante ist eliminiert. Der Automat akzeptiert aber weiterhin dieselbe Sprache. Zuguterletzt sei noch auf den Unterschied zwischen endlichen und unendlichen Automaten hingewiesen. Wir haben bereits angedeutet, dass endliche Automaten eine endliche Menge an Endzuständen haben. Insofern waren alle bisher betrachteten Automaten endlich.

^aEs existieren auch *unendliche* Automaten. Da diese jedoch für unsere Zwecke nicht relevant sind und den Rahmen des Exkurses sprengen würden, verzichten wir auf ihre Diskussion.

4.1.2 Reguläre Ausdrücke in Python

Wo ist nun der Zusammenhang zwischen den Automaten und den regulären Ausdrücken? Die regulären Ausdrücke erlauben es uns, den schrittweisen Akzeptanzprozess zu modellieren. Bezogen auf den Automaten aus dem Exkurs, der unter anderem das Wort *Gehalt* akzeptiert, würde ein regulärer Ausdruck, der alle drei Wörter dieser Mini-Sprache matcht, so aussehen: `Ge(wa|ha|sta)lt`. In natürliche Sprache übersetzt heisst das in etwa: “Akzeptiere das Wort, das mit *G* beginnt und mit einem *e* weiterfährt. Dann folgt entweder die Buchstabenfolge $w \cdot a$, $h \cdot a$, oder $s \cdot t \cdot a$. Die Sequenz endet dann mit $l \cdot t$.” Der reguläre Ausdruck akzeptiert also die Sprache $\{\textit{Gewalt}, \textit{Gehalt}, \textit{Gestalt}\}$. Die runden Klammern ... erlauben es uns, Alternativen zu definieren, die wir durch das Pipe-Symbol `|` abtrennen. Im Hintergrund wird also ein regulärer Ausdruck, den wir niederschreiben, in einen Automaten umgewandelt. Deshalb ist auch ein Verständnis der Automaten so essenziell.

Wir schauen uns nun also an, wie wir reguläre Ausdrücke in Python definieren und wie wir sie zur Bearbeitung von historischen Daten verwenden können. Dazu ist es wichtig, spezielle Zeichen und Sequenzen zu kennen, mit welchen wir bestimmte Strings in Texten matchen können. Diese schauen wir uns zuerst an (siehe Tabelle 4.1):

Zeichen	Beschreibung	Beispiel
.	Punkt, matcht ein beliebiges Zeichen	<code>Ge.alt</code> $\rightarrow$ $\{\textit{Gehalt}, \textit{Gewalt}\}$
*	Kleene-Stern, matcht keines oder beliebig viele des mit dem Stern markierten Zeichens	<code>Ge.*alt</code> $\rightarrow$ $\{\textit{Gealt}, \textit{Gehalt}, \textit{Gewalt}, \textit{Gestalt}, \textit{Gewekrkeralt}\}$
+	Plussymbol, matcht das mit dem Plussymbol markierte Zeichen <i>mindestens</i> ein Mal	<code>Ge.+alt</code> $\rightarrow$ $\{\textit{Gehalt}, \textit{Gewalt}, \textit{Gestalt}, \textit{Gewekrkeralt}\}$
?	Fragezeichen, matcht das mit dem Fragezeichen markierte Zeichen 0 oder <i>maximal</i> ein Mal	<code>Ge.?alt</code> $\rightarrow$ $\{\textit{Gealt}, \textit{Gehalt}, \textit{Gewalt}\}$
[]	Eckige Klammern, definieren einen Bereich (ein Set an Zeichen)	<code>Ge[wh]alt</code> $\rightarrow$ $\{\textit{Gehalt}, \textit{Gewalt}\}$
()	Runde Klammern, markieren eine Gruppe, oft in Verbindung mit dem Pipe-Symbol	<code>Ge(w h st)alt</code> $\rightarrow$ $\{\textit{Gewalt}, \textit{Gehalt}, \textit{Gestalt}\}$
	Pipe-Symbol, ein entweder-oder	<code>Gewalt Gestalt Gehalt</code> $\rightarrow$ $\{\textit{Gewalt}, \textit{Gestalt}, \textit{Gehalt}\}$
^	Caret, matcht den Beginn eines Strings	<code>^Gewalt</code> $\rightarrow$ $\{\textit{Gewalt}\}$
\$	Dollar-Symbol, matcht das Ende eines Strings	<code>Gewalt\$</code> $\rightarrow$ $\{\textit{Gewalt}\}$

Tabelle 4.1 Auflistung einiger spezieller Zeichen (Metazeichen), die sich in einem regulären Ausdruck speziell verhalten.

Zudem gibt es eine spezielle Gruppe an Zeichen, die eine spezielle Sequenz definiert und entsprechend matcht. In Verbindung mit den oben in Tabelle 4.1 erwähnten

Sonderzeichen kann schon beinahe jeder beliebig komplexe String gematcht werden. Die wichtigsten sind in Tabelle 4.2 gelistet:

Zeichen	Beschreibung	Beispiel
\d	Matcht eine beliebige Ziffer zwischen 0 und 9	\d\d\d\d → {1990, 2024, 9999}
\D	Matcht ein Zeichen, welches NICHT einer Ziffer zwischen 0 und 9 entspricht	\D\D\D\D → {Hund, Buch, gelb}
\s	Matcht ein sogenanntes “White-Space-Zeichen”, also einen Abstand	Der\sHund\sfrisst. → {Der Hund frisst.}
\S	Matcht alles AUSSER Abstände	\S\S\S\S → {Hund, Buch, gelb}
\w	Matcht alle sogenannten alphanumerischen Zeichen (inklusive \texttt{_})	\w+ → {1, Gehalt, Gewalt, Gestalt, 1990, 2024, 9999}
\W	Matcht alle Zeichen die NICHT alphanumerisch sind	\W → {“ ”} (ein Abstand)
\b	Matcht Wortgrenzen	\bin\b → {in}

Tabelle 4.2 Übersicht über Zeichen, die in regulären Ausdrücken eine Gruppe von Zeichen (oder ein Set) repräsentieren.

Wir schauen uns nun einige Beispiele an. Um in Python mit regulären Ausdrücken operieren zu können, muss man zuerst das sogenannte Regex-Modul laden. Ein Modul¹ ist im Prinzip nichts anderes als eine Python-Datei, die spezielle Klassen (siehe Kapitel 7) und Funktionen (siehe Kapitel 6) enthält, auf die man in seinem Programmcode zurückgreifen möchte. Durch ein sogenanntes `import`-Statement, das man üblicherweise zu Beginn des Programms notiert, werden die entsprechenden Funktionen und Klassen geladen und stehen dem Code dann so lange zur Verfügung, wie der Code ausgeführt wird. Viele solche Packages sind Teil der sogenannten “Python Standard Library” und stehen somit direkt nach der Installation von Python zur Verfügung. Andere Packages muss man sich zuerst installieren (siehe dazu Fussnote 1). Das Modul für reguläre Ausdrücke ist in der Python Standard Library enthalten und heisst `re`. Wir arbeiten uns nun durch einige Beispiele mit `re`.

Code-Beispiel 4.1 Beispiel für einfache Wort-Matches mittels Regex.

```
>>> import re
>>> text = '''Ex hac namque existunt inimici, aduersus quos defendenda est dei
ciuitas, quorum tamen multi correcto inpietatis errore ciues in ea fiunt
satis idonei; multi uero in eam tantis exardescunt ignibus odiorum tamque
manifestis beneficiis redemptoris eius ingrati sunt, ut hodie contra eam
linguas non mouerent, nisi ferrum hostile fugientes in sacratis eius locis
uitam, de qua superbiunt, inuenirent. an non etiam illi Romani Christi nomini
infesti sunt, quibus propter Christum barbari pepercerunt? testantur hoc
martyrum loca et basilicae apostolorum, quae in illa uastatione urbis ad se
confugientes suos alienosque receperunt. hucusque cruentus saeuiebat inimicus,
ibi accipiebat litem trucidatoris furor, illo ducebantur a miserantibus
hostibus, quibus etiam extra ipsa loca pepercerant, ne in eos incurrerent, qui
similem misericordiam non habebant. qui tamen etiam ipsi alibi truces atque
hostili more saeuientes posteaquam ad loca illa ueniebant, ubi fuerat
interdictum quod alibi belli iure licuisset, tota ferendi refrenabatur
inmanitas et captiuandi cupiditas frangebatur. '''
>>> re.findall('in', text)
['in', 'in', 'in', 'in', 'in', 'in', 'in', 'in', 'in', 'in', 'in', 'in', 'in',
'in', 'in', 'in']
>>> re.findall(' in ', text)
[' in ', ' in ', ' in ', ' in ', ' in ']
```

¹Auch “Python Packages” genannt. Eine Übersicht über viele Packages, die man bei sich installieren kann, findet man auf <https://pypi.org/>.

```
>>> re.findall(r'\bin\b', text)
[' in ', ' in ', ' in ', ' in ', ' in ']
```

Wir nehmen als Beispiel einen Textausschnitt aus Augustinus “De Civitate Dei”, 1,1. Wenn wir beispielsweise nach allen Vorkommen der Präposition “in” suchen möchten, wäre die naivste Lösung, dies mit `re.findall('in', text)` zu tun. Wir sprechen mir `re`, das entsprechende Modul an, das wir importiert haben. Mit der Funktion `findall()`, die zwei Argumente erwartet, suchen wir nach allen Vorkommen, und zwar vom String `'in'` im Text, den wir in der Variablen `text` abgelegt haben. `re` liefert uns 16 Treffer in einer List zurück. Wenn wir nun manuell die Vorkommen von `'in'` durchzählen, merken wir, dass es in Tat und Wahrheit viel weniger sein müssten. Das rührt daher, dass der Code, so wie wir ihn geschrieben haben, auch das `'in'` in Wörtern wie *inimici*, *inpetiatis*, *linguas*, *ingrati*, etc. matcht.

Um nur die Präposition *in* zu matchen, können wir einfach davor und danach einen Leerschlag setzen. Das reduziert die Vorkommen auf fünf. Wir können aber auch eine der speziellen Sequenzen bemühen, beispielsweise jene für Wortgrenzen `\b`. Bei dieser dritten Variante aus Code-Beispiel 4.1 ist es wichtig zu beachten, dass dem String ein `r` vorangeht. Dieses `r` sagt Python, dass es den folgenden String als regulären Ausdruck lesen muss, der Spezialzeichen enthält. Wenn wir das `r` weglassen, dann sucht der Automat nach der Folge “Backslash, b, i, n, Backslash, b” und wird in diesem Text selbstverständlich nicht fündig. Acht muss gegeben werden, wenn man einen String mit einem `r` versieht und danach nach einem Backslash oder einem Punkt suchen will. Denn mit dem vorangestellten `r` interpretiert Python den String ja als regulären Ausdruck. Wollen wir also zum Beispiel Punkte explizit matchen, dann müssen wir sie “escapen.” Das tun wir mit einem Backslash. Einen Punkt matchen wir also beispielsweise mit `re.findall('\.', text)`, während ein Backslash mit `re.findall('\\', text)` gefunden wird.

Wir wollen uns noch ein paar Beispiele anschauen. Da das Lateinische ja eine überaus flektierende Sprache ist, können wir beispielsweise nicht einfach nach dem Wort *domus* (DE *Haus*) suchen und erwarten, alle Instanzen zu finden. Das Wort kann ja auch als *domo*, *domi*, oder *domum* im Text vorkommen. Wenn wir nun aus der Augustinus-Passage alle Wörter herausfiltern wollen, die etwas mit “Feind” (LA *hostis*) zu tun haben, dann könnten wir das so machen, wie in Code-Beispiel 4.2 erklärt:

Code-Beispiel 4.2 Beispiel für das Matchen von Wörtern mit verschiedenen Endungen.

```
>>> re.findall(r'host.*', text)
['hostile fugientes in sacratis eius locis uitam, de qua superbiunt,
inuenirent. an non etiam illi Romani Christi nomini infesti sunt, quibus
propter Christum barbari pepercerunt? testantur hoc martyrum loca et basilicae
apostolorum, quae in illa uastatione urbis ad se confugientes suos alienosque
receperunt. hucusque cruentus saeuiebat inimicus, ibi accipiebat limitem
trucidatoris furor, illo ducebantur a miserantibus hostibus, quibus etiam
extra ipsa loca pepercerant, ne in eos incurrerent, qui similem misericordiam
non habebant. qui tamen etiam ipsi alibi truces atque hostili more saeuientes
posteaquam ad loca illa ueniebant, ubi fuerat interdictum quod alibi belli
iure licuisset, tota ferendi refrenabatur inmanitas et captiuandi cupiditas
frangebatur.']
```

Was ist hier schiefgegangen? Vorneweg, wir gehen davon aus, dass wir alle Wörter, die etwas mit Feindschaft zu tun haben, mit der Wortstamm “host-” gefolgt von einer beliebigen Anzahl an Zeichen finden. Wir erwarten, dass der obige reguläre Ausdruck Instanzen wie *hostis*, *hostilis*, *hostilia*, etc. matcht. Dieses Beispiel zeigt aber eine besondere Eigenheit von regulären Ausdrücken, speziell dann, wenn man den Kleene-Stern benutzt (selbiges gilt aber auch für das Plus-Symbol): Sie sind standardmässig “gierig” (EN *greedy*), matchen also so viele Zeichen, wie es nur geht. In diesem Fall matcht der reguläre Ausdruck das mit Buchstaben abgebildete **host** und danach sämtliche weiteren Buchstaben bis zum Ende des Strings. Wenn wir diese Gier eingrenzen möchten, müssen wir nach dem Kleene-Stern und dem Plus-Symbol ein Fragezeichen platzieren. Das Code-Beispiel 4.3 zeigt das nun abgeänderte Verhalten:

Code-Beispiel 4.3 Beispiel für das Matchen von Wörtern mit verschiedenen Endungen in einer nicht gierigen Variante.

```
>>> re.findall(r'host.*?', text)
['host', 'host', 'host']
```

Dank dieser Variante wissen wir nun, dass es drei Wörter gibt, die mit **host** beginnen. Da der Kleene-Stern eben auch 0 Zeichen matcht, nun aber diese “Gierigkeit” mit dem Fragezeichen ausgehebelt ist, matcht der reguläre Ausdruck nach “host” kein Zeichen und gibt nur die gefundenen Instanzen der Wortstammes zurück. Nun stellt sich die Frage, ob es sich denn auch um die gesuchte Bedeutung, nämlich der der Feindschaft handelt? Hierzu wäre es von Vorteil, wenn uns Python die kompletten Wörter zurückliefern würde. Dies erreichen wir, indem wir die Wortenden noch dazumatchen lassen. Code-Beispiel 4.4 macht das klar:

Code-Beispiel 4.4 Beispiel für das Matchen von Wörtern mit verschiedenen Endungen in einer nicht gierigen Variante mit Wortenden.

```
>>> re.findall(r'host.*?\b', text)
['hostile', 'hostibus', 'hostili']
```

Durch das blosses Anfügen des Wortendes mittels `\b` erreichen wir das gewünschte Verhalten. Der reguläre Ausdruck wird nun gezwungen, alle Buchstaben nach “host” bis zur nächsten Wortgrenze, repräsentiert durch ein Leerzeichen, zu matchen. So werden alle Wörter zurückgeliefert, die mit **host** beginnen, danach von einer beliebigen Zeichenfolge komplettiert werden, die aber maximal bis zum Wortende reicht. Und siehe da: Alle Wortbedeutungen entsprechen der von uns gesuchten Feindschaft. So ermöglichen es uns also die regulären Ausdrücke, nach Wortstämmen zu suchen.

- Q Die Endungen werden sekundär und die sogenannten [Wildcards](#) übernehmen das Matching derer.

Neben der `findall()`-Funktion gibt es noch die `search()`-Funktion. Diese gibt anstatt einer Liste von allen Treffern nur der erste “Hit” als `re.Match`-Objekt aus. Diese Funktion kann man beispielsweise verwenden, wenn man nur auf den ersten Treffer angewiesen ist (wie beim Matchen von eindeutigen Meta-Daten in einer Datei, wo man exakt weiss, welches Meta-Datum wo vorkommt und die Struktur gut kennt). Das `re.Match`-Objekt enthält auch Positionsangaben. Wir erfahren also

die exakte Position des Strings mit Start- und End-Index. Schauen wir uns das im Code-Beispiel 4.5 an:

Code-Beispiel 4.5 Beispiel für das Matchen von Wörtern mit der `search()`-Funktion.

```
>>> re.search(r'host.*?\b', text)
<re.Match object; span=(319, 326), match='hostile'>
>>> match = re.search(r'host.*?\b', text)
>>> match.span()
(319, 326)
>>> text[319:326]
'hostile'
```

Wir sehen, dass das Objekt mit `span()` den Start- und End-Index im String `text` als Tupel zurückliefert. Somit wissen wir genau, wo der reguläre Ausdruck den Match im String gefunden hat. Wir können mit einem “Slice” dann auch das Wort gesondert extrahieren.

Spezielle Matchverfahren

Reguläre Ausdrücke bieten eine Vielzahl von Möglichkeiten, nach Strings zu suchen. Besonders erwähnenswert sind an dieser Stelle der sogenannte “Lookahead” sowie der “Lookbehind”. Die Bezeichnungen sind fast selbstredend: Der Lookahead matcht Strings, wenn auf sie eine bestimmte Zeichenfolge folgt. Der Lookbehind hingegen matcht Strings, wenn dem String eine bestimmte Zeichenfolge vorangeht. Man kann Lookahead und Lookbehind auch kombinieren, sodass nur Strings gematcht werden, die von einer bestimmten Zeichfolge umgeben sind. Wir verdeutlichen alle drei Szenarien am Code-Beispiel 4.6:

Negativer
Lookahead/-behind
einfügen

Code-Beispiel 4.6 Beispiel Lookahead, Lookbehind und einer Kombination davon.

```
>>> re.findall(r'in(?:=se)', text)
['in', 'in', 'in']
>>> re.findall(r'inse.+?\b', text)
['in ea', 'in eam', 'in eos']
>>> re.findall(r'(?<=o\s)in', text)
['in', 'in']
>>> re.findall(r'\b\w+o\s(in', text)
['correcto in', 'uero in']
>>> re.findall(r'(?<=o\s)in(?:=se)', text)
['in']
>>> re.findall(r'\b\w+o\s(inse.+?\b', text)
['uero in eam']
```

Die erste Code-Zeile entspricht einem Lookahead. Diesen schreiben wir generell als `(?=)`, wobei nach der Gleichheitszeichen die Zeichenfolge definiert wird, die auf den vorangehenden regulären Ausdruck folgen soll. Wir matchen hier konkret ein String `in`, der von einem Leerzeichen und einem `e` gefolgt wird. Wichtig: Der Lookahead zwischen den runden Klammern wird nicht mitgematcht, das sich an den drei Instanzen von `in` zeigt. In der nächsten Zeile schreiben wir aus, dass wir nun gerne wissen möchten, welche Wörter auf `in` folgen, die mit einem `e` beginnen.

Darauf folgt ein Beispiel eines Lookbehind, den wir als (?<=) beschreiben. Wieder ausgehend von `in` interessiert uns aber nun, welche Wörter diesem vorangehen und gleichzeitig auf ein `o` enden. Wir erhalten zwei solche Vorkommen und schauen uns die Wörter genauer an. Es sind *uero* und *correcto*. Diese matchen wir mit einem etwas komplizierteren Ausdruck, ausgehend von einer Stringgrenze `\b` über mindestens ein alphanumerisches Zeichen `\w+?` (“non-greedy”) bis zum nächsten `o`, gefolgt von einem Leerzeichen `~` und schliesslich dem `in`.

Zuguterletzt kombinieren wir Lookahead und Lookbehind und finden so das eine Vorkommen von `in`, dem ein Wort vorangeht, das auf ein `o` endet und das von einem Wort gefolgt wird, das mit einem `e` beginnt.

Des Weiteren sei auf die Möglichkeit hingewiesen, Alternativen zu matchen. Code-Beispiel `lst:alternatives` zeigt dies gleich:

Code-Beispiel 4.7 Beispiel für das Matchen von Alternativen.

```
>>> re.findall(r'(in\s)(ea|eam|eos)', text)
[('in ', 'ea'), ('in ', 'ea'), ('in ', 'eos')]
```

Wir suchen in der zweiten Klammer nach verschiedenen Varianten, die auf `in` folgen könnten. Abgetrennt sind diese jeweils mit dem Pipe-Symbol. Die erste Klammer matcht einen String `in` gefolgt von einem Leerzeichen. Bei diesem Beispiel muss beachtet werden, dass wir in diesem Fall genau wissen, wonach wir suchen. Das könnte uns zum Beispiel auch bei der Suche nach Eigennamen zunutze machen (siehe Code-Beispiel 4.8).

Code-Beispiel 4.8 Beispiel für das Matchen von Alternativen.

```
>>> inscription = '''[Cn(aeus) Tremel]ius urb(anus) /
[M(arcus) Aureliu]s Co[tt]a per(egrinus) /
[Ti(berius)] Caes(ar) Au]g(ustus) III /
[Germani]c(us) Caes(ar) II /
[suf(ecti) L(ucius) Seiu]s Tubero /
M(arcus) Livineius /'''
>>> re.findall(r'(\[?Germani\]?c\(?us\)?|Ti\(?berius\)?\])\s\w\].]+(?:=Caes\(?ar\)?)', inscription)
['Ti(berius)', '[Germani]c(us)']
```

Wir sehen hier einen Auszug aus der Inschrift #4 der mit der ID EDCS-43900008, wie man sie in der *Epigraphik-Datenbank Claus-Slaby*² vorkommt. Es handelt sich hier um eine edierte Inschrift, weswegen wir zwischen eckigen Klammern mögliche Ergänzungen und zwischen runden Klammern Auflösungen von Abkürzungen finden. Die Forward-Slashes stellen Zeilenumbrüche im Original dar. Der obige reguläre Ausdruck sucht jetzt entweder nach *Germanicus* oder *Tiberius*, sofern der Name von *Caesar* ergänzt wird. Somit könnten wir die Kaisernamen extrahieren.

Zuletzt illustrieren wir noch das Matching mit Quantifikatoren. Es gibt eine einfache Möglichkeit, wie man mit regulären Ausdrücken eine exakte Anzahl oder einen vordefinierten Bereich an einem Zeichen oder einer Gruppe matchen kann. Wir unterscheiden zwischen drei möglichen Arten:

1. `\d{3}` → matcht genau drei aufeinanderfolgende Ziffern, zum Beispiel 523.

²siehe <https://edcs.hist.uzh.ch/>.

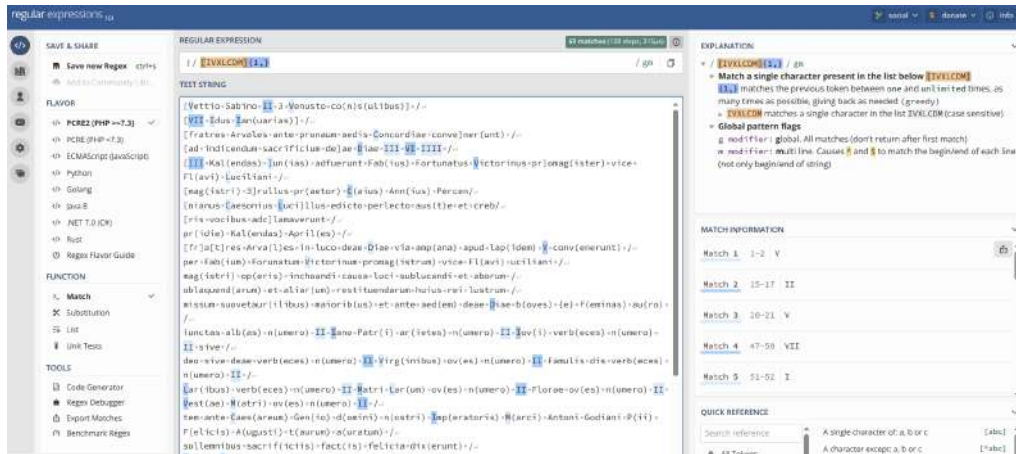


Abbildung 4.1 Beispiel für das Matching von römischen Zahlen an der Inschrift #1 mit der ID EDCS-43900008 auf regex101.

2. `\d{3,}` → matcht mindestens drei aufeinanderfolgende Zahlen, wobei die Begrenzung nach oben offen ist. Matcht zum Beispiel 523, aber auch 2025, 111 oder 23920482092942.
3. `\d{2,4}` → matcht verschieden lange Zahlen bestehend aus mindestens zwei und maximal vier Ziffern. Zum Beispiel matcht dies nach wie vor 523, 2025 oder 111, aber zusätzlich auch 41, 15 oder 9999.

Wir können nun im Text vom vorangehenden Code-Beispiel nach römischen Zahlen suchen. Anstatt jede Möglichkeit, die aus dem Set {I, V, X, L, C, D, M, } aufzulisten, können wir solche Ausdrücke viel einfacher formulieren:

Code-Beispiel 4.9 Beispiel für das Matchen römischen Zahlen.

```
>>> re.findall(r'I{1,}', inschrift)
['III', 'II']
```

Das Code-Beispiel 4.9 zeigt, wie wir nach römischen Zahlen suchen, die aus mindestens einem I bestehen. Das Problem ist nur, dass dies auch das I des Namens “Iulius” matchen würde, wenn es denn vorkäme. Wenn wir also das Zahlensystem der Römer exakt wiedergeben möchten, müssen wir den regulären Ausdruck anders definieren.

Das Testen von regulären Ausdrücken

Es kann sich lohnen, reguläre Ausdrücke zuerst in einer Test-Umgebung auszuprobieren, bevor man sie bei sich im Code verwendet. Eine dafür passende Umgebung bietet die Webseite <https://regex101.com> (siehe Abbildung 4.1).

Wir sehen, dass der oben eingegebene reguläre Ausdruck noch nicht das matcht, was gesucht ist. Der Vorteil von regex101 ist, dass man in der rechten Sidebar eine Erklärung erhält, welche Zeichen im regulären Ausdruck was genau matchen. Des Weiteren würden verschiedene Match-Gruppen farbkodiert, sodass man gezielte Informationen erhält, was am regulären Ausdruck eventuell abgeändert werden muss.

4.1.3 Video zu regulären Ausdrücken

 Schaue dir jetzt das [Video 4.1.3](#) zu den regulären Ausdrücken an und bearbeite dann das Quiz in OLAT.

4.2 Quiz zu regulären Ausdrücken

☒ Löse das [Quiz auf OLAT](#) und bearbeite dann das nächste Kapitel.

If-Else-Statements und Entscheidungslogik

Meine göttliche Mutter, die silberfüssige Thetis,
Sagt, mich führe zum Tod' ein zweifach endendes Schicksal.
Wenn ich allhier verharrend die Stadt der Troer umkämpfe;
Hin sei die Heimkehr **dann**, doch blühe mir ewiger Nachruhm.
Aber **wenn** heim ich kehre zum lieben Lande der Väter;
Dann sei verwelkt mein Ruhm, doch weithin reiche des Lebens
Dauer, und nicht frühzeitig ans Ziel des Todes gelang' ich.

Homer, *Ill.* IX, 410-416 (meine Hervorhebung)

Lernziele Kapitel 5: If-Else-Statements und Entscheidungslogik

- 1. Verstehen** Die Studierenden können den Zweck von *if*-, *else*- und *elif*-Bedingungen erläutern und erklären, wie Entscheidungslogik in Programmen den Ablauf steuert.
- 2. Analysieren** Die Studierenden können Bedingungen mit *Vergleichsoperatoren* (z. B. `==`, `<`, `>`) und *booleschen Operatoren* (**and**, **or**, **not**) kombinieren und ihre Wirkung auf Entscheidungsstrukturen beschreiben.
- 3. Anwenden** Die Studierenden können einfache Entscheidungslogiken in Python implementieren, um historische Daten (z. B. Datierungen oder Quellentypen) unterschiedlich zu behandeln.
- 4. Bewerten** Die Studierenden können verschachtelte Bedingungen auf Verständlichkeit und Effizienz beurteilen und einschätzen, wann eine Vereinfachung sinnvoll ist.
- 5. Erzeugen** Die Studierenden können kleine Entscheidungsprogramme entwickeln, die historische Szenarien modellieren (z. B. "Wenn das Jahr `<` 641,

dann Spätantike; sonst Frühmittelalter”).

5.1 Wenn ... dann ...

Code ist wie ein Pfad. Er hat einen Anfang. Er hat ein Ende. Aber zwischen Anfang und Ende gibt es immer wieder Abzweigungen. Wenn wir jetzt an eine solche geraten, fragen wir uns: “Welchen Pfad nehmen wir?” Sofort überlegen wir “Wenn wir Pfad A weiter folgen, dann passiert X.” Aber auch “Und wenn wir stattdessen auf Pfad B wechseln, dann passiert Y.” An einer solchen Weggabelung muss also eine Entscheidung getroffen werden. Auch Homers Achilles stand in der *Illias* vor einer Entscheidung: “Bleibe ich und kämpfe um Troja oder gehe ich nach Hause?” Auch über die Konsequenzen machte er sich Gedanken: “Wenn ich bleibe, ist mir ewiger Ruhm sicher. Wenn ich gehe, dann versiegt der Ruhm aber ich kann mich an einem langen Leben erfreuen.” Wer die Illias kennt, weiss um Achilles’ Schicksal.

Den unten folgenden Exkurs durch einen, der das Filesystem erklärt, ersetzen.

In diesem Kapitel widmen wir uns einem—im wahrsten Sinne des Wortes—entscheidenden Paradigma: der Entscheidungslogik. Sie gibt vor, wie und unter welchen Bedingungen welcher Pfad genommen wird, also wie genau euer Code ausgeführt wird. Wir verwenden diese Entscheidungslogik oft, wenn wir Dateien einlesen und aus ihnen Informationen extrahieren wollen. Neben ganz simplen Text-Files, die bekannt sein dürften und in der Regel auf `.txt` enden, gibt es auch Textdateien, die mit zusätzlichen Annotationen versehen sind. Eine dieser Annotations-Standards nennt sich *eXtensible Markup Language*, kurz XML. Diesen betrachten wir im folgenden Exkurs genauer. Bevor wir uns dem XML widmen: Wir hätten uns auch nur Text-Dateien anschauen können. Die Realität sieht aber anders aus: Immer mehr Daten liegen in standardisierten XML-Formaten vor, weswegen wir auch gleich damit beginnen.

Aus XML (und dann auch anderen File-Formaten, inkl. Encodings) ein eigenes Kapitel machen?

Exkurs

5.1.1 XML

Viele Daten, mit denen auch in der Geschichtsforschung gearbeitet wird, liegen uns im sogenannten XML-Format vor. XML steht für *eXtensible Markup Language*. Sie ist eine Auszeichnungssprache, die wir in Textdateien verwenden und üblicherweise mit der `.xml`-Endung abspeichern. XML erlaubt es uns, Daten hierarchisch zu ordnen.

Der Standard hat sich seit 1998, wo er zum ersten Mal veröffentlicht wurde, etabliert und stetig weiterentwickelt.^a Für Textdaten hat sich zudem das sogenannte TEI-XML durchgesetzt. TEI steht für *Text Encoding Initiative*.^b EpiDoc^c ist eine an TEI angelehnte Version, welche beispielsweise das Erfassen von Inschriften vereinfacht. Liegen die Daten einmal in einem standardisierten XML-Format vor, so ist es ein Leichtes, sie miteinander zu teilen, die Annotationen zu erweitern oder zu veröffentlichen.^d Als Motivation, auf etablierte Annotationsschemata wie (TEI-)XML zurückzugreifen, steht der FAIR-Grundsatz. FAIR ist kurz für:

1. **F**indable → wenn wir Daten veröffentlichen, wollen wir das so tun, dass man sie einfach findet.

2. **Accessible** → wir wollen, dass die Daten maschinenlesbar sind.
3. **Interoperable** → Daten sollen in gängigen, offenen Formaten vorliegen und mit anderen Systemen kompatibel sein.
4. **Reusable** → die Daten sollen einfach wiederzuverwerten sein.

Ein weiterer Vorteil, den XML-Dateien bieten, ist deren Standardisierung. Standards liegen sogenannte **Schemata** zugrunde. Sie definieren, wie eine XML-Datei strukturiert sein muss. Das Praktische daran: Man kann im Zuge der Erstellung einer XML-Datei wiederholt gegen das Schema prüfen und so in Erfahrung bringen, ob die erzeugte Datei *valide* (also gültig) ist. Im gegenteiligen Fall verläuft die Validierung mit einem Fehler, den man dann korrigieren sollte. So stellt man sicher, dass man stets dem Schema folgt und gültiges XML produziert. Das ist wichtig, da ungültige XML-Dateien nicht eingelesen werden können, auch von Python nicht.

Wenden wir uns nun aber dem Aufbau einer XML-Datei zu. Für gewöhnlich steht an erster Stelle einer XML-Datei die Deklaration. Hier spezifizieren wir, um was für eine XML-Datei es sich handelt und welche Version wir verwenden. Es folgen einige Beispiele von Deklarationen:

Code-Beispiel 5.1 Beispiel 1 für eine XML-Deklaration.

```
<?xml version="1.0" encoding="UTF-8"?>
```

Code-Beispiel 5.2 Beispiel 2 für eine TEI-XML-Deklaration.

```
<TEI xml:lang="en" xmlns="http://www.tei-c.org/ns/1.0">
```

In den recht kurzen Code-Beispielen 5.1 und 5.2 passiert schon recht viel. Im Ersten definieren wir, dass der folgende Inhalt eine XML-Datei darstellt. Zusätzlich geben wir die Version mit dem Attribut-Namen **version** und die Zeichenkodierung mit **encoding** an.^e Im Letzteren spezifizieren wir den Inhalt als TEI. Gleichzeitig sagen wir mit **xml:lang**, dass der Text der Datei in Englisch gehalten ist. **xmlns** definiert den Namespace und weist darauf hin, dass die Elemente in dieser XML-Datei zum TEI-Standard gehören.

Was sind nun die Zutaten, die wir für die Repräsentation von Daten in XML brauchen?

1. **Elemente** → Elemente sind die Baublöcke von XML-Dateien. Sie erhalten einen Namen (der theoretisch frei wählbar ist) und sie werden zwischen spitze Klammern `<...>` gefasst. Elemente können in sich geschlossen sein, z. B., `<coin/>` (beachte den Forwardslash vor der abschliessenden Klammer), oder als Start- und End-Tag aufgebaut sein: `<coin>Dupondius</coin>`. Das End-Tag wird hier mit dem Forwardslash markiert und muss gleich heissen, wie das Start-Tag.
2. **Attribute** → Elemente können Attribute enthalten. Diese spezifizieren das Element genauer. Attribute sind wie Key-Value-Paare in Dictionaries aufgebaut. Sie müssen mit einem Buchstaben oder Unterstrich

starten. Der Wert muss zwischen doppelten Anführungszeichen stehen und durch ein Gleichheitszeichen abgetrennt sein. Als Beispiel: `<coin denomination="Dupondius"/>`.

3. **Hierarchie** → Wie bereits erwähnt, lässt sich mit XML die Datenstruktur hierarchisch repräsentieren. Elemente können beliebig tief verschachtelt werden. Allerdings muss die Reihenfolge beachtet werden. Wenn die Elemente 1, 2 und 3 geöffnet werden, müssen sie in umgekehrter Reihenfolge geschlossen werden, als zuerst 3, dann 2 und schliesslich 1. Würde man nach 3 zuerst 1 schliessen wollen, ist das XML ungültig.

Wir verwenden jetzt diese Zutaten und adaptieren die Münzen, die wir in Code-Beispiel 3.10 definiert haben, als XML-Struktur:

Code-Beispiel 5.3 Teil eines Münzdatensatz als XML-Struktur.

```
<?xml version="1.0" encoding="UTF-8"?>
<coins>
  <coin id="6">
    <weight unit="g">5.41</weight>
    <denomination>Dupondius</denomination>
    <issuer>Claudius I</issuer>
    <date from="43" to="64"/>
    <reverse_type>CERES AVGVSTA SC</reverse_type>
  </coin>
  <coin id="4">
    <weight unit="g">6.49</weight>
    <denomination>As</denomination>
    <issuer>Agrippa (Gaius)</issuer>
    <date from="43" to="64"/>
    <reverse_type>Neptun SC</reverse_type>
  </coin>
</coins>
```

Wir sehen, wie die verschachtelte Struktur funktioniert. Wir können uns das Element `<coins>` wie eine Liste vorstellen, in welcher wir alle Münzen sammeln. Jeder `<coin>` in dieser Liste ist eindeutig durch seine `id` identifizierbar. Was darauf folgt, sind weitere Eigenschaften der jeweiligen Münze. Der Datensatz wäre auch viel flacher strukturierbar:

Code-Beispiel 5.4 Teil eines Münzdatensatz als XML-Struktur.

```
<?xml version="1.0" encoding="UTF-8"?>
<coins>
  <coin id="6" weight="5.41g" denomination="Dupondius" issuer="Claudius I" date_from="43" date_to="64" reverse_type="CERES AVGVSTA SC"/>
  <coin id="4" weight="6.49g" denomination="As" issuer="Agrippa (Gaius)" date_from="43" date_to="64" reverse_type="Neptun SC"/>
</coins>
```

Code-Beispiel 5.4 zeigt, wie im Prinzip alle Elemente aus Code-Beispiel 5.3 auch als Attribute definiert werden können. Das erfordert jedoch eine leichte Umstrukturierung (beispielsweise des Datums oder des Gewichts). Benutzen wir den XML-Standard, sind wir bei der Gestaltung der Datenstruktur relativ frei. In der Numismatik hat sich jedoch das *Numismatic Description Schema* (NUDS) etabliert. Es orientiert sich ebenfalls am TEI-Standard. Ein einfacher Eintrag für bloss eine Münze kann mit diesem Schema auf gut über 100 Zeilen anschwellen. Vor allem den Metadaten kommt ein grösseres Gewicht zu. Hier ist ein Auszug einer Münze aus dem 2. Jhr. v. Chr., die den Herakles zeigt:^f

Code-Beispiel 5.5 Münze ‘Price 2051’ im NUDS-XML-Format.

```
...
<typeDesc>
  <objectType xlink:type="simple" xlink:href="http://nomisma.org/id/
coin">Coin</objectType>
  <dateRange>
    <fromDate standardDate="-0200">200 BCE</fromDate>
    <toDate standardDate="-0196">196 BCE</toDate>
  </dateRange>
  <denomination xlink:type="simple" xlink:href="http://nomisma.org/
id/tetradrachm">Tetradrachm</denomination>
  <manufacture xlink:type="simple" xlink:href="http://nomisma.org/id
/struck">Struck</manufacture>
  <material xlink:type="simple" xlink:href="http://nomisma.org/id/ar
">Silver</material>
  <authority>
    <persname xlink:type="simple" xlink:role="statedAuthority"
xlink:href="http://nomisma.org/id/alexander_iii">Alexander III of Macedon<
/persname>
  </authority>
  <geographic>
    <geogname xlink:type="simple" xlink:role="mint" xlink:href="
http://nomisma.org/id/magnesia_ad_maeandrum">Magnesia ad Maeandrum</
geogname>
  </geographic>
  <obverse>
    <type>
      <description xml:lang="en">Head of beardless Heracles
right wearing lion skin headdress</description>
      <description xml:lang="de">Kopf des jugendlichen Herakles
nach r. mit Löwenfellexuvie</description>
      <description xml:lang="fr">Tête d'Héraclès imberbe à dr.
coiffé d'une dépouille de lion</description>
    </type>
    <persname xlink:type="simple" xlink:role="deity" xlink:href="
http://nomisma.org/id/heracles">Heracles</persname>
  </obverse>
...
```

Das Code-Beispiel 5.5 zeigt die Beschreibung der Münze, nachdem die Metadaten definiert worden sind. Wir sehen eine `dateRange` mit den Werten “200 BCE” bis “196 BCE” oder auch die schon uns bekannte `denomination` (hier “Tetradrachme”). Es sind überdies noch `material`, `authority`, geographische Angaben zum Prägunsort und zur Bildseite der Münze gegeben. Zudem sehen wir viele Attribute, meist mit dem Präfix `xlink:`. Der Wert dieses Attributs ist dann ein Link, der auf eine Konzept-Seite in NUDS verweist. Beispielsweise verlinkt http://nomisma.org/id/alexander_iii auf die Konzept-Seite Alexander des Grossen:

The screenshot shows the Nomisma.org website. The main heading is "Alexander III of Macedon (Person, Concept)". Below this, there is a "Canonical URI" and a "Labels" section with preferred and alternate labels in various languages. The "Definitions" section provides a description in English. The "Roles" section lists the organization "Macedonian Kingdom" and the time period "336 BCE" to "323 BCE". The "URI" section shows a specific URI for the concept. The "Relations" section lists various external links, including those to the British Museum, Wikipedia, and other databases. The "Export" section offers options to download data in various formats. A map of the Mediterranean region is displayed on the right, showing the locations of mints, hoards, and finds.

Da alle Münzen, welche unter Alexander dem Grossen geprägt worden sind, mit `xlink:` auf diese Seite verweisen, findet man weiter unten eine Sammlung aller Münzen unter Alexander dem Grossen. Das ist ein Beispiel von **Linked Open Data**, die sich dadurch auszeichnet, dass ein Datensatz mit weiteren Datensätzen aus dem Netz verbunden ist (siehe dazu die “Relations”, die auf Alexander des Grossen Konzeptseite zu finden sind). Dies zeigt die Nützlichkeit von der Strukturierung der Daten in XML.

Als Letztes schauen wir uns noch das eingangs erwähnte EpiDoc XML zur Auszeichnung von Inschriften an. Dafür nehmen wir uns ein Dokument aus den *Inscriptions of Roman Tripolitania 2021*⁹ zur Hilfe. Die Inschrift unter <https://irt2021.inslib.kcl.ac.uk/en/inscriptions/IRT0002.html> ist in TEI-XML strukturiert. TEI zeichnet sich in der Regel dadurch aus, dass zuerst ein `teiHeader`-Element folgt, das sämtliche Metadaten enthält. Im darauf folgenden `text`-Element ist dann der Text verzeichnet. Wenn wir uns einen Ausschnitt aus diesem XML anschauen (wir fokussieren uns auf die ersten fünf Zeilen), sieht dies eher kompliziert aus:

Code-Beispiel 5.6 Teil eines TEI-XMLs zu einer Inschrift.

```
<div type="edition" xlink:lang="la" xlink:space="preserve">
```

```

<ab>
  <lb n="1"/>
    <persName type="divine" ref="#caelestis"><addName nymRef="
domina"><supplied reason="lost">Do</supplied>minae</addName>
      <name nymRef="Caelestis">Caelesti</name></persName>
    <lb n="2"/><w lemma="pro"><supplied reason="lost">pro</supplied
></w> <w lemma="salus">salute</w>
      <w lemma="imperator"><expan><abbr>Imp<del rend="erasure">p</
del></abbr><ex>eratorum</ex></expan></w>
    <lb n="3"/><persName type="emperor" ref="#marcusaurelius"><name
type="praenomen" nymRef="Marcus"><supplied reason="lost"><expan><abbr>M</
abbr><ex>arci</ex></expan></supplied></name>
      <name type="gentilicium" nymRef="Aurelius"><supplied reason="
lost">A</supplied>ureli</name>
      <name type="cognomen" nymRef="Antoninus">Antonini</name></
persName> <del rend="erasure"><w lemma="et">et</w></del>
    <lb n="4"/><persName type="emperor" ref="#commodus"><del rend="
erasure"><name type="praenomen" nymRef="Marcus"><expan><abbr>M</abbr><ex>
arci</ex></expan></name>
      <name type="gentilicium" nymRef="Aurelius">Aureli</name></del>
      <name type="cognomen" nymRef="Commodus"><del rend="erasure">
Commod</del>i</name>
    <lb n="5"/><addName nymRef="Augustus"><expan><abbr>Aug<del
rend="erasure">g</del></abbr><ex>ustorum</ex></expan></addName> </
persName>
    <del rend="erasure"><w lemma="et"><supplied reason="lost">e</
supplied><unclear>t</unclear></w>
      <persName type="emperor" ref="#crispina"><name type="
gentilicium" nymRef="Bruttia"><supplied reason="lost">Brutt</supplied><
unclear>i</unclear><supplied reason="lost">ae</supplied></name>

```

Wir sehen, dass die eigentliche Edition des Texts mit einem `div`-Element startet. Danach folgen die einzelnen Zeilen, separiert durch nummerierte linebreaks (`lb`). Nun wird der Text wiedergegeben, wobei dieser kräftig ausgezeichnet ist. Ein Blick auf die Repräsentation dieser Inschrift auf der offiziellen Homepage hilft:

IRT2021
INSCRIPTIONS OF ROMAN TRIPOLITANIA 2021



[Home](#) - [Inscriptions](#) - [Search](#) - [Indices](#) - [Bibliographic concordance](#)

0002. Dedication to Caelestis for health of emperors

«Previous: 0001 Next: 0003»

Description: Moulded marble base (w: 0.59 x h: 1.04 x d: 0.66) with socket for statue on top.
Text: Inscribed on one face (die, w: 0.44 x h: 0.55).
Letters: Second century capitals: 0.03.

Date: CE 177-180 (reign)

Findspot: Sabratha: Port area
Original location: Unknown
Last recorded location: Sabratha Museum.

Interpretive

Diplomatic

[Do]minae Caelesti
 [pro] salute imp[er]atorum
 [M]arci Aureli Antonini [et]
 [M]arci Aureli Commodi
 5 Aug[ust]i [et] Brut[us]i
 C[ri]spinae Aug[ustae]
 L[ucius] Aemilius Calpurni-
 anus Muse et Mucia
 Pudencilla eius
 10 ob honorem
 L[ucii] Aemili Muciani et
 Q[ui]nti Aemili Angurini
 quondam filiorum
 piissimorum d[ed]erunt d[ed]icauerunt

Zudem ist auch der Blick auf die tatsächliche Inschrift einem besseren Verständnis dienlich:



Die erste Zeile im XML lautet `<persName type="divine" ref="#caelestis"><addName nymRef="domina"><supplied reason="'lost'">Do</supplied>minae</addName><name nymRef="'Caelestis'">Caelesti</name></persName>`. Wir sehen hier zu Beginn eine Beschädigung einer Inschrift. Das “Do” ist verloren. Im XML wird dies hinzugefügt, mit einem `supplied`-Element und der Angabe des Grundes. Auf der Homepage wird dies dann automatisch als “[Do]” gerendert, wobei dies bei der diplomatischen Umschrift als “[.]” wiedergegeben wird.

In Zeile drei wird es noch interessanter: `<persName type='emperor' ref="#marcusaurelius"><name type='praenomen' nymRef='Marcus'><supplied reason='lost'><expan><abbr>M</abbr><ex>arci</ex></expan></supplied></name><name type='gentilicium' nymRef='Aurelius'><supplied reason='lost'>A</supplied>ureli</name><name type='cognomen' nymRef='Antoninus'>Antonini</name></persName><del rend='erasure'><w lemma='et'>et</w>.` Hier ist die Rede vom Kaiser Marc Aurel (121–180). Es wird also der Name “Marci Aureli Antonini” getagged, obwohl das “Marci” ergänzt werden muss. Gleichzeitig erhält der `persName` ein `ref`-Attribut. Dieses referenziert, oder zeigt, auf `#marcusaurelius`. Dass dieser String mit einem Hashtag beginnt, will heissen, dass es in einem anderen Dokument, in welchem zum Beispiel Personennamen gesammelt werden, ein Element mit der ID `xml:id="marcusaurelius"` gibt. Unter diesem Element sind dann alle weiteren Informationen wie Lebensdaten, Ernennung zum Kaiser, und Links zu externen Wissensresourcesn verzeichnet. Das ermöglicht es dann, eine Personenseite wie die Konzeptseite Alexander des Grossen von der Numismatik-Datenbank zu bauen. Wenn mit `ref` und den entsprechenden IDs auf andere XML-Dokumente verwiesen wird, spricht man von einer sogenannten **Stand-off Annotation**. Somit müssen wir nicht an Ort und Stelle alle Infos zu Marc Aurel wiedergeben, sondern können diese Infos gesondert in einem anderen Dokument aggregieren und, wenn immer nötig, wieder ändern. In der Folge können wir immer dann, wenn Marc Aurel wieder in einer Inschrift vorkommt, einfach auf den Eintrag im Personenverzeichnis verweisen. Wichtig ist, dass die IDs eindeutig sind.

^aFür weitere Informationen, siehe die Homepage des World Wide Web Consortiums (W3C): <https://www.w3.org/XML/>.

^bEine umfassende Dokumentation dieses Standards ist auf <https://tei-c.org/> zu finden.

^cSiehe <https://epidoc.stoa.org/>.

^dZum Beispiel mit dem *TEI-Publisher*, siehe <https://teipublisher.com/exist/apps/tei-publisher-home/index.html>.

^eMehr dazu im Abschnitt 5.2.1.

^fSiehe <https://www.numismatics.org/pella/id/price.2051>.

^gSiehe <https://irt2021.inslib.kcl.ac.uk/en/>.

5.2 Operatoren und Ausdrücke

Für die Operatoren greifen wir auf Grundschulmathematik zurück. Es gibt folgende Arten (siehe Unterkapitel 5.4 für Anwendungsbeispiele):

Kompakt Operatoren für Vergleiche

- `==` → “ist gleich”, drückt die Gleichheit von zwei Werten oder Objekten aus.
- `!=` → “ist nicht gleich”, drückt die Ungleichheit von zwei Werten oder Objekten aus.

- `<` → “kleiner als”, Wert des links stehenden Objekts ist kleiner als der Wert, der rechts steht.
- `<=` → “kleiner gleich”, Wert des links stehenden Objekts ist kleiner als der Wert oder er entspricht genau dem, der rechts steht.
- `>` → “grösser als”, Wert des links stehenden Objekts ist grösser als der Wert, der rechts steht.
- `>=` → “grösser gleich”, Wert des links stehenden Objekts ist grösser als der Wert oder er entspricht genau dem, der rechts steht.

Zudem gibt es folgende Keywords, die für die Python-Syntax reserviert sind und die man sonst nicht verwenden darf (siehe Unterkapitel 5.3 für Anwendungsbeispiele):

Kompakt Keywords für If-Else-Statements

- `if` → beginnt ein if-else-Statement.
- `else` → ist die “dann”-Bedingung, die auf ein `if` folgen kann.
- `elif` → kurz für “else if” und kann nur auf ein `if` folgen. Dient dazu, eine Reihe von Bedingungen zu prüfen.
- `pass` → sagt dem Code, fortzufahren und nichts zu unternehmen, wenn eine bestimmte Bedingung erfüllt ist.

Zum Abschluss seien noch folgende Operatoren erwähnt, die sich für Entscheidungslogik im Programm-Code einbauen lassen (siehe Unterkapitel 5.4 für Anwendungsbeispiele):

Kompakt Logische/Boolesche Operatoren

- `and` → verknüpft zwei Bedingungen mit einem logischen “Und”.
- `or` → verknüpft zwei Bedingungen mit einem logischen “Oder”. Das heisst, nur eine der beiden links und rechts stehenden Bedingungen muss erfüllt sein (im Gegensatz zum “Und”, wo beide erfüllt sein müssen).
- `not` → ein einem `not` folgenden Statement muss falsch (i.e., `False`) sein.

Nun, da wir uns einen Überblick über all die Operatoren und Keywords verschafft haben, geht es darum, diese auch zu nutzen. In der Folge werden alle Keywords und Operatoren kurz illustriert.

5.2.1 Dateien laden

Im XML-Exkurs wurde bereits auf das IRT-Korpus verwiesen. In der Folge arbeiten wir mit dem gesamten Korpus an Inschriften. Es ist im `data`-Ordner unter `IRT2021` verfügbar. Wir lernen jetzt zuerst, wie man mit Python Ordner-Strukturen inspi-

ziert und Dateien aufsammelt. Dazu benutzen wir das Built-in-Module `os`.¹ Dieses erlaubt es uns mit seinen Funktionalitäten, das Dateisystem auf unseren Rechnern zu inspizieren, Pfade zu identifizieren, zu manipulieren und Dateien und Ordner zu erstellen, zu löschen, oder zu kopieren. Wir illustrieren im Code-Beispiel 5.7 den Import und dann das schrittweise Durchiterieren, bis wir bei den Korpus-Dateien sind.

Code-Beispiel 5.7 Beispiel für den Einsatz von `os` um mehr über die Ordnerstruktur des Korpus zu erfahren.

```
>>> import os
>>> os.listdir()
['epidoc']
>>> os.listdir('epidoc')
['IRT0961.xml', 'IRT0768.xml', 'IRT0372.xml', 'IRT0520.xml', 'IRT1575.xml', '
IRT0489.xml', 'IRT0390.xml', 'IRT1296.xml', 'IRT0378.xml', 'IRT0608.xml', '
IRT0189.xml', 'IRT1610.xml', 'IRT0754m.xml', 'IRT1112.xml', 'IRT0278.xml', '
IRT0886b.xml', 'IRT0402.xml', 'IRT1532.xml', 'IRT1201.xml', ...]
>>> xml_files = os.listdir('epidoc')
>>> len(xml_files)
1618
```

Nach dem Import erfahren wir mit `listdir()` welche Ordner und Dateien sich auf der obersten Stufe befinden. Wir finden heraus, dass es im IRT2021-Ordner einen Ordner namens `epidoc` gibt. Inspizieren wir diesen wieder mit `listdir()`, wird uns die gesamte Liste an Korpus-Dateien (in diesem Fall XML-Dateien) zurückgegeben. Wir können diese Liste in der Variable `xml_files` speichern und mit `len()` in Erfahrung bringen, dass das Korpus 1618 Dateien umfasst.

Wollen wir jetzt eine Datei öffnen, so gibt uns Python bereits die Built-in-Funktion `open()` an die Hand. Code-Beispiel 5.8 zeigt die Funktion in Aktion, wenn wir beispielsweise die erste Datei in der Liste öffnen möchten:

Code-Beispiel 5.8 Beispiel für das Öffnen einer XML-Datei

```
>>> f_open = open(xml_files[0], 'r', encoding='utf-8')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'IRT0961.xml'
>>> f_open = open('epidoc/'+xml_files[0], 'r', encoding='utf-8')
>>> f_open
<_io.TextIOWrapper name='epidoc/IRT0961.xml' mode='r' encoding='utf-8'>
```

Der erste Versuch misslingt, da `open()` hier direkt auf der obersten Ebene nach der ersten XML-Datei sucht und natürlich nicht fündig wird. Das ist ein Problem, das einem oft begegnet: Man hat die Ordnerstruktur nicht berücksichtigt und den falschen Pfad zum Öffnen angegeben. Wenn wir den String ``epidoc/'` vor den Dateinamen stellen, dann funktioniert es. Wir haben ein Datei-Objekt, mit dem wir nun arbeiten können. Wichtig zu erwähnen sind noch die Parameter, die wir `open` mitgegeben haben: Das ``r'` steht für “read”, d.h., wir öffnen die Datei zum Lesen. Wir können Dateien auch noch anders öffnen:

¹Siehe <https://docs.python.org/3/library/os.html>.

Kompakt Öffnen von Dateien

- **r** → “read”, die Datei ist nur zum Lesen geöffnet.
- **w** → “write”, die Datei ist zum Schreiben geöffnet. Wenn schon eine Datei mit demselben Namen existiert, wird der Inhalt gelöscht.
- **a** → “append”, die Datei ist so geöffnet, dass alles, was wir herausschreiben, ans Ende der Datei hinzugefügt wird.
- **rw/r+** → “read-write”, die Datei ist zum Lesen und zu Schreiben geöffnet

Der zweite Parameter (**encoding**) gibt an, wie der Text encodiert ist. Wir öffnen hier mit dem Encoding **utf-8**. Wir gehen im nächsten Kapitel im Abschnitt 6.1.1 genauer darauf ein.

Dieses Datei-Objekt hat eine Funktion namens **readlines()**. Diese speichert alle Zeilen in einer Liste, durch welche wir iterieren können:

Code-Beispiel 5.9 Beispiel die Funktion **readlines()**.

```
>>> for line in f_open.readlines():
      print(line)
```

Mit dem obigen Code iterieren wir Zeile für Zeile durch die ganze XML-Datei und geben jede Zeile aus. Das kann für Rohtext-Dateien nützlich sein. Für XML-Dateien gibt es aber nützlichere Wege, sie einzulesen. Wie im Exkurs dieses Kapitels erwähnt, ist XML-strukturierter Text hierarchisch aufgebaut. Wir können ihn als Baum betrachten, an dessen Stamm viele Äste und schliesslich Blätter hängen. Ein solcher Baum hat auch eine Wurzel. Das Built-In Paket **xml** macht sich diese Baum-Metapher zunutze, spricht in der Folge aber auch von “Eltern-Knoten” (EN *parents*) und “Kinder-Knoten” (EN *children*). Code-Beispiel 5.10 zeigt, wie wir eine XML-Datei einlesen, d.h., “parsen” und durch die Elemente iterieren:

Code-Beispiel 5.10 Einlesen einer XML-Datei mit **xml.etree**.

```
>>> import xml.etree.ElementTree as ET
>>> xml_file = ET.parse('epidoc/'+xml_files[0])
>>> root = xml_file.getroot()
>>> print(root)
<Element '{http://www.tei-c.org/ns/1.0}TEI' at 0x7e197ff6ebb0>
```

Das Modul **ElementTree** zum Parsen von XML-Dateien wird üblicherweise wie oben gezeigt importiert. Wir können Modulen mit **as** einen anderen Namen geben (hier **ET**), um nicht ständig den ganzen Modulnamen ausschreiben zu müssen. Wir sehen, dass das Wurzel-Element ein Element namens **TEI** ist. Daran hängen jetzt die ganzen Kinder-Knoten. Dank manueller Inspektion einer solchen Datei wissen wir, dass sich der Text in einem **text**-Element befindet, das mit **lb**-Elementen (für EN *linebreak*) die Zeilenumbrüche markiert. Wir könnten nun also Zeilenumbrüche aufsammeln und so in Erfahrung bringen, wie viele Zeilen diese Inschrift hat.

Code-Beispiel 5.11 Beispiel zur Ermittlung der Anzahl Zeilen.

```
>>> lines = root.findall('.//{http://www.tei-c.org/ns/1.0}lb')
```

```
>>> print(len(lines))
21
```

Wie Code-Beispiel 5.11 verdeutlicht, können wir die `findall()`-Funktion auf der Wurzel des XML anwenden. `./` heisst, dass es in allen Kindern nach diesem Element suchen soll. Würde man nur mit `./` suchen, also nur gerade den Kindern, die der Wurzel entspringen, würde man die Zeilenumbrüche nicht finden. So aber stellen wir fest, dass wir in dieser Inschrift 21 Zeilen vorliegen haben. Schnell können wir nun eine Durchschnittsanzahl an Zeilen berechnen, wenn wir über das ganze Korpus iterieren:

Code-Beispiel 5.12 Beispiel zur Ermittlung des Durchschnitts an Zeilen pro Inschrift.

```
>>> line_counter = []
>>> for xml in xml_files:
...     xml_file = ET.parse('epidoc/'+xml)
...     root = xml_file.getroot()
...     lbs = root.findall('.//{http://www.tei-c.org/ns/1.0}lb')
...     line_counter.append(len(lbs))
...
>>> print(sum(line_counter)/len(line_counter))
5.69
```

5.3 If-Else-Statements

Wie uns das Code-Beispiel 5.12 verrät, ist die durchschnittliche Inschrift in diesem Korpus 5.69 Zeilen lang. Was aber, wenn wir spezifischere Suchen machen wollen, vor allem, was den Text anbelangt? Hier kommen nun If-Else-Statements ins Spiel. Was, wenn wir daran interessiert sind, zu erfahren, wie oft Kaiser Hadrian in den Inschriften vorkommt? In den XML-Dateien sind solche Vorkommnisse wie folgt annotiert: `<persName type="emperor" ref="#hadrianus"><name nymRef="Hadrianus"><supplied reason="lost">H</supplied>adria<supplied reason="lost">ni</su` Um den String “adria”, der auf dem Meilenstein noch lesbar ist, haben die Editoren das “H” und das “ni” ergänzt. Den Namen fasst dann das TEI-standardisierte `persName`-Element ein. Dieses hat ein `ref`-Attribut, das wahrscheinlich auf eine externe XML-Datei verweist, wo die Person “Hadrian” mit der ID `hadrian` gelistet ist. Wir können nun durch alle `persName`-Elemente iterieren und diejenige, welche ein passendes `ref`-Attribut haben, aufsammeln:

Code-Beispiel 5.13 Beispiel zur Ermittlung der Vorkommen von Hadrian.

```
>>> hadrian_counter = 0
>>> for xml in xml_files:
...     xml_file = ET.parse('epidoc/'+xml)
...     root = xml_file.getroot()
...     persNames = root.findall('.//{http://www.tei-c.org/ns/1.0}persName')
...     for persName in persNames:
...         if persName.get('ref') == '#hadrianus':
...             hadrian_counter += 1
```

```
...
>>> print(hadrian_counter)
52
```

Wir schliessen aus dem Code-Beispiel 5.13, dass es 52 annotierte Vorkommen von Hadrian im Inschriftenkorpus gibt. Dadurch, dass die Annotation standardisiert ist, können die Vorkommen beliebig flexiert auftauchen. Was wir hierfür benötigt haben, sind zwei `for`-Schlaufen und ein `if`-Statement. Immer dann, wenn `#hadrianus` gefunden wurde, wurde der `hadrian_counter` um 1 nach oben gesetzt.

Das praktische an `etree` ist, dass wir noch viel einfacher nach den Vorkommen von Hadrian suchen können:

Code-Beispiel 5.14 Beispiel zur Ermittlung der Vorkommen von Hadrian.

```
>>> hadrian_counter_easy = 0
>>> for xml in xml_files:
...     xml_file = ET.parse('epidoc/'+xml)
...     root = xml_file.getroot()
...     hadrians = root.findall(".//{http://www.tei-c.org/ns/1.0}persName[@ref
= '#hadrianus']")
...     hadrian_counter_easy += len(hadrians)
...
>>> print(hadrian_counter_easy)
52
```

Im Gegensatz zu Code-Beispiel 5.13 spart uns Code-Beispiel 5.14 eine komplette `for`-Schleife. Das Einzige, was wir geändert haben, ist das Anfügen einer Spezifikation in eckigen Klammern, die sagt, dass das Element `persName` ein Attribut `ref` enthalten muss, das auf `'#hadrianus'` lautet.²

Wir haben aber trotzdem schon einmal mit `if` arbeiten können. Wir erweitern nun das Repertoire um `elif` und `else`. Wir wollen zum Beispiel wissen, wie viele Inschriften vor Christus datiert werden und wie viele ins erste, zweite oder dritte Jahrhundert fallen.

Code-Beispiel 5.15 Beispiel zur Einteilung der Inschriften in Jahrhunderte.

```
>>> bc = 0
>>> first_century = 0
>>> second_century = 0
>>> third_century = 0
>>> for xml in xml_files:
...     xml_file = ET.parse('epidoc/'+xml)
...     root = xml_file.getroot()
...     date = root.find('.//{http://www.tei-c.org/ns/1.0}origDate')
...     if date == None:
...         continue
...     else:
...         if 'when' in date.attrib:
```

²Beachte, dass, wenn wir Strings innerhalb von Strings definieren, doppelte UND einfache Apostrophe gebraucht werden müssen. Entweder wird der "äussere" String mit einfachen angeführt und der innere dann mit doppelten, oder umgekehrt.

```

...         if date.get('when').startswith('-'):
...             bc += 1
...         else:
...             if '-' in date.get('when'):
...                 new_date = int(date.get('when').split(
-')[0].lstrip('0'))
...             else:
...                 new_date = int(date.get('when').lstrip(
'0'))
...             if 0 <= new_date <= 99:
...                 first_century += 1
...             elif 100 <= new_date <= 199:
...                 second_century += 1
...             else:
...                 third_century += 1
...
>>> print(bc, first_century, second_century, third_century)
3 14 32 98

```

Das Code-Beispiel 5.15 ist schon recht fortgeschritten und erfordert einiges an Trial-and-Error, vor allem dann, wenn man die Daten noch nicht gut kennt. Zuerst muss man herausfinden, in welchem Element das Datum gespeichert ist. Es stellt sich heraus, dass Epidoc `origDate` dafür vorsieht. Aber nicht alle Inschriften verfügen über ein Datum. `root.find` würde deshalb in solchen Fällen `None` in der Variablen `date` abspeichern. Ist dann `date==None`,³ sagen wir mit `continue`, dass der Code direkt zur nächsten Inschrift in der Liste übergehen soll. Alles unterhalb `continue` wird also nicht mehr berücksichtigt.

Als Nächstes ist es so, dass nicht alle `origDate`-Elemente das Attribut `when` enthalten. Wenn sie es aber tun, dann wollen wir zuerst prüfen, ob es sich um eine der vorchristlichen Inschriften handelt. Diese zeichnen sich nämlich dadurch aus, dass dem Datum ein - vorangestellt ist (was wir mit `startswith('-')` prüfen), also zum Beispiel ``-0003'` (für 3 v. Chr.). Wenn es sich um kein solches Datum handelt, dann leiten wir mit `else` zum nächsten Code-Block über. Wir müssen noch einen Spezialfall abfangen. Gewisse Inschriften können mit einem ganz genauen Datum versehen werden (beispielsweise ``0259-07-30'`). Uns interessiert aber nur die Jahreszahl, und ausserdem müssen wir die Jahreszahl als Integer vorliegen haben, damit wir herausfinden können, in welchem Jahrhundert wir uns befinden. Wenn wir nun also ein komplettes Datum vorliegen haben, dann ist unser neues Datum `new_date` das erste (`[0]`) Item nach dem Split an den Bindestrichen (`split('-')`), von welchem wir mit `lstrip('0')` alle vorangestellten Zahlen 0 entfernt haben. Mit `int()` wandeln wir den so entstandenen Zahlenstring in einen Integer um. Wenn allerdings kein Bindestrich vorkommt, entfernen wir die Nullen direkt und wandeln den String in einen Integer um.

Nun startet ein neues If-Else-Statement, mit welchem wir die zuvor initiierten Zähler (`bc`, `first_century`, `second_century`, `third_century`) jeweils um 1 ergänzen, wenn ein Datum in ein bestimmtes Jahrhundert fällt. Wir greifen hier jetzt

³Wir benutzen hier einen booleschen Operator, der im nächsten Unterkapitel genauer erläutert wird.

schon den Vergleichen aus dem Kapitel 5.4 vor. Wir werden im folgenden Abschnitt auf dieses Beispiel zurückkommen und uns die Funktionsweise genauer anschauen.

Wenn wir uns die Verteilung über die Jahrhunderte anschauen, sehen wir, dass wir für spätere Jahrhunderte mehr Inschriften vorliegen haben. Wir merken uns (vorerst) abschliessend für If-Else-Statements folgende Eigenheiten:

- Ein `if` braucht nicht zwingend ein `else`.
- Die Verwendung von `elif` und `else` ist nur möglich, wenn mit einem `if` begonnen wurde.

5.3.1 Video zu If-Else-Statements

 Schaue dir jetzt das [Video 5.3.1](#) zu If-Else-Statements an und bearbeite dann das nächste Unterkapitel.

5.4 Vergleiche und Boolesche Operatoren

Wir haben im Code-Beispiel 5.15 bereits zwei Vergleiche verwendet: `==` und `<=`. Den Operator `==` kann man auch durch das Keyword `is` ersetzen. `if date is None:` meint also das gleiche wie `if date == None`. Gleichzeitig gilt auch, dass `is not` dem `!=` entspricht. Mit `<=` konnten wir den Bereich markieren, in den eine Jahreszahl fallen musste, um, zum Beispiel, zum ersten Jahrhundert zu gehören. Dazu musste die Bedingung `if 0 <= new_date <= 99` erfüllt sein, oder in Worten ausgedrückt: “0 ist kleiner oder gleich dem Wert in der Variablen `new_date` und dieser ist wiederum kleiner oder gleich 99.” Das ist gerade ein Beispiel, das zeigt, wie mehrere Bedingungen miteinander verknüpft werden können.

Auch boolesche Operatoren können dazu verwendet werden, um Aussagen miteinander zu verknüpfen. Wir illustrieren die Funktionsweise der Booleschen Operatoren gleich wieder an den Inschriften, aber dieses Mal verknüpfen wir Information zum Datum mit Informationen zum Material, welche der Inschrift als Träger dient:

Code-Beispiel 5.16 Beispiel zum Auffinden der Inschriften welche eine Angabe zum Datum und zum Material haben.

```
>>> for xml in xml_files:
...     xml_file = ET.parse('epidoc/'+xml)
...     root = xml_file.getroot()
...     material = root.find('.//{http://www.tei-c.org/ns/1.0}material')
...     date = root.find('.//{http://www.tei-c.org/ns/1.0}origDate')
...     if date != None and material != None:
...         if date.get('when') != None:
...             print(date.get('when'), material.text)
...
0216 limestone
0157 marble
0200 brown limestone
0120 marble
0222 limestone
```

```
0199 marble
0153 limestone
# hier abgeschnitten aufgrund der langen Ausgabe
```

Das Code-Beispiel 5.16 zeigt in der drittletzten Zeile, wie ein **and** zwei Aussagen miteinander verbindet. Damit erreichen wir, dass zum Schluss nur Jahresdaten und Material ausgegeben zu Inschriften ausgegeben werden, zu welchen beide Informationen vorliegen. Es muss also ein **origDate**-Element vorliegen (das gleichzeitig auch noch das Attribut **when** führt) sowie ein Element vorhanden sein, das **material** heisst. Davon können wir mit **.text** auf den Text zugreifen. Wir finden dann, dass gewisse Inschriften auf Marmor angefertigt wurden oder, was recht häufig war, Kalkstein, der noch in verschiedene Kategorien eingeteilt wird (brauner, gelber, grauer, etc.).

Betrachten wir noch eine Variante des obigen Codes, um die Funktionsweise von **or** und **not** zu erläutern:

Code-Beispiel 5.17 Beispiel zum Auffinden von Inschriften, welche entweder aus (normalem) Kalkstein oder Marmor bestehen.

```
>>> import re
>>> for xml in xml_files:
...     xml_file = ET.parse('epidoc/'+xml)
...     root = xml_file.getroot()
...     date = root.find('.//{http://www.tei-c.org/ns/1.0}origDate')
...     material = root.find('.//{http://www.tei-c.org/ns/1.0}material')
...     if not material == None:
...         if re.match(r'^[ll]limestone$', material.text) or re.match('^[mM]arble$', material.text):
...             print(material.text)
...
...
limestone
marble
marble
limestone
Limestone
marble
limestone
```

Hier sehen wir in Code-Beispiel 5.17, wie wir nur nach Inschriften filtern können, welche ENTWEDER aus Kalkstein ODER aus Marmor gefertigt sind. Durch **^** und **\$** verhindern wir, dass Strings gematcht werden, welche lediglich “limestone” oder “marble” enthalten (wie oben “brown limestone”). Um mit **or** zu operieren, muss uns also bewusst sein, dass, wenn wir es mit **if** kombinieren, der darauf folgende Code-Block nur ausgeführt wird, wenn entweder das Statement links oder dasjenige, das rechts des **or** steht, wahr ist.

Zudem steht oberhalb des **or**-Statements ein leicht modifizierter Ausdruck aus dem Code-Beispiel ???: Um sicherzustellen, dass im XML auch Informationen über das Material vorhanden sind, prüfen wir, ob dazu etwas in der Variablen **material** abgespeichert worden ist (das **find()** also erfolgreich war). Das tun wir ausgedeutet so: “Wenn nicht der Wert von **material** **None** entspricht”. Wenn das zu

True evaluiert, wird der Code-Block unter dem `if`-Statement ausgeführt.

5.4.1 Video zu Vergleichen und Booleschen Operatoren

 Schaue dir jetzt das [Video 5.4.1](#) zu den Vergleichen und Booleschen Operatoren an und bearbeite dann das Quiz in OLAT.

5.5 Quiz zu If-Else-Statements und Entscheidungslogik

☒ Löse das [Quiz auf OLAT](#) und bearbeite dann das nächste Kapitel.

Kapitel

6

Funktionen

Sich selbst aber denkt das Denken im Ergreifen des Denkbaren.

Aristoteles, *Metaphysik*, 12,1072b

Lernziele Kapitel 6: Funktionen

- 1. Verstehen** Die Studierenden können den Zweck von *Funktionen* erläutern und ihre Bedeutung für die Strukturierung und Wiederverwendbarkeit von Code erklären.
- 2. Analysieren** Die Studierenden können Parameter, Argumente und Rückgabewerte unterscheiden und verstehen, wie diese das Verhalten einer Funktion bestimmen.
- 3. Anwenden** Die Studierenden können eigene Funktionen definieren, aufrufen und in historischen Analyseaufgaben (z. B. Normalisierung von Datumsangaben) einsetzen.
- 4. Bewerten** Die Studierenden können die Lesbarkeit und Effizienz von Funktionsdefinitionen einschätzen und begründen, wann eine Funktion sinnvoll ist.
- 5. Erzeugen** Die Studierenden können Programme entwickeln, die mehrere Funktionen kombinieren, um historische Fragestellungen modular zu bearbeiten.

6.1 Der Repetition entfliehen

Ständig immer wieder das Gleiche zu machen, kann relativ mühsam werden. So ist es auch beim Coden: Wenn sich gewisse Funktionalitäten ständig wiederholen und quasi immer dasselbe auf anderen Daten ausgeführt werden muss, dann wird es mühsam,

immer denselben Code hinzuschreiben. Der Code wird dadurch auch sehr schnell unleserlich, da er unüberblickbare Dimensionen annimmt. Dazu kommt: Merkt man im Nachhinein, dass man irgendetwas am Code ändern muss, muss man das an jeder Stelle machen, wo man denselben Code-Abschnitt hinkopiert hat. Zum Glück schafft hier Python mit dem Konstrukt der Funktionen Abhilfe! Diese rücken nun in den Fokus dieses Kapitels. Dabei ist es wichtig, zu betonen, dass wir in den vorherigen Kapiteln schon eine ganze Reihe von Funktionen angewendet haben. Was nun neu dazu kommt, ist, dass wir lernen, wie wir unsere eigenen Funktionen definieren. Bevor wir aber zu den Funktionen kommen, wollen wir uns in einem kurzen Exkurs noch den Encodings widmen.

Exkurs

6.1.1 Datei-Encodings

Wir sind im Exkurs des Kapitels 5 bereits über den Begriff des Encodings gestolpert. Da steht beispielsweise irgendetwas von `utf-8` in der XML-Deklaration. Als Nächstes haben wir eine Datei mit demselben Encoding und der `open()`-Funktion geöffnet. Was steckt aber dahinter?

- Um das zu verstehen, ist es wichtig, sich zu vergegenwärtigen, wie eine Textdatei für den Computer aussieht. Rufen wir uns vor Augen, dass ein Computer im Prinzip nur Nullen und Einsen lesen kann. Das heisst, dass er auch einen Text so **encodieren** muss. Es gilt also, für jedes Schriftzeichen eine Kombination aus Nullen und Einsen zu finden, sodass beim **Decodieren**, also immer dann, wenn uns zum Beispiel Text angezeigt wird, die richtigen Buchstaben stehen.

Dabei können verschiedene Formate einen bestimmten Bereich von Zeichen abdecken. Eines der ersten Encoding-Systeme war das sogenannte ASCII-System.^a Es wurde entwickelt, um das englische Alphabet und einen Grossteil von Satz- und Sonderzeichen abzudecken. Insgesamt wurden in ASCII 127 Zeichen definiert.

- Wir haben oben erwähnt, dass der Computer mit Nullen und Einsen operiert. Das heisst, er arbeitet mit dem sogenannten **Binär-System** (wir sind uns aus der Schule und dem Alltag das Dezimalsystem gewohnt) und repräsentiert alles als eine Folge von 0 und 1. Wenn wir nun also eine 2 darstellen wollen, dann reicht nur eine Stelle nicht mehr aus und wir fügen eine hinzu: 10 heisst in diesem Fall: 2, da die zweite Stelle von rechts für die 2 steht. Die 3 repräsentieren wir mit 11, da $2+1 = 3$. Für die 4 reichen zwei Stellen nicht mehr, also repräsentieren wir sie so: 100, oder $4+0+0 = 4$, die 5 dementsprechend 101, da $4+0+1 = 5$, usw. Für das 127 Zeichen umfassende ASCII-Encoding brauchen wir also sieben Stellen, da die achte bereits die Zahl 128 repräsentieren würde. Die ASCII-Encodings sehen binär also so aus: 000 0000, wobei die Stellen für 64 32 16 8 4 2 1 stehen. 111 1111 entspricht demnach 127 (der letzten Encodierung im ASCII-Encoding).

Schauen wir uns ein Beispiel an:

Code-Beispiel 6.1 ASCII-Encoding eines Strings.

```
>>> t = "Gallia est omnis divisa in partes tres, quarum unam incolunt"
```

```

Belgae, aliam Aquitani, tertiam qui ipsorum lingua Celtae, nostra Galli
appellantur."
>>> ascii = ' '.join(str(byte) for byte in t.encode('ascii'))
>>> print(ascii)
71 97 108 108 105 97 32 101 115 116 32 111 109 110 105 115 32 100 105 118
105 115 97 32 105 110 32 112 97 114 116 101 115 32 116 114 101 115 44 32
113 117 97 114 117 109 32 117 110 97 109 32 105 110 99 111 108 117 110 116
32 66 101 108 103 97 101 44 32 97 108 105 97 109 32 65 113 117 105 116 97
110 105 44 32 116 101 114 116 105 97 109 32 113 117 105 32 105 112 115
111 114 117 109 32 108 105 110 103 117 97 32 67 101 108 116 97 101 44 32
110 111 115 116 114 97 32 71 97 108 108 105 32 97 112 112 101 108 108 97
110 116 117 114 46

```

Das Code-Beispiel 6.1 zeigt, wie wir einen String zuerst in ASCII encodieren und danach den Wert jedes 7-Bits ausgeben. Die Folge ist eine Reihe von Zahlen. Wenn wir in der ASCII-Tabelle nachschauen, entspricht 71 dem grossen G, 97 dem kleinen a, die 108 dem l und die 32 dem Space-Character (also dem Abstand). Jeder Buchstabe entspricht also einer Zahl. Somit können wir bereits das erste Wort *Gallia* in ASCII repräsentieren. Da das Lateinische grösstenteils ohne Sonderzeichen auskommt, liesse sich lateinischer Text also ohne Probleme in ASCII encodieren.

Viele Sprachen haben aber weitere Sonderzeichen. Nehmen wir nur schon Französisch als zusätzliche Sprache, so hat man über fast allen Vokalen die Möglichkeit, *aigu*, *grave* und *circumflexe* zu setzen, beispielsweise é, à oder ô, zusätzlich zum Trema wie in ë oder dem *cedille* ç. Das erfordert die Schaffung von weiteren Stellen. Deshalb folgte als eine logische Erweiterung der ASCII-Encodierung *Latin-1*, die mit 8 Bit, also einem kompletten Byte, operiert. Mit einem Byte kann man nun 255 Zeichen abdecken. Das reicht aber immer noch nicht, wenn wir alle Zeichen aus allen Schriftsystemen encodieren möchten. Und für jedes Schriftsystem eine eigene Kodierung zu schaffen, ist (1) zu aufwändig und führt (2) zu Konvertierungsproblemen.

Aus diesen Gründen wurde der sogenannte UTF-8-Standard ins Leben gerufen. Dieser encodiert jedes Zeichen der Unicode-Tabelle in einem bis maximal vier Bytes. Dabei entspricht beispielsweise die Encodierung der ersten 127 in UTF-8 derjenigen der ASCII-Encodierung.

Beim Einlesen von Dateien ist es wichtig, das Encoding zu beachten. Wenn wir eine Datei mit dem falschen Encoding öffnen, kann das zu Fehlern führen, wie hier an einem einfachen String illustriert:

Code-Beispiel 6.2 Encoding-Fehler, da der String mit dem falschen Encoding (hier ASCII) encodiert wird.

```

>>> t = "Δαρείου καὶ Παρυσάτιδος γίνονται παῖδες δύο, πρεσβύτερος μὲν
Ἄρταξέρξης, νεώτερος δὲ Κῦρος: ἐπεὶ δὲ ἡσθένει Δαρεῖος καὶ ὑπώπτει
τελευτὴν τοῦ βίου, ἐβούλετο τῷ παιδί ἀμφοτέρω παρεῖναι."
>>> ascii = ' '.join(str(byte) for byte in t.encode('ascii'))
Traceback (most recent call last):
File "<stdin>", line 1, in <module> UnicodeEncodeError: 'ascii' codec can'

```



```
x96f'Îµ á¼\x80f¼Ï\x86f¼Ï\x84f\xadÏ\x81Ï\x89 Ï\x80f±Ï\x81fµá¼\x96f½f±f¹.'
```

Der Text ist vollkommen unkenntlich geworden. Es ist also wichtig, sich bewusst zu sein, mit welchen Daten man arbeitet und wie diese encodiert sind. Das zählt insbesondere für das Altgriechische.

^aSiehe <https://en.wikipedia.org/wiki/ASCII>.

6.2 Funktionen in Python – Definition & Aufruf

Wie schon eingangs erwähnt, sind Funktionen dafür da, immer wieder auftretenden Code zu operationalisieren, sodass das Programm insgesamt übersichtlicher und besser zu verändern ist.

6.2.1 Funktionsdefinition (def), Signatur, Aufruf

Kümmern wir uns als Erstes darum, wie man eine Funktion definiert. Das Keyword, das einer Funktion vorangeht, ist `def`. Danach folgt der Funktionsname. Wir kennen bereits einige Funktionen, die in Python immer abrufbar sind, zum Beispiel `len()` zum Berechnen der Länge von Strings und Listen, `print()` um etwas auszugeben, oder `str()` und `int()` für die Typenkonversion. Eigens definierte Funktionsnamen können beliebig gewählt werden. Es gelten dieselben Regeln wie für die Wahl von Namen von Variablen (siehe Kapitel 2.2.1). Danach folgen runde Klammern `()`, in denen die Argumente aufgelistet werden. Die Argumente werden innerhalb der Funktion dann verarbeitet. Der abschliessenden runden Klammer für die Definition der Argumente folgt ein Doppelpunkt. Darunter folgt um einen Tab oder vier Leerzeichen eingerückt der “Funktionskörper.” Dieser drückt aus, was mit den Argumenten geschieht. Die Funktion kann abschliessend ein `return`-Statement enthalten. Dies bedeutet, dass sie einen Wert oder ein Objekt zurückliefert. Schauen wir uns ein simples Beispiel an, wie wir selbst die Länge eines Strings berechnen könnten:

Code-Beispiel 6.5 Selbst definierte Funktion für die Berechnung der Länge eines Strings.

```
>>> name = "Gaius Iulius Caesar"
>>> def string_length(our_string):
...     counter = 0
...     for character in our_string:
...         counter += 1
...     return counter
...
>>> print(string_length(name))
19
>>> print(len(name))
19
```

Das Code-Beispiel 6.5 zeigt, wie wir eine Funktion definieren können, die ein Argument (nämlich `our_string`) akzeptiert. In der Funktion wird ein Zähler initiiert. Danach iteriert ein `for`-Loop über den String und zählt die Buchstaben, während

nach jedem Buchstaben der Zähler um 1 erhöht wird. Schliesslich wird mit `return counter` der Zähler ausgeliefert. Das ist sozusagen das Resultat der Funktion. Wenn wir nun diese Funktion auf dem zuvor definierten Namen anwenden, also aufrufen, erhalten wir als Antwort, die Leerzeichen mit einschliesst, 19, was dasselbe Resultat ist, wie es die Built-In-Funktion `len()` zurückliefert.

Es ist guter Stil, genau zu dokumentieren, was für Arten von Argumenten die Funktion erwarten. In älteren Python-Versionen wurden den jeweiligen Definitionen eine sogenannte **Signatur** (in EN auch *docstring* genannt) angefügt. Die Signatur ist ein String, dessen Struktur vorgegeben ist und der jedes Argument aufgreift und erklärt. Diese Praxis ist heute noch so umsetzbar. Das Code-Beispiel 6.6 zeigt die Funktion `string_length` mit Signatur:

Code-Beispiel 6.6 Selbst definierte Funktion für die Berechnung der Länge eines Strings inklusive Signatur.

```
>>> def string_length(our_string):
...     """Function to compute the length of a string.
...
...     Args:
...         our_string (str): String for which length is computed.
...     """
...     counter = 0
...     for character in our_string:
...         counter += 1
...     return counter
...
```

Die Signatur wird direkt im Anschluss an die Definition mit einem langen String, den wir zwischen drei Anführungszeichen `"""..."""` schreiben, spezifiziert. Nachdem die Funktion so definiert wurde, ist die Signatur mit der `__doc__`-Funktion abrufbar:

Code-Beispiel 6.7 Ausgabe der Signatur.

```
>>> print(string_length.__doc__)
Function to compute the length.

    Args:
        our_string (str): String.
```

In neueren Python-Versionen wird ein Teil der Signatur direkt in die Funktionsdefinition integriert.

Code-Beispiel 6.8 Selbst definierte Funktion für die Berechnung der Länge eines Strings inklusive Signatur für neuere Python-Versionen.

```
>>> def string_length(our_string: str) -> int:
...     counter = 0
...     for character in our_string:
...         counter += 1
...     return counter
...
>>> import inspect
```

```
>>> inspect.signature(string_length)
<Signature (our_string: str) -> int>
```

Code-Beispiel 6.8 zeigt, wie wir definieren, von welchem Typ `our_string` sein muss, damit der Funktionsaufruf funktioniert. Diese Typenspezifikation folgt nach dem Argument, abgetrennt durch ein Komma. Des Weiteren geben wir an, welchen Output-Typen wir erwarten. Dazu hängen wir an die Argumentenklammer `-> int` an. Dadurch, dass wir also keinen *Docstring* haben, würde die Funktion `__doc__` ein `None` retournieren. Das Code-Beispiel zeigt aber gleichzeitig, wie es möglich ist, eine Beschreibung der Funktionssignatur zu erhalten. Das Modul `inspect` hat eine `signature`-Funktion, die beim Aufruf die Signatur wiedergibt.

6.2.2 Argumentarten kompakt

Wichtig ist zunächst, dass wir beachten, dass wir der Funktion so viele Argumente liefern, wie sie erwartet. In unserem obigen Beispiel ist das exakt ein Argument. Würden wir zwei liefern, würde dies in einem Fehler resultieren. Des Weiteren unterscheiden wir zwischen drei Arten von Argumenttypen:

1. **Positionsargumente:** Sind die Argumente nicht speziell gekennzeichnet, erwartet die Funktion die Argumente in der Reihenfolge, in der sie in der Funktionsdefinition auftauchen. Das folgende Code-Beispiel illustriert, was passiert, wenn Argumente nicht in der richtigen Reihenfolge verwendet werden.

Code-Beispiel 6.9 Positionsargumente

```
>>> def roman_names(praenomen, nomen_gentile, cognomen):
...     return f"{praenomen} {nomen_gentile} {cognomen}"
...
>>> roman_name("Marcus", "Tullius", "Cicero")
'Marcus Tullius Cicero'
>>> roman_name("Tullius", "Cicero", "Marcus")
'Tullius Cicero Marcus'
```

Gleichzeitig ist zu betonen, dass die Funktion `roman_name` genau drei Argumente erwartet.

2. **Benannte Argumente:** Diese Argumente machen die Zuordnung von Werten zu Argumenten spezifisch, was im Umkehrschluss bei der Anwendung eine beliebige Reihenfolge zulässt. Wir können das obige Beispiel dahingehend modifizieren:

Code-Beispiel 6.10 Benannte Argumente

```
>>> def roman_names(praenomen='', nomen_gentile='', cognomen=''):
...     return f"{praenomen} {nomen_gentile} {cognomen}"
...
>>> roman_name(praenomen="Marcus", nomen_gentile="Tullius", cognomen="Cicero")
'Marcus Tullius Cicero'
```

```
>>> roman_name(nomen_gentile="Tullius", cognomen="Cicero", praenomen="
Marcus")
'Marcus Tullius Cicero'
```

Wir definieren die verschiedenen Namensbestandteile als leere Strings. Somit erhalten wir immer eine Ausgabe, auch wenn wir diese nicht definieren. Mit den Keywords können wir nun aber die Argumentreihenfolge beliebig gestalten. Als beispielsweise vor 200 v. Chr. die Verwendung des Cognomens noch nicht üblich war, lässt es diese Funktion zu, nur mit zweiteiligen Namen zu operieren.

3. **Arbiträre Argumente:** Damit wird spezifiziert, dass die Anzahl der Argumente beliebig sein kann. Wir verwenden dazu das Asterisk *. Wiederum auf das obige Beispiel angewandt, könnte diese Funktion so aussehen:

Code-Beispiel 6.11 Arbiträre Argumente

```
>>> def roman_names(*names):
...     return " ".join(name for name in names)
...
>>> roman_name("Marcus", "Tullius", "Cicero")
'Marcus Tullius Cicero'
>>> roman_name("Tullius", "Cicero", "Marcus")
'Tullius Cicero Marcus'
>>> roman_name("Marcus", "Tullius")
'Marcus Tullius'
```

Die Funktion erhält in diesem Fall die Namen, oder eben Argumente, als Tupel, mit welchem man in der Folge arbeiten kann. Hier nutzen wir die `join()` Funktion über einer aus dem Tupel generierten Liste.

Argumenttypen können gemischt werden. Es ist beispielsweise möglich, dass auf zwei Positionsargumente drei benannte Argumente folgen.

6.2.3 Rekursivität

Eine Funktion kann nicht nur andere Funktionen aufrufen, sondern auch sich selbst. Dieses Verhalten nennen wir **Rekursivität**. Eine rekursive Funktion ruft also innerhalb ihres eigenen Funktionskörpers erneut dieselbe Funktion auf und dies solange, bis eine definierte Abbruchbedingung erreicht ist.

Rekursion ist ein elegantes Werkzeug, um Probleme zu lösen, die sich in kleinere Teilprobleme derselben Struktur zerlegen lassen. Typische Beispiele sind das Durchlaufen von Verzeichnisbäumen, das Berechnen von Fakultäten oder das Zählen von Elementen in verschachtelten Listen.

Das Prinzip ist immer gleich:

1. **Abbruchbedingung:** Bestimmt, wann die Rekursion aufhört.
2. **Rekursiver Aufruf:** Ruft die Funktion mit einem “kleineren” oder “einfacheren” Argument erneut auf.

Das folgende Beispiel zeigt eine klassische rekursive Funktion zur Berechnung der Fakultät einer Zahl n (also $n!$). Die Fakultät ist definiert als $n! = n \times (n - 1)!$, mit der Basis $1! = 1$.

Code-Beispiel 6.12 Einfache rekursive Funktion zur Berechnung der Fakultät.

```
>>> def fakultaet(n):
...     """Berechnet die Fakultät einer Zahl n rekursiv."""
...     if n == 1:
...         return 1
...     else:
...         return n * fakultaet(n - 1)
...
>>> print(fakultaet(5))
120
```

Im Code-Beispiel 6.12 sehen wir, dass sich die Funktion `fakultaet()` selbst aufruft, solange `n` grösser als 1 ist. Sobald `n == 1` erreicht ist, greift die Abbruchbedingung, und die Rekursion endet. Die Ergebnisse der einzelnen Aufrufe werden in der Rückgabekette multipliziert.

Ein weiteres, etwas anschaulicheres Beispiel stammt aus der Geschichtsforschung: Wir wollen alle übergeordneten Institutionen einer archivalischen Sammlung ausgeben, bis wir die oberste erreicht haben. Die hierarchische Struktur liesse sich rekursiv durchlaufen:

Code-Beispiel 6.13 Rekursive Durchsuchung einer hierarchischen Struktur.

```
>>> archives = {
...     "Archiv der Stadt Rom": None,
...     "Museo Epigrafico": "Archiv der Stadt Rom",
...     "Sammlung CIL": "Museo Epigrafico"
... }
...
>>> def print_parent(archive):
...     """Gibt die übergeordneten Archive rekursiv aus."""
...     parent = archives[archive]
...     if parent is None:
...         print(f"{archive} ist das oberste Archiv.")
...     else:
...         print(f"{archive} gehört zu {parent}.")
...         print_parent(parent)
...
>>> print_parent("Sammlung CIL")
Sammlung CIL gehört zu Museo Epigrafico.
Museo Epigrafico gehört zu Archiv der Stadt Rom.
Archiv der Stadt Rom ist das oberste Archiv.
```

Die Funktion `print_parent()` ruft sich so lange selbst auf, bis das oberste Archiv (also die Basisbedingung `None`) erreicht ist. So lässt sich elegant eine hierarchische Struktur von unten nach oben durchlaufen.

Ein häufiger Fehler bei rekursiven Funktionen ist eine fehlende oder falsch gesetzte Abbruchbedingung. In solchen Fällen ruft sich die Funktion endlos selbst auf,

bis der Speicher erschöpft ist. Python gibt dann eine Fehlermeldung aus:

```
RecursionError: maximum recursion depth exceeded
```

Rekursionen sind mächtig, aber nicht immer die effizienteste Lösung. Wo immer möglich, sollte man prüfen, ob sich ein Problem auch mit einer Schleife lösen lässt, da diese in Python meist performanter sind. Dennoch ist die Rekursion ein zentrales Konzept des algorithmischen Denkens.

6.2.4 Gute Praxis, kompakt

Wir wollen hier noch ein paar Tipps geben, wie man vorbildlich mit Funktionen umgeht:

Kompakt Tipps

- **Eine Aufgabe pro Funktion:** Kleine, klar benannte Funktionen, z. B. `normalize_date()` oder `extract_names()`.
- **Sprechende Namen:** Verben für Funktionen, Nomen für Daten.
- **Benannte Argumente & Defaults:** Erhöhen Lesbarkeit und Robustheit.
- **Dokumentieren:** Kurz-Docstring (13 Zeilen) mit Zweck und Rückgabe.
- **Rückgabe klar halten:** Eindeutige Typen; lieber `None` klar abfangen.
- **Kleine Checks im Code:** `if x is None: return ...` (Guard Clauses).

6.2.5 Video zur Einführung in die Funktionen

 Schaue dir jetzt das [Video 6.2.5](#) zur Einführung in die Funktionen an und bearbeite dann das nächste Unterkapitel.

6.3 Quiz zu Funktionen

- ☑ Löse das [Quiz auf OLAT](#) und bearbeite dann das nächste Kapitel.

Klassen und objektorientierte Programmierung

Et quidem summa divisio de iure personarum haec est, quod omnes homines aut liberi sunt aut servi.

Gaius, *Institutiones*, I,9.

Lernziele Kapitel 7: Klassen und objektorientierte Programmierung

- 1. Verstehen** Die Studierenden können erklären, was Klassen und Objekte sind, und deren Rolle in der objektorientierten Programmierung beschreiben.
- 2. Analysieren** Die Studierenden können den Unterschied zwischen Instanz- und Klassenvariablen sowie zwischen Methoden und Attributen nachvollziehen und einordnen.
- 3. Anwenden** Die Studierenden können eigene Klassen mit Attributen und Methoden definieren, Objekte instanziiieren und diese in kleinen Beispielen gezielt einsetzen.
- 4. Bewerten** Die Studierenden können die Vorteile und Grenzen der objektorientierten Programmierung im Vergleich zu prozeduralem Code einschätzen, insbesondere im Hinblick auf historische Datenanalysen.
- 5. Erzeugen** Die Studierenden können Programme entwickeln, die Vererbung und das Überschreiben von Methoden nutzen, um komplexere Strukturen für historische Forschungsfragen abzubilden.

7.1 Zwei Paradigmen

Gerade im vorherigen Kapitel haben wir die Funktionen kennengelernt. Diese spielen auch in diesem Kapitel, das sich auf Klassen und objektorientierte Programmierung (OOP) bezieht, wieder eine zentrale Rolle. Jedoch sei an dieser Stelle noch auf zwei unterschiedliche Traditionen—oder besser: Paradigmen—verwiesen. Unter den Programmierer:innen gibt es (mindestens) zwei Sorten: die Vertreter:innen des **Functional Programming** und jene der **objektorientierten Programmierung**.

7.2 Was sind Klassen und Objekte?

Der Mensch hatte schon immer das Bedürfnis, seine Welt zu kategorisieren und zu klassifizieren. Schon Aristoteles hat sich in den *Kategorien* darüber Gedanken gemacht, alles Seiende in Kategorien zu ordnen. Und auch Claude Lévi-Strauss führt aus, dass die Forderung nach Ordnung die Grundlage allen Denkens ist (vgl. 1973, S. 21). Wir bedienen uns in der Folge eines Beispiels aus der Antike. Gaius ist ein Rechtsgelehrter aus dem 2. Jhd. n. Chr. Mit seiner Schrift *Institutiones*, die als Lehrschrift für Anfänger in der Juristerei gedacht war, ist ein bedeutendes Werk erhalten geblieben, das uns wertvolle Einsichten in die Funktionsweise des römischen Rechts liefert. Es behandelt in vier Buchrollen verschiedene Aspekte des Privat- und Zivilprozessrechts.

Wir widmen uns dem ersten Abschnitt, der sich hauptsächlich um das Personenrecht dreht. Hier finden wir das Eingangszitat wieder, woraus wir lernen, dass alle Menschen grundsätzlich entweder zu den Freien oder zu den Sklaven gehören. Ihnen gemeinsam ist, dass beide Menschen sind. Gaius hat hier jedoch bereits eine Klassifizierung vorgenommen. Dass grundsätzlich alle Menschen auch der Klasse der Menschen angehören, können wir in Python so abbilden, wie in Code-Beispiel 7.1 erklärt:

Code-Beispiel 7.1 Eine minimale Klasse Homo ohne Attribute oder Methoden.

```
>>> class Homo:
...     pass
...
>>> marcus = Homo()
>>> tiro = Homo()
>>> marcus
<class '__main__.Homo'>
>>> type(marcus)
<class '__main__.Homo'>
```

Das ist ein minimales Beispiel, das zeigt, wie man in Python Klassen definieren kann. Entscheidend ist das Keyword `class`, auf welches der Klassenname folgt. Nach einem Doppelpunkt, wie bei Funktionsdefinitionen, folgt eingerückt die Definition der Klasse. Das Keyword `pass`, das übrigens auch in Funktionen verwendet werden kann, bedeutet hier, dass eine leere Klasse erstellt wird. Wir können aber die Klassen trotzdem schon instanziiieren. Wir erstellen mit `marcus` und `tiro` zwei Objekte der Klasse `Homo`. Beachten müssen wir hier lediglich, dass wir die runden Klammern `()`

zur Erstellung von Objekten verwenden. So erzeugen wir ein Abbild des Bauplans der Klasse im Speicher.

7.2.1 Einführungsvideo zu Klassen

 Schaue dir jetzt das [Video 7.2.1](#) zu den Klassen generell an.

7.3 Konstruktor `__init__` und Instanziierung

Die Instanziierung erzeugt also ein referenzierbares Objekt. Dabei wird intern die `__init__`-Funktion aufgerufen (wir kommen in Unterkapitel 7.8 noch darauf zu sprechen, wieso hier diese Unterstriche nötig sind). Diese definiert, welche Argumente bei der Instanziierung des Objekts mitgegeben werden müssen. Wir können nun im Code-Beispiel 7.2 bereits eine Unterscheidung zwischen den erwähnten *liberi* und *servi* machen.

Code-Beispiel 7.2 Grundklasse `Homo` mit Name und Status.

```
>>> class Homo:
...     def __init__(self, name, status):
...         self.name = name
...         self.status = status
...
>>> marcus = Homo("Marcus", "liber")
>>> tiro = Homo("Tiro", "servus")
>>> marcus
<__main__.Homo object at 0x72a086f72780>
>>> marcus.status
'liber'
>>> tiro.status
'servus'
```

Wir sehen hier, dass wir die beiden Objekte `marcus` und `tiro` im Gegensatz zum Code-Beispiel 7.1 mit zwei Argumenten instantiieren, nämlich dem Namen und dem Status. Das sind auch exakt die Argumente, die `__init__` verlangt. Zwar gibt es noch dieses Argument `self`; dieses erfüllt aber einen völlig anderen Zweck. Es referenziert zurück auf die Klasse selbst. Indem wir `self.name = name` definieren, legen wir innerhalb der Klasse `Homo` sozusagen eine Schublade an, in welcher wir den Namen abspeichern. Diese Schublade ist mit `self.name` innerhalb der kompletten Klassendefinition und, sobald die Klasse instantiiert ist, mittels `.name` abrufbar.

Im Unterschied zum Code-Beispiel 7.1 wird hier auch deutlich, dass nach der Instantiierung der Klasse ein Objekt vorliegt, das an einer spezifischen Adresse gespeichert ist.

7.3.1 Video zu Konstruktor `__init__` und Instanziierung

 Schaue dir jetzt das [Video 7.3.1](#) zum Konstruktor `__init__` und Instanziierung an.

7.3.2 Mutierbarkeit von Objekten

Im Kapitel zu den Variablen haben wir kurz die Mutierbarkeit von Objekten erwähnt. Wir nehmen diese hier noch kurz genauer unter die Lupe. Wir unterscheiden generell zwischen mutierbaren und nicht mutierbaren Objekten. Code-Beispiel 7.3 zeigt im ersten Teil, was wir schon wissen, nämlich dass Strings sich nicht verändern, auch wenn man String-Methoden darauf anwendet. Damit das Resultat von `.upper()` gespeichert werden würde, müsste man es in einer neuen Variablen anlegen (oder die alte `name`-Variable wiederverwenden).

Code-Beispiel 7.3 Mutierbarkeit: Strings sind unveränderbar, Objekte können verändert werden.

```
>>> # Strings: unveränderbar
>>> name = "Marcus"
>>> name.upper()
'MARCUS'
>>> name
'Marcus'      # bleibt unverändert

>>> # Objekte: veränderbar
>>> class Homo:
...     def __init__(self, name, status):
...         self.name = name
...         self.status = status
...
>>> marcus = Homo("Marcus", "liber")
>>> marcus.status
'liber'
>>> marcus.status = "servus"    # Attribut ändert sich
>>> marcus.status
'servus'
```

Im Falle des Status von Marcus kommt die Mutierbarkeit zum Tragen. Wir haben ja in der Schublade zum Status als erstes `liber` gespeichert. Mit `.status` sprechen wir die Schublade nun direkt an und mit `= "servus"` sagen wir, dass wir den Status zum Sklaven ändern.

7.3.3 Video zu Mutierbarkeit

 Schaue dir jetzt das [Video 7.3.3](#) zur Mutierbarkeit an und bearbeite dann das nächste Unterkapitel.

7.4 Attribute und Methoden

Wir haben in den letzten zwei Unterkapiteln die Existenz von Variablen wie `self.name` für selbstverständlich angenommen. Dabei haben diese einen spezifischen Namen.

- q Wir sprechen nämlich von [Attributen](#). Instanzattribute sind solche, welche je nach Instantiierung verschieden sind, also zum Beispiel der Name oder der Status. Dann

gibt es auch Klassenattribute, wie dasjenige der Gattung (oder **genus**) im Code-Beispiel 7.4. Diese ist für alle Instantiierungen der Klasse dasselbe. Im Beispiel unten sind alle Instantiierungen der Klasse **Homo** von der Gattung **Mensch**.

Code-Beispiel 7.4 Die Klasse **Homo** mit einer ersten Methode **beschreibe()**.

```
>>> class Homo:
...     genus = "Mensch"
...     def __init__(self, name, status):
...         self.name = name
...         self.status = status
...
...     def beschreibe(self):
...         return f"{self.name} est {self.status}."
...
>>> marcus = Homo("Marcus", "liber")
>>> tiro = Homo("Tiro", "servus")
>>> marcus.beschreibe()
'Marcus est liber.'
>>> tiro.genus
'Mensch'
>>> marcus.genus == tiro.genus
True
```

Was als Zusatz noch dazukommt, ist eine weitere Funktion. Im Zusammenhang mit Klassen sprechen wir von einer Methode. Das sind Zusatzfunktionen, die mit Punktnotation an den instanziierten Objekten ausgeführt werden können. Die im obigen Beispiel definierte Funktion **beschreibe()** verlangt keine Parameter. Das **self** hier sagt lediglich aus, dass sich die Funktion auf sich selbst, also das Objekt bezieht und macht alle anderen Attribute erhältlich. So wird mit **.beschreibe()** ein String zurückgegeben, der den Namen und den Status konkateniert und beispielsweise **'Marcus est liber.'** (DE *Markus ist frei.*) ausgibt.

7.4.1 Video zu Attributen und Methoden

 Schaue dir jetzt das **Video 7.4.1** zu Attributen und Methoden an und bearbeite dann das Quiz auf OLAT.

7.5 Quiz I zu Klassen und OOP

☒ Löse das **Quiz auf OLAT** und bearbeite dann das nächste Unterkapitel.

7.6 Vererbung und Subklassen

Wir sind in unserer Approximation von Gaius' Menschbegriff erst am Anfang. Wir lesen bei ihm weiter, dass die freien Menschen aus zwei weiteren Klassen bestehen, nämlich den *ingenui*, also jenen, die wirklich frei geboren sind, und den *libertini*, den Freigelassenen. An dieser Stelle teilt Gaius dann nochmals die *libertini* in drei

weitere Gruppen ein: die Bürger mit römischem Bürgerrecht (die *cives Romani*), dem lateinischen Bürgerrecht (die *Latini*) und den *Dediticii*. Letztere sind solche, die sich den Römern auf einem ihrer Feldzüge ergaben. Man kann die Hierarchie so darstellen, wie in Abbildung 7.1.

Bis jetzt haben wir die Status *liber* und *servus* als Attribute definiert. Sie stellen aber in Wirklichkeit wieder eigene Klassen dar, weswegen wir auch hier auf Pythons Klassenkonstrukt zurückgreifen können. Dies illustriert das Code-Beispiel 7.5. Wir definieren wieder eine Klasse `Homo`. Anstatt nun aber den Status in die Initiierungsfunktion zu verfrachten, definieren wir über `__init__` nur, dass die Klasse `Homo` einen Namen erhält. Für die Definition von freien Menschen und Sklavinnen und Sklaven benutzen wir wiederum Klassen. Anstatt diese aber “leer” zu spezifizieren, schreiben wir die Klasse `Homo` in runden Klammern. Das heisst, dass die neu definierten Klassen `Liber` und `Servus` von der `Homo`-Klasse erben. Was genau erben sie? Das Klassenattribut `genus` und die `__init__`-Funktion. Beide Klassen definieren dann ihre eigene Statusnachricht, obwohl der Status eigentlich schon an der Klasse ablesbar ist. Das zeigen die letzten zwei Code-Zeilen im Beispiel 7.5.

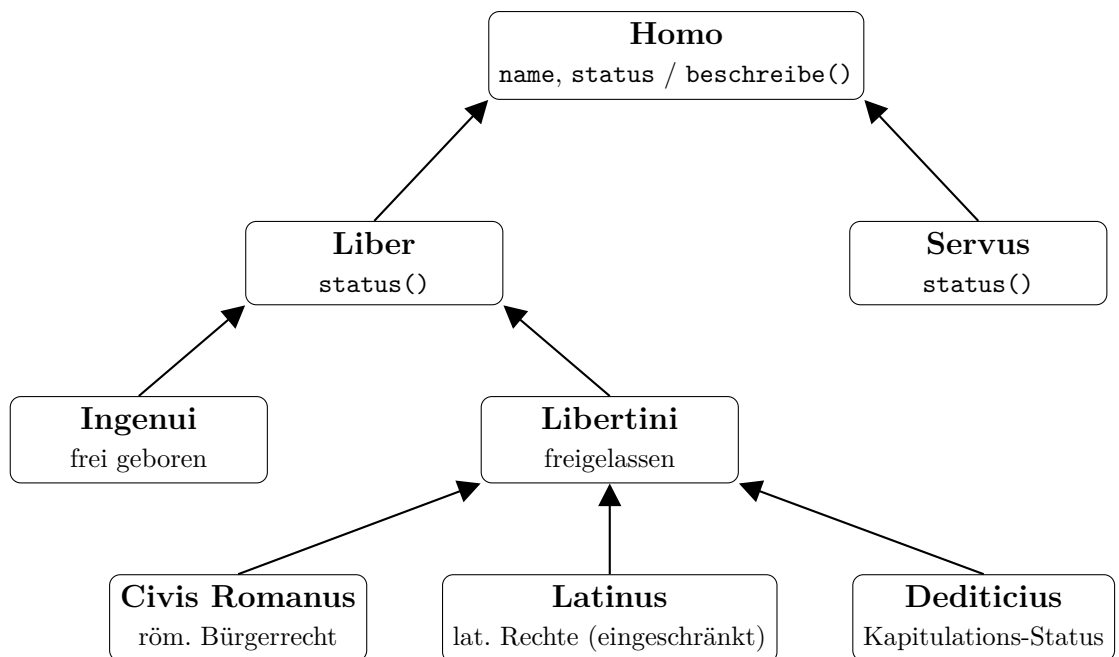


Abbildung 7.1 Vererbung nach Gaius’ *Interventiones*, 9 ff.: Klassenhierarchie `Homo` → `Liber`/`Servus` → `Ingenui`/`Libertini` → `Civis Romanus`, `Latinus`, `Dediticius`. Bei `Homo`, `Liber`, und `Servus` sind die bereits definierten (oder noch zu definierenden) Attribute und Methoden angegeben.

Code-Beispiel 7.5 Subklassen für `Liber` und `Servus`.

```

>>> class Homo:
...     genus = "Mensch"
...     def __init__(self, name):
...         self.name = name
...
>>> class Liber(Homo):
...     def status(self):
...         return f"{self.name} est liber."

```

```

...
>>> class Servus(Homo):
...     def status(self):
...         return f"{self.name} est servus."
...
>>> marcus, tiro = Liber("Marcus"), Servus("Tiro")
>>> marcus.status(), tiro.status()
('Marcus est liber.', 'Tiro est servus.')
>>> type(tiro) == Servus
True

```

Wir treiben das Beispiel nun weiter. Die *ingenui* und *libertini* sind beides Instanzen der Klasse `Liber`, was wir wiederum mit der Vererbung ausdrücken. Code-Beispiel 7.6 veranschaulicht dies. Wir fahren in derselben Manier fort wie oben. Nun erben aber die Klassen `Ingenui` und `Libertini` von der Klasse `Liber`. Beide Klassen definieren wiederum ihre eigenen `status`-Funktionen, weshalb diejenige der Klasse `Liber` überschrieben wird. Wir sehen aber auch, dass die Attribute der `Homo`-Klasse immer noch verfügbar sind. Wir haben zudem noch je ein weiteres Attribut hinzugefügt, welches wichtig sein wird.

Code-Beispiel 7.6 Freie werden weiter unterschieden: Freigeborene und Freigelassene.

```

>>> class Ingenui(Liber):
...     status_achieved = "frei geboren"
...     def status(self):
...         return f"{self.name} est ingenuus (frei geboren)."
...
>>> class Libertini(Liber):
...     status_achieved = "freigelassen"
...     def status(self):
...         return f"{self.name} est libertinus (freigelassen)."
...
>>> julius = Ingenui("Iulius")
>>> tiro = Libertini("Tiro")
>>> julius.status()
'Iulius est ingenuus (frei geboren).'
>>> tiro.status()
'Tiro est libertinus (freigelassen).'
>>> tiro.status_achieved
'frei gelassen'
>>> Libertini.genus
'Mensch'

```

Zuguterletzt erfolgt die Unterscheidung in *Civis Romanus*, *Latinus* und *Dediticius*. Wir lassen die drei Zusatzklassen von der in der Hierarchie nächst höheren Klasse erben, in diesem Fall `Libertini`. Das Code-Beispiel 7.7 zeigt einen solchen Fall. Auch hier sind Attribute aus zuvor definierten Klassen weiterhin verfügbar.

Code-Beispiel 7.7 Freigelassene können Römer, Latiner oder Dediticii sein.

```

>>> class CivisRomanus(Libertini):
...     def status(self):

```

```

...         return f"{self.name} est civis Romanus (römischer Bürger)."
```

```

...
>>> class Latinus(Libertini):
...     def status(self):
...         return f"{self.name} est Latinus (mit eingeschränkten Rechten)."
```

```

...
>>> class Dediticius(Libertini):
...     def status(self):
...         return f"{self.name} est dediticius (feindliche Kapitulation)."
```

```

...
>>> a = CivisRomanus("Felix")
>>> b = Latinus("Faustus")
>>> c = Dediticius("Victor")
>>> [p.status() for p in [a,b,c]]
['Felix est civis Romanus (römischer Bürger).',
 'Faustus est Latinus (mit eingeschränkten Rechten).',
 'Victor est dediticius (feindliche Kapitulation).']
>>> a.status_achieved
'frei gelassen'
>>> a.genus
'Mensch'
```

7.6.1 Video zu Vererbung und Subklassen

 Schaue dir jetzt das [Video 7.6.1](#) zu Vererbung und Subklassen an und bearbeite dann das nächste Unterkapitel.

7.7 Polymorphie und *Method Overriding*

Wir haben im vorherigen Unterkapitel gesehen, dass wir für jede Klasse eine eigene Methode namens `status` definiert haben. Wir kommen deshalb mit jedem Aufruf zu einem anderen Ergebnis. Das ist der Effekt der Polymorphie. Dies geschieht durch *Method Overriding*: Eine Subklasse überschreibt die von der Oberklasse geerbte Methode durch eine eigene Implementierung. Man muss sich dieses Effekts bewusst sein, wenn man beginnt, Klassen voneinander erben zu lassen.

Code-Beispiel 7.8 Polymorphie: unterschiedliche Subklassen reagieren auf denselben Methodenaufruf.

```

>>> homines = [Ingenui("Marcus"), Servus("Tiro"),
...             CivisRomanus("Felix"), Dediticius("Victor")]
>>> for h in homines:
...     print(h.status())
...
Marcus est ingenuus (frei geboren).
Tiro est servus.
Felix est civis Romanus (römischer Bürger).
Victor est dediticius (feindliche Kapitulation).
```

7.7.1 Video zu Polymorphie und *Method Overriding*

 Schaue dir jetzt das [Video 7.7.1](#) zu Polymorphie und *Method Overriding* an und bearbeite dann das nächste Unterkapitel.

7.8 Speziellen Methoden (“dunder”)

Sogenannte *dunder*-Methoden (von *double underscore*) sind spezielle Methoden in Python, die bestimmte Standardoperationen steuern. Zu den Standardoperationen in Python gehören die mathematischen Operatoren wie `+`, `-` oder `,` aber auch logische Operatoren wie `not` oder `is`. Die speziellen *dunder*-Methoden werden in der Regel nicht direkt aufgerufen, sondern implizit über eingebaute Funktionen oder Operatoren verwendet.

Im Code-Beispiel 7.9 sorgt die Methode `__repr__` dafür, dass die Objekte beim Ausgeben in der Konsole eine lesbare Darstellung erhalten, anstatt nur als Speicheradresse angezeigt zu werden (vergleiche mit der Ausgabe im Code-Beispiel 7.2). Die Methode `__eq__` legt fest, wann zwei Objekte als gleich gelten. Hier wird definiert, dass zwei `Homo`-Instanzen gleich sind, wenn sie denselben `status` besitzen, unabhängig von ihrem `name`. Dadurch, dass wir die *dunder*-Methode `__eq__` einsetzen, können wir Objekte mit `==` vergleichen.

Solche speziellen Methoden machen Klassen deutlich flexibler und besser in die Python-Syntax integrierbar.

Code-Beispiel 7.9 Mit `__repr__` und `__eq__` werden Objekte lesbarer und vergleichbar.

```
>>> class Homo:
...     def __init__(self, name, status):
...         self.name = name; self.status = status
...     def __repr__(self):
...         return f"Homo({self.name!r}, {self.status!r})"
...     def __eq__(self, other):
...         return isinstance(other, Homo) and self.status == other.status
...
>>> a = Homo("Marcus", "liber")
>>> b = Homo("Lucius", "liber")
>>> a, b, a == b
(Homo('Marcus', 'liber'), Homo('Lucius', 'liber'), True)
```

7.8.1 Video zu speziellen Methoden

 Schaue dir jetzt das [Video 7.8.1](#) zu den speziellen Methoden an und bearbeite dann das Quiz in OLAT.

7.9 Quiz II zu Klassen und OOP

☒ Löse das [Quiz auf OLAT](#) und bearbeite dann das nächste Kapitel.

Fehlerbehandlung und robuste Programme

Humanum fuit errare, diabolicum est per animositatem in errore manere.

Augustinus, *Sermones* 164,14

Lernziele Kapitel 8: Fehlerbehandlung und robuste Programme

- 1. Verstehen** Die Studierenden können erklären, warum *Fehlerbehandlung* und *robuste Programmierung* zentral sind, und zwischen Syntaxfehlern, Laufzeitfehlern und logischen Fehlern unterscheiden.
- 2. Analysieren** Die Studierenden können typische Fehlermeldungen in Python (z. B. `ValueError`, `KeyError`, `FileNotFoundError`) lesen, interpretieren und deren Ursachen im Code nachvollziehen.
- 3. Anwenden** Die Studierenden können mithilfe von `try`, `except`, `else` und `finally` robuste Programme schreiben, die Eingabe-, Datei- oder Berechnungsfehler erkennen und kontrolliert behandeln, anstatt das Programm abbrechen zu lassen.
- 4. Bewerten** Die Studierenden können verschiedene Strategien der Fehlerbehandlung vergleichen, deren Auswirkungen auf Stabilität, Lesbarkeit und Wartbarkeit des Codes beurteilen und sinnvolle Fehlermeldungen für Nutzer:innen formulieren.
- 5. Erzeugen** Die Studierenden können kleine Programme zur historischen Datenverarbeitung entwickeln, die mögliche Fehlerfälle (z. B. fehlende Dateien, fehlerhafte XML-Tags, unvollständige Datensätze) antizipieren, mit geeig-

neten Ausnahmen absichern und nachvollziehbare Log- oder Warnmeldungen erzeugen.

Bis dato hat sich folgendes Szenario sicher bereits einmal abgespielt: Du möchtest eine Zelle in deinem Jupyter-Notebook ausführen und als Ausgabe erhältst du dann eine Meldung wie `IndexError: list index out of range`. Das ist im Grunde genommen sehr nett von Python: Die Ausgabe verrät dir nämlich, woran die Ausführung eines Teils des Codes gescheitert ist. Es gibt andere Programmiersprachen, welche die Fehlerbehandlung weitaus kryptischer gestalten. Bei einem sogenannten `IndexError` können wir, ohne dass wir jetzt den dazugehörigen Code gesehen haben, darauf schliessen, dass entweder über ein Objekt iteriert wird oder auf ein Element in einer Liste zugegriffen wird, das aber nicht existiert. Die Fehlermeldung ermöglicht es uns, basierend auf den Fehlertypen den Fehler im Code schneller zu finden und letzten Endes auch zu korrigieren.

Python erlaubt aber auch noch weitaus mehr. Normalerweise bricht der Code beim ersten Fehler, auf den Python bei der Ausführung eines Programms trifft, ab und gibt eben diese Fehlermeldung aus. Wir können aber einen solchen Fehler, der bei unserer Datenverarbeitung auftauchen könnte, auch antizipieren und “abfangen” (Engl. *catch*). Danach haben wir zwei Möglichkeiten: (1) Wir beenden das Programm und geben eine vielsagende Fehlermeldung aus, damit Nutzer:innen entsprechend darauf reagieren können. (2) Wir fangen den Fehler ab und führen den Code unter Berücksichtigung des Fehlers weiter aus. Letzteres sorgt für robuste Programme und stellt sicher, dass diese bis zum Ende ausgeführt werden.

Zugleich ist eine angemessene Fehlerbehandlung auch eine Art und Weise, Code zu *debuggen*. Anstatt ein Programm mit zahlreichen `print`-Statements zu versehen, besteht mittels Fehlerbehandlung und gegebenenfalls dem `Logging` eine Alternative, die—vor allem wenn der Code durchläuft—nicht in übermässiger Ausgabe resultiert, sondern halt nur dann etwas ausgibt, wenn der Code auf einen Fehler trifft.

Zu robusten Programmen gehört im Übrigen auch das Schreiben sogenannter Tests. Diese prüfen insbesondere selbstgeschriebene Funktionen und Klassen. Dabei gibt es zwei Paradigmen: Zum einen besteht die Möglichkeit, Testfälle in den Docstring zu schreiben und mit dem Modul `doctest`¹ die Funktionsweise zu prüfen. Zum anderen könnt ihr mittels `unittest`² ganze Test-Suiten erstellen, die ebenfalls sicherstellen, dass Inputs und Outputs dem entsprechen, was ihr erwartet. Wir gehen in dieser Einführung aber nicht weiter darauf ein.

`unittest` und `doctest` als Exkurs hinzufügen.

8.1 Fehlertypen verstehen

Um die Fehlertypen, die uns begegnen können, zu verstehen, müssen wir uns zuerst einmal einen Überblick über die möglichen Fehlertypen verschaffen. Die nachstehende Tabelle 8.1 zeigt uns einige davon:

Es gibt jedoch noch weitaus mehr Fehlertypen. Das liegt daran, dass man auch seine eigenen Fehlertypen definieren kann. Das wird in Modulen oft gemacht. Nehmen wir als Beispiel das `xml`-Modul. Wir haben da bereits `etree` verwendet, um

¹Siehe <https://docs.python.org/3/library/doctest.html>.

²Siehe <https://docs.python.org/3/library/unittest.html>.

Tabelle 8.1 Auswahl häufig auftretender Fehlertypen in Python

Fehlertyp	Beschreibung
SyntaxError	Der Code verstößt gegen die Regeln der Python-Syntax (z. B. fehlende Klammer, Doppelpunkt vergessen).
NameError	Eine Variable oder Funktion wird verwendet, bevor sie definiert wurde.
TypeError	Ein Vorgang wird auf einen ungeeigneten Datentyp angewendet (z. B. <code>'abc' - 3</code>).
ValueError	Ein Wert hat den richtigen Typ, aber einen ungeeigneten Inhalt (z. B. <code>int('abc')</code>).
IndexError	Zugriff auf eine Position in einer Liste, die es nicht gibt.
KeyError	Zugriff auf einen nicht existierenden Schlüssel in einem Dictionary.
AttributeError	Ein Objekt besitzt das angeforderte Attribut nicht (z. B. <code>42.append()</code>).
ImportError	Ein Modul oder Objekt kann nicht importiert werden.
ZeroDivisionError	Division durch Null wurde versucht.
FileNotFoundError	Eine Datei, auf die zugegriffen werden soll, existiert nicht.

XML-Dateien zu parsen und Informationen daraus zu extrahieren. Nehmen wir an, wir wollten mit `et.parse()` eine korrupte—also falsch formatierte—XML-Datei parsen, dann würde uns Python folgende Fehlermeldung zurückgeben: `xml.etree.ElementTree.ParseError: mismatched tag: line 1, column 12`. Wir erhalten ziemlich genaue Informationen darüber, was diesen `ParseError` verursacht hat, und können damit das Format in Ordnung bringen.

8.2 Fehler abfangen mit try/except

Logging hinzufügen

Wir lernen im Umgang mit Fehlern nun vier neue Keywords kennen, nämlich `try`, `except`, `finally` und `raise`. Zudem können wir das uns bereits bekannte Keyword `else` bei der Fehlerbehandlung verwenden. Die hierarchische Struktur, wie man diese Keywords verwendet, sieht wie in Code-Beispiel 8.1 dargelegt aus:

Code-Beispiel 8.1 Hierarchie von Fehlerbehandlung.

```
>>> try:
...     # Zu testendes Szenario hier
... except:
...     # Alternativ auszuführender Block, falls Block unter `try` fehlschlägt
... else:
...     # wird ausgeführt, wenn keine Fehler auftreten
... finally:
...     # wird in jedem Fall ausgeführt
```

Mit `raise` definieren wir unseren eigenen Fehler, der auftritt und ein Programm beendet, wenn die Prüfung einer Bedingung fehlschlägt (Code-Beispiel 8.2):

Code-Beispiel 8.2 Beispiel der Definition eines eigenen Fehlers mittels `raise`

```
>>> l = [1, 2, 3]
>>> if not l[0] is str:
...     raise Exception('Das ist kein str.')
```

Kommen wir nun zu ein paar Beispielen, wie wir die Fehlerbehandlung im Code einbauen könnten. Das Code-Beispiel 8.3 zeigt uns die einfachste Form eines `try/except`-Blocks. Wir haben 10 Münzen aber keine Fundorte (evt. weil wir im Unklaren darüber sind, wo die Münzen genau gefunden wurden). Wenn man nun die durchschnittliche Anzahl von Münzen pro Fundort berechnen möchte, würde man mit den aktuell geltenden Werten durch Null dividieren. Wir können diesen Fehler abfangen. Da der `ZeroDivisionError` in Python bereits definiert ist, können wir ihn explizit abfangen. Das Statement `except ZeroDivisionError` sorgt dafür, dass, falls wir eben durch Null teilen wollen, der darunter aufgeführte Code-Block ausgeführt wird. Danach läuft das Programm normal weiter. An dieser Stelle sei erwähnt, dass das `ZeroDivisionError` auch weggelassen werden könnte. In einem solchen Fall könnte im Block unter `try` jedweder Fehler auftreten, und dann würde der Block unter `except` ausgeführt werden. Besonders dann, wenn verschiedene Fehlertypen auftauchen können, von denen man jeden anders behandeln möchte, empfiehlt es sich nicht, ein generisches `except`-Statement zu verwenden.

Code-Beispiel 8.3 Ein einfacher Fehler und seine Behandlung mit `try` und `except`.

```
>>> try:
...     coins = 10
...     findspots = 0
...     ratio = coins / findspots
...     print(f"Durchschnittlich {ratio} Münzen pro Fundort.")
... except ZeroDivisionError:
...     print("Fehler: Division durch 0 – keine Fundorte vorhanden.")
Fehler: Division durch 0 – keine Fundorte vorhanden.
```

Ein weiteres Code-Beispiel (8.4) führt vor, wie sich ein `else`-Block integrieren lässt. Dieser wird ausgeführt, wenn der Code unter `try` keinen Fehler verursacht. In diesem Beispiel ist die Ausgabe also die Anzahl der Wörter in einer Inschrift, ausser eben, der Code schlägt unter `try` schon fehl. Dann würde der entsprechende Fehler ausgegeben werden. Das erreichen wir hier mit einer etwas speziellen Syntax. `Exception` ist eine Klasse in Python. Wenn ein Fehler, also eine `Exception` unter `try` auftritt, dann speichern wir diesen als (`as`) `e` und geben den entsprechenden Fehler zurück.

Code-Beispiel 8.4 Verwendung von `else`: Code, der nur bei erfolgreicher Ausführung ausgeführt wird.

```
>>> try:
...     inscription = "IMP CAESAR DIVI F"
...     words = inscription.split()
...     length = len(words)
```

```
... except Exception as e:
...     print(f"Fehler: {e}")
... else:
...     print(f"Die Inschrift enthält {length} Wörter.")
```

Die Inschrift enthält 4 Wörter.

Im nächsten Code-Beispiel 8.5 schauen wir uns an, was das Hinzufügen des Keywords `finally` für eine Auswirkung auf die Ausführung des Codes hat. Dies ist schnell erklärt: Ganz unabhängig davon, was unter `try` und `except` passiert, wird der Block unter `finally` in jedem Fall ausgeführt. Das Beispiel macht dies mit dem entsprechenden `print`-Statement offensichtlich.

Code-Beispiel 8.5 `finally` sorgt dafür, dass eine Datei immer geschlossen wird.

```
>>> try:
...     file = open("inscription.txt", "r", encoding="utf-8")
...     content = file.read()
...     print(content[:50]) # nur die ersten 50 Zeichen
... except FileNotFoundError:
...     print("Datei wurde nicht gefunden.")
... finally:
...     print("Dateioperation abgeschlossen.")
Datei wurde nicht gefunden.
Dateioperation abgeschlossen.
```

Wir wenden uns im Code-Beispiel 8.6 noch dem `raise`-Keyword. Wir befinden uns wieder im Bereich der Numismatik und prüfen in einer Funktion mittels eines Statements und eben `raise` das Gewicht einer Münze. Wir fügen hier eine Bedingung ein: Das Gewicht einer Münze muss (logischerweise) grösser als 0 sein. Wenn ein:e Benutzer:in fälschlicherweise oder aus Versehen ein negatives Gewicht einzugeben versucht, wird eine Fehlermeldung provoziert und der Code stoppt. Wir erzeugen in diesem Fall einen `ValueError` und fügen einen String hinzu, der erklärt, welcher Wert hier erwartet wird.

Code-Beispiel 8.6 Mit `raise` kann gezielt ein eigener Fehler erzeugt werden.

```
>>> def normalise_weight(weight):
...     if weight <= 0:
...         raise ValueError("Das Gewicht muss grösser als 0 sein.")
...     return round(weight, 2)
...
>>> normalise_weight(5.413)
5.41
>>> normalise_weight(-2.0)
Traceback (most recent call last):
...
ValueError: Das Gewicht muss grösser als 0 sein.
```

Schliesslich betrachten wir im Code-Beispiel 8.7 noch die Kombination aller nun neu gelernten Keywords. Die erste Funktion `format_year` gibt einen Fehler für das Jahr 0 aus, das in der historischen Zeitrechnung nicht existiert (sehr wohl aber in der astronomischen). Danach prüft sie das Vorzeichen und gibt entweder "v. Chr." oder

“n. Chr.” zurück. Die Funktion `print_birth_year` prüft, ob eine Person in den in den Argumenten mitgegebenen Daten vorhanden ist und speichert zuerst Jahr und die daraus abgeleitete Formatierung. Je nachdem, ob ein Fehler ausgelöst wird, wird eines der `except`-Statement ausgelöst. Gibt es keine Fehler, kommt das `else`-Statement zum Zuge. Und final wird dann mitgeteilt, dass der Nachschlagvorgang im Dictionary abgeschlossen ist. Die Beispiele verdeutlichen erneut die Funktionsweise dieser `try/except/else/finally`-Kaskade.

Code-Beispiel 8.7 Kombination von `try`, `except`, `else`, `finally` und `raise` mit historischen Jahreszahlen.

```
>>> def format_year(year):
...     if year == 0:
...         raise ValueError("Jahr 0 ist historisch nicht definiert.")
...     if year < 0:
...         return f"{abs(year)} v. Chr."
...     return f"{year} n. Chr."
...
>>> def print_birth_year(person, data):
...     try:
...         year = data[person]           # kann KeyError auslösen
...         text = format_year(year)     # kann ValueError auslösen
...     except KeyError:
...         print(f"Keine Daten für {person} gefunden.")
...     except ValueError as e:
...         print(f"Fehler in den Daten für {person}: {e}")
...     else:
...         print(f"{person} wurde {text} geboren.")
...     finally:
...         print("Nachschlagvorgang abgeschlossen.")
...
>>> birth_data = {
...     "Cicero": -106,
...     "Augustus": -63,
...     "Trajan": 53
... }
...
>>> print_birth_year("Cicero", birth_data)
Cicero wurde 106 v. Chr. geboren.
Nachschlagvorgang abgeschlossen.
...
>>> print_birth_year("Plato", birth_data)
Keine Daten für Plato gefunden.
Nachschlagvorgang abgeschlossen.
...
>>> birth_data["Test"] = 0
>>> print_birth_year("Test", birth_data)
Fehler in den Daten für Test: Jahr 0 ist historisch nicht definiert.
Nachschlagvorgang abgeschlossen.
```

8.3 Video zu Fehlertypen und Fehler abfangen

 Schaue dir jetzt das **Video 8.3** zu Fehlertypen und Fehler abfangen an und bearbeite dann das Quiz in OLAT.

8.4 Quiz zu Fehlerbehandlung und robusten Programmen

☒ Löse das **Quiz auf OLAT** und bearbeite dann das nächste Kapitel.

Ethische Reflexion und Verantwortung

“Raw data” is both an oxymoron and a bad idea.

Geoffrey C. Bowker (2005)

Lernziele Kapitel 9: Ethische Reflexion und Verantwortung

- 1. Verstehen** Die Studierenden können grundlegende ethische Prinzipien der *digitalen Geschichtsforschung* erläutern und erklären, warum Verantwortung beim Programmieren und Datenverarbeiten zentral ist.
- 2. Analysieren** Die Studierenden können ethische Problemfelder (z. B. Bias in Daten, algorithmische Verzerrungen, unsichere Interpretationen, Repräsentationsfragen) in historischen Datensätzen und digitalen Modellen identifizieren und kritisch bewerten.
- 3. Anwenden** Die Studierenden können ethische Leitlinien auf eigene Programmierprojekte anwenden, etwa durch transparente Dokumentation, nachvollziehbare Entscheidungen, faire Modellierung und verantwortungsvollen Umgang mit unsicheren oder sensiblen Daten.
- 4. Bewerten** Die Studierenden können algorithmische und datenbezogene Entscheidungen im historischen Forschungskontext reflektieren, Risiken abwägen und alternative Vorgehensweisen argumentativ beurteilen.
- 5. Erzeugen** Die Studierenden können kleine Projekte entwerfen, die ethische Prinzipien der digitalen Geschichtsforschung integrieren, z. B. durch offene Annotation, reflexive Modellierungsentscheidungen oder verantwortungsbewusstes Datenhandling.

9.1 Algorithmischer Bias und Quellenkritik

9.2 Verantwortlicher Umgang mit Datenquellen

Textanalyse historischer Daten mit NLTK, CLTK und spaCy

τοῦ δὲ λόγου μέρη ἐστὶν ὀκτώ· ὄνομα, ῥῆμα, μετοχή, ἄρθρον,
ἀντωνυμία, πρόθεσις, ἐπίρρημα, σύνδεσμος· ἡ γὰρ προσηγορία ὡς
εἶδος τῷ ὀνόματι ὑποβέβληται.

Dionysios Thrax, *Τέχνη γραμματική*.

Lernziele Kapitel 10: Textanalyse historischer Daten mit NLTK, CLTK und spaCy

- 1. Verstehen** Die Studierenden können die grundlegenden Konzepte der *Computerlinguistik* erläutern, insbesondere die Funktionsweise von *Tokenisierung*, *Lemmatisierung*, *Part-of-Speech-Tagging* und *Parsing*, und den Unterschied zwischen *NLTK*, *CLTK* und *spaCy* erklären.
- 2. Analysieren** Die Studierenden können unterschiedliche Annotationen historischer Texte vergleichen, typische Fehlerquellen (z. B. bei stark flektierten Sprachen wie Latein oder Altgriechisch) erkennen und beurteilen, welche Werkzeuge für welche Art historischer Quellen geeignet sind.
- 3. Anwenden** Die Studierenden können mit *NLTK*, *CLTK* und *spaCy* einfache linguistische Analysen durchführen, darunter Tokenisierung, Frequenzanalysen, Stoppwortfilterung, Lemmatisierung sowie POS-Tagging moderner und historischer Textkorpora.
- 4. Bewerten** Die Studierenden können die Qualität automatischer Annotationen kritisch beurteilen, Unterschiede zwischen regelbasierten und statistischen Modellen reflektieren sowie die Grenzen von NLP-Tools in der historischen Forschung evaluieren.
- 5. Erzeugen** Die Studierenden können ein kleines Analyse-Projekt umsetzen,

das einen historischen Text mit mindestens einem der behandelten NLP-Frameworks verarbeitet, annotiert und visualisiert (z. B. Frequenzen, Kollokationen, Named Entities).

Für die Lernziele gilt am 24.11.2025: Grossflächigere Integration von CLTK und spaCy noch in Planung. Im Moment werden als Lernziel nur Tokenisierung, Frequenzanalyse, Stoppwörter und Funktionsweise von NLTK erwartet. Zudem soll man die NER erklären können

Wie das Epigraph dieses Kapitels zeigt, haben sich bereits schlaue Köpfe aus der Antike Gedanken zum Aufbau der Sprache gemacht. Dionysios Thrax, der vermutlich aus Alexandrien stammte und ins 2. Jh. v. Chr. verortet wird, unterscheidet in seiner “Grammatik” bereits acht verschiedene Wortarten (neun, zählt man Gattungsnamen (προσηγορία) dazu): Nomen (ὄνομα), Verben (ῥῆμα), Partizipien (μετοχή), Artikel (ἄρθρον), Pronomen (ἀντωνυμία), Präpositionen (πρόθεσις), Adverbien (ἐπίρρημα), Konjunktionen (σύνδεσμος). Das ist von unserem heute angewandten Klassifikationsschema, wie wir es bereits in der Grundschule lernen, gar nicht weit entfernt.

Damals mussten wir in Sätzen die Wortarten (EN *Parts-of-Speech*, oder kurz, *PoS/POS*) selbst bestimmen. Wenn wir mit einer grossen Menge an Textdaten operieren, ist das nicht mehr realistisch (oder nur noch begrenzt). Hier kommt die automatische Textverarbeitung natürlicher Sprache ins Spiel. Im Englischen sprechen wir von *natural language processing* (NLP). Der Computer wird seit den 1960er-Jahren zur Sprachverarbeitung genutzt. Mit dem Aufkommen von Verfahren, die schon damals auf Prinzipien des maschinellen Lernens und der künstlichen Intelligenz (und hier sind explizit nicht die seit 2022 bekannten grossen Sprachmodelle von OpenAI und Google angesprochen) basierten, begann man, Sprachdaten mit dem Computer zu verarbeiten und zu analysieren. Daraus entwickelte sich auch eine ganz eigene Forschungsrichtung: die Computerlinguistik.

Als sich Python als Programmiersprache immer mehr etablierte, setzten sich Forschende an die Entwicklung einer Bibliothek, mit der man Texte verarbeiten konnte. Einer der bekanntesten Module ist das sogenannte *Natural Language Processing Toolkit* (NLTK), das seit 2001 besteht und noch immer weiterentwickelt wird.¹ Dazu haben die Urheber Edward Loper, Steven Bird und später auch Ewan Klein auch ein frei zugängliches Buch publiziert.²

Von der kontinuierlichen Arbeit an NLTK profitierten zusehends auch historische Sprachformen. Das *Classical Language Toolkit* (CLTK) (Johnson et al., 2021) wurde für die Verarbeitung von historischen Sprachdaten auf der Basis von NLTK entwickelt.³ Dieses Software-Paket bietet die Möglichkeit, Sprachen wie Altgriechisch und Latein, aber auch Koptisch, Altenglisch, Mittelhochdeutsch oder Sanskrit zu verarbeiten. Wir müssen dazu jedoch anmerken, dass nicht alle Funktionen in allen Sprachen verfügbar sind. Gerade deshalb wird auf diesem Gebiet mit Hochdruck geforscht.

Mit der ab 2012 intensivierten Entwicklung von neuronalen Netzen im Bereich des maschinellen Lernens erlebte auch das NLP—und viele Anwendungen, die es mit sich brachte—einen Boost. Das seit 2015 gepflegte Open-Source-Projekt *spaCy*,⁴ das am MIT entwickelt wird und eigentlich für die Implementierung von NLP-Bausteinen in der Industrie gedacht war, hat sich gerade durch die einfache Integra-

¹Siehe <https://www.nltk.org>.

²Siehe <https://www.nltk.org/book>.

³Siehe <https://cltk.org>.

⁴Siehe <https://spacy.io>.

tion von neuronalen Netzen auch in der Forschung etabliert. So erlaubt *spaCy* auch das Trainieren und Testen von eigenen Modellen für jegliche sprachverarbeitende Modelle. Es bildet übrigens auch einen Teil des Grundgerüsts von CLTK.

Aufgrund dieser Entwicklung geben wir hier einen kurzen Überblick über zwei der drei Module: NLTK und CLTK. Diese müssen vorgängig installiert werden, wenn man diese auf dem eigenen Rechner verwenden möchte. CLTK und *spaCy* laufen in der JupyterLite-Umgebung nicht. Deshalb funktionieren die Beispiele, die wir besprechen, vollumfänglich nur auf den eigenen Geräten, auf welchen alle nötigen Packages installiert sind. Trotzdem illustrieren wir die Funktionsweise zweier Libraries anhand von drei häufig auftretenden Problemen: der Tokenisierung (siehe Abschnitt 10.1), der Filterung von Stoppwörtern und der einfachen Wortfrequenzanalyse (siehe Abschnitt 10.2) sowie der Eigennamenerkennung (siehe Abschnitt 10.3). Der Fokus liegt hierbei hauptsächlich auf der Anwendung ausgesuchter Funktionen. Wir werden aber in jedem Code-Beispiel jeweils gleich alle die Herangehensweisen für zwei Libraries präsentieren. Wichtig ist, dass man für das Bearbeiten der Text alle Module geladen hat.⁵ Es ist jedoch nicht möglich, die Breite aller Module in diesem Kapitel abzudecken. Hier verweisen wir auf die jeweiligen Online-Dokumentationen der Libraries. Auch auf methodische Einzelheiten können wir nur kurz eingehen. Wer sich hier vertiefen möchte, sei auf das Buch *Speech and Language Processing* von Jurafsky and Martin (2025) verwiesen.⁶

Disclaimer: Für historische Sprachen (und digitalisierte Texte mit OCR- oder HTR-Fehlern, also Fehlern aus der automatischen Zeichenerkennung) sind alle hier gezeigten Werkzeuge mit Vorsicht zu geniessen: Fehlende Trainingsdaten, orthografische Varianz und beschädigte Überlieferung können die Qualität der Annotation stark beeinflussen. **Ergebnisse müssen immer historisch und philologisch abgesichert werden.**

10.1 Textvorverarbeitung und Tokenisierung

Wir beginnen unsere Exploration mit einer ganz grundlegenden Operation in der Textverarbeitung, nämlich der Text*vor*verarbeitung (EN *preprocessing*). Ein ganz wichtiger Schritt, auf welchem dann viele folgende Schritte aufbauen, ist die sogenannte *Tokenisierung*. Im Prinzip heisst das für ein Programm nichts anderes als “Splitte einen beliebig langen Textstring an seinen Wortgrenzen auf.” Ein Token ist in der praktischen Computerlinguistik die Basiseinheit der Textverarbeitung, typischerweise ein Wort oder Satzzeichen.⁷ Ihr könnt euch bereits vorstellen, dass der Split an Leerzeichen je nach Sprache unterschiedlich gut funktioniert. Gewisse er-

⁵Die Modelle für *spaCy* muss man zuerst herunterladen. Via Konsole funktioniert das folgendermassen. Für ein kleines englisches Modell: `python -m spacy download en_core_web_sm`. Für ein lateinisches Modell: `pip install https://huggingface.co/latincy/la_core_web_lg/resolve/main/la_core_web_lg-any-py3-none-any.whl`. Und für ein griechisches Modell: zuerst `pip install greycy` und dann `python -m greycy install grc_proiel_trf`. In jedem Fall bedingen die Modelle, dass eine passende *spaCy*-Version installiert ist. Das kann unter Umständen mühsam zu konfigurieren sein.

⁶Siehe auch hier: <https://web.stanford.edu/~jurafsky/slp3>.

⁷Das Token ist nicht die kleinste semantische Einheit, das wären die Morpheme.

achten das Problem der Tokenisierung als gelöst an. Gute Tokenisierer erreichen auch durchaus Performance-Levels von 98% und mehr. Das heisst, dass sie in 98% der Fälle die Wortgrenzen korrekt erkennen. Das heisst aber auch, dass sie bei 100 Wortgrenzen immer noch 2 Fehler machen. Hinzu kommt, dass je nach Anwendung andere Arten von Tokenisierung erwünscht sein könnten.

Aber der Reihe nach. Betrachten wir zuallererst die einfachste Form der Tokenisierung: das Aufsplitten an den Leerzeichen. Das Code-Beispiel 10.1 veranschaulicht dies an einem Text, den wir aus dem Code-Beispiel 4.2 bereits kennen:

Code-Beispiel 10.1 Beispiel der Tokenisierung mit Leerzeichen

```
>>> text_la = "hostile fugientes in sacratis eius locis uitam, de qua
superbiunt, inuenirent. an non etiam illi Romani Christi nomini infesti sunt,
quibus propter Christum barbari pepercerunt? testantur hoc martyrum loca et
basilicae apostolorum, quae in illa uastatione urbis ad se confugientes suos
alienosque receperunt. hucusque cruentus saeuiebat inimicus, ibi accipiebat
limitem trucidatoris furor, illo ducebantur a miserantibus hostibus, quibus
etiam extra ipsa loca pepercerant, ne in eos incurrerent, qui similem
misericordiam non habebant. qui tamen etiam ipsi alibi truces atque hostili
more saeuientes posteaquam ad loca illa ueniebant, ubi fuerat interdictum quod
alibi belli iure licuisset, tota feriendi refrenabatur inmanitas et
captiuandi cupiditas frangebatur."
>>> text_la_tok = text_la.split()
>>> text_la_tok[:20]
['hostile', 'fugientes', 'in', 'sacratis', 'eius', 'locis', 'uitam,', 'de', '
qua', 'superbiunt,', 'inuenirent.', 'an', 'non', 'etiam', 'illi', 'Romani', '
Christi', 'nomini', 'infesti', 'sunt,']
>>> len(text_la_tok)
103
```

Wir sehen, dass die Anwendung der Built-In-Funktion `split()` genau an den Leerzeichen auftrennt. Das resultiert in 103 lateinischen und 26 griechischen Tokens. Wenn wir uns die ersten 20 Tokens aus beiden tokenisierten Texten anschauen, dann fällt uns auf, dass die Satzzeichen an den Worten “kleben.” Das ist für die weitere Textverarbeitung ungünstig. Solche Fälle könnte man mit regulären Ausdrücken (siehe Kapitel 4) abfangen und so die Tokenisierung verbessern, denn auch Satzzeichen sind für sich genommen einzelne Tokens. Tatsächlich basierten die ersten Tokenisierer auf regulären Ausdrücken und zählen somit zu den regelbasierten Tokenisierern. Einen solchen finden wir im Übrigen auch in NLTK.

Ein genauerer Blick mit `text_la_tok[30:45]` in den tokenisierten lateinischen Text offenbart zudem: `['quae', 'in', 'illa', 'uastatione', 'urbis', 'ad', 'se', 'confugientes', 'suos', 'alienosque', 'receperunt.']. Wir lenken unsere Aufmerksamkeit auf alienosque. Das sind eigentlich zwei Wörter: alienos und que. Der Split an Leerzeichen hat uns dieses Token unterschlagen. Deswegen sind viele Tokenisierer sprachabhängig. Es ist um einiges komplexer, einen universellen Tokenisierer zu erstellen, als einen, der sich auf eine Sprache fokussiert. Betrachten wir das Verhalten der unterschiedlichen Tokenisierer in Code-Beispiel 10.2 (wir gehen davon aus, dass text_la und text_gr schon definiert sind):`

Code-Beispiel 10.2 Vergleich verschiedener Tokenisierer (NLTK / CLTK)

```

>>> import nltk
>>> import cltk
>>> cltk_nlp_la = cltk.NLP("lat")
>>> text_en = "There's a hearty menu of pan-Hispanic dishes like Peruvian
ceviche and Cuban sandwich empanadas, as well as nightly live music acts often
playing trova, a genre of Cuban music most famously represented by the Buena
Vista Social Club band and its eponymous 1999 documentary. \"But if it's
Friday or Saturday late at night, the back becomes an '80s room and it's so
much fun.\""
>>> text_la_nltk_tok = nltk.word_tokenize(text_la)
>>> text_en_nltk_tok = nltk.word_tokenize(text_en)
>>> len(text_la_nltk_tok), len(text_en_nltk_tok)
(119, 74)
>>> doc_la = cltk_nlp_la(text_la)
>>> len(doc_la.tokens)
119

```

NLTK und CLTK funktionieren nicht genau gleich. Wir merken uns, dass wir für NLTK explizit eine Methode aufrufen müssen, hier `word_tokenizer()`. Das Resultat ist dann eine Liste mit dem tokenisierten Text. In CLTK definieren wir zuerst, mit welchem Modell wir arbeiten möchten (hier mit dem lateinischen). Danach lösen wir eine Pipeline aus, indem wir der definierten Modelklasse direkt den Text mitgeben. Dies kreiert ein `Doc`-Objekt, das sämtliche Informationen enthält. Wir können uns vorstellen, dass die Tokens als Attribut in eine Liste gespeichert wurden. Deshalb könnten wir mit `.tokens` auf die Wörter zugreifen und auch die Länge dieser Liste in Erfahrung bringen. Wir sehen, dass wir 16 Tokens mehr erhalten, wenn wir mit NLTK und CLTK operieren (im Gegensatz zum Splitting an Leerzeichen). Interessanterweise splittet CLTK das angehängte *-que* nicht ab. Das könnte anders aussehen, wenn die gleiche Pipeline mit einem anderen Modell in *spaCy* implementieren würde.

Nichtsdestotrotz zeigt eine Inspektion der von CLTK erstellten Lemmata mit `doc_la.lemmata`, dass das Lemma von der Oberflächenform *alienusque* korrekt mit *alienus* identifiziert worden ist. Insofern scheint das Modell keine Probleme mit einem angehängten *-que* zu haben.

10.1.1 Video zu Textvorbereitung und Tokenisierung

 Schaue dir jetzt das [Video 10.1.1](#) zu Textvorbereitung und Tokenisierung an und bearbeite dann das nächste Unterkapitel.

10.2 Stoppwörter und Wortfrequenzanalyse

Als Stoppwörter bezeichnen wir Wörter, die wenig bis keinen semantischen Gehalt haben (trotzdem können sie für den semantischen Inhalt eines Satzes durchaus relevant sein). Im Deutschen wären das Wörter wie *in*, *zu*, *bei*, *ja*, etc. Für gewisse Anwendungen ist es ratsam, diese Wörter zu entfernen. Beispielsweise verzichtet man beim sogenannten [Topic Modeling](#), welches zum Ziel hat, Themenbereiche 

in einer Textsammlung zu erkennen, meist komplett auf diese Stoppwörter, da sie nichts zum Inhalt beitragen. In NLTK gibt es für viele Sprachen schon vordefinierte Stoppwortlisten. Anhand dieser kann man dann jeden beliebigen Text filtern. CLTK bietet uns bereits eine stoppwortbereinigte Version vom Text, den wir analysiert haben. Das Code-Beispiel 10.3 veranschaulicht dies:

Code-Beispiel 10.3 Entfernen von Stoppwörtern mit NLTK

```
>>> en_stopwords = nltk.corpus.stopwords.words('english')
>>> en_no_stopwords = [t.lower() for t in text_en_nltk_tok if t.lower() not in
    en_stopwords]
>>> len(en_no_stopwords )
51
>>> len(doc_la
>>> len(doc_la.tokens_stops_filtered)
81
```

Für den englischen Text haben wir die Anzahl Tokens von 74 auf 51 reduziert. Wörter wie *there*, *a*, *of*, *and*, usw. sind aus dem Text verschwunden. Für den lateinischen Text haben wir ebenfalls eine Reduktion von 119 auf 81 Wörter. Auch hier fehlen grösstenteils Präpositionen, Konjunktionen und Pronomen. Da die Eliminierung der Stoppwörter bereits in der Pipeline von CLTK vorhanden ist, gelangen wir einfach mit `tokens_stops_filtered` an diese Textversion.

Das Herausfiltern von Stoppwörtern hat selbstverständlich einen direkten Einfluss auf Wortfrequenzanalysen. Wir illustrieren das am Beispiel der englischen Version der Genesis, die in NLTK verfügbar ist. NLTK wird mit einer Vielzahl von Korpora geliefert. Trotzdem müssen diese zuerst heruntergeladen werden. Das Code-Beispiel 10.4 zeigt, wie wir das Korpus herunterladen und erste Statistiken basierend auf den darin vorkommenden Wörtern berechnen. Wir benutzen dazu die Stoppwortliste aus dem Code-Beispiel 10.3.

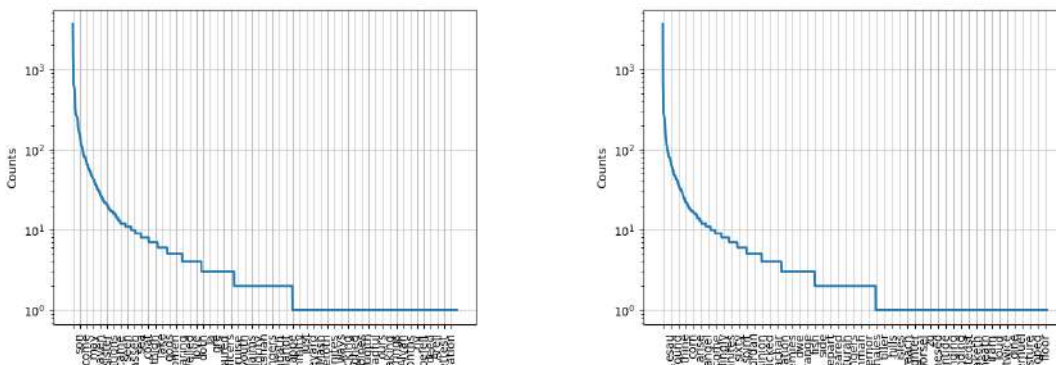
Code-Beispiel 10.4 Frequenzanalyse im Genesis-Korpus

```
>>> nltk.download('genesis') # lädt das Korpus herunter und muss nur ein Mal
ausgeführt werden
>>> from nltk.corpus import genesis
>>> genesis_words = genesis.words('english-kjv.txt')
>>> genesis_words[:10]
['In', 'the', 'beginning', 'God', 'created', 'the', 'heaven', 'and', 'the', '
earth']
>>> len(genesis_words)
44764
>>> fd = nltk.FreqDist(genesis_words)
>>> fd
FreqDist({'': 3681, 'and': 3678, 'the': 2461, 'of': 1364, '.': 1315, 'he':
655, 'his': 653, 'to': 612, ';': 605, 'in': 600, ...})
>>> len(fd)
2628
>>> import matplotlib.pyplot as plt
>>> plot = fd.plot()
>>> labels = plt.xticks()[0]
```

```
>>> plt.xticks(labels[::50])
>>> plt.savefig('fd-genesis-50-log-en.png')
# gleiche Prozedur, aber mit entfernten Stoppwörtern in einer neuen Session
>>> fd_stopwords_rm = nltk.FreqDist([w.lower() for w in genesis_words if w.
lower() not in en_stopwords])
>>> len(fd_stopwords_rm)
2508
...
```

Das Code-Beispiel nutzt `FreqDist()` aus NLTK um eine Worthäufigkeitsanalyse aus dem Text zu erstellen. Zum einen auf dem unbereinigten Text und zum anderen auf einer Version, aus welcher wir die Stoppwörter entfernt haben. Vor der Wortzählung haben wir 44764 Tokens vorliegen. Danach schrumpft der Bestand auf 2628, respektive 2508 (stoppwortbereinigt) **Types** zusammen. Als Type bezeichnen wir in der Computerlinguistik einmalig vorkommende Oberflächenformen von Wörtern (im Grunde genommen das Set der Wörter, aus welchem der Text besteht).

Die Abbildung 10.1 zeigt die Verteilung der Worthäufigkeiten (a) vor und (b) nach Entfernung der Stoppwörter auf einer logarithmierten Skala. Die Verteilungen sehen an sich sehr ähnlich aus. Es fällt jedoch bei beiden eines klar ins Auge: Es gibt sehr wenige Wörter, die sehr häufig vorkommen und ganz viele, die nur wenige Male, meist sogar nur ein Mal auftauchen. Die einmalig vorkommenden Wörter werden **hapax legomena** genannt.



(a) Häufigkeitsverteilung Genesis (Log-Scale, jedes 50. Wort auf x-Achse)

(b) Häufigkeitsverteilung Genesis Stoppwort-Entfernung (Log-Scale)

Abbildung 10.1 Vergleich der Wortfrequenzen in Genesis (KJV) mit und ohne Stoppwörter.

Was sollen nun solche Worthäufigkeiten bezwecken? Das hängt natürlich immer von der Anwendung ab. Jedoch wurden in der Vergangenheit in der Forschung um den Ursprung von Homers Illiad und Odyssee lexikalische Metriken dazu benutzt, um herauszufinden, ob alle Bücher dieser zwei Werke vom gleichen Autor abstammen oder zur gleichen Zeit geschrieben worden sind (siehe auch Lamar (2024)). Das simpelste aller Masse ist beispielsweise die Type-Token-Relation (TTR). Diese gibt die lexikalische Diversität eines Textes an. Für die Genesis beträgt diese basierend auf der stoppwortbereinigten Häufigkeitsanalyse $\frac{2508}{44764} = 0.056$. Dieser Wert alleine sagt natürlich noch nichts aus, aber im Vergleich mit anderen TTR-Werten aus anderen Büchern aus der Bibel können interessante Thesen aufgestellt werden. Für längere

Texte sinkt die TTR typischerweise, weil neue Types seltener hinzukommen; TTR ist also nur eingeschränkt zwischen Texten unterschiedlicher Länge vergleichbar.

10.2.1 Video zu Stoppwörtern und Wortfrequenzanalyse

 Schaue dir jetzt das [Video 10.2.1](#) zu Stoppwörtern und Wortfrequenzanalyse an und bearbeite dann das nächste Unterkapitel.

10.3 Named Entity Recognition (NER)

Wir betrachten zum Schluss noch kurz die Eigennamenerkennung (EN *named entity recognition*, oder kurz NER). Diese ist eine gängige Methode im Bereich des *Information Retrievals*, also der Informationsextraktion. Dabei geht es darum, Eigennamen zu erkennen. Was Eigennamen sind ist wiederum anwendungsabhängig. Generell dazu gehören aber Namen von Personen, Orten, Bergen und Gewässern, aber auch Firmennamen oder Namen von Institutionen kann man dazu zählen. In CLTK ist eine solche Eigennamenerkennung für das Lateinische vorhanden, wie das Code-Beispiel 10.5 illustriert:

Code-Beispiel 10.5 Beispiel der Tokenisierung mit Leerzeichen

```
>>> from cltk.tag import ner
>>> tagged = ner.tag_ner('lat', input_text=text_la, type=list)
>>> [t for t in tagged if len(t) == 2]
[('Romani', 'Entity'), ('Christi', 'Entity'), ('Christum', 'Entity')]
```

Dieses minimale Beispiel zeigt, wie CLTK bereits mit Unterstützung zur Erkennung von Eigennamen geliefert wird. Dieses Modell wird aber den neuesten in Präzision und Ausbeute den neueren Modellen nachstehen. Wir sehen, dass das in CLTK integrierte NER-Modul Wörter, die es als Eigennamen erkannt hat, in einem Tupel um das Element 'Entity' ergänzt. Wir erfahren hingegen nicht, um was für einen Typen von Entität es sich handelt. Bezüglich der Qualität von NER-Verfahren lässt sich sagen: Gerade für historische Texte ist NER noch weit von “fertig” entfernt: Orthografievarianz, Abkürzungen und kasusbedingte Flexion erschweren die Erkennung, und viele Modelle sind auf klassischen Standardtexten trainiert, nicht auf Inschriften oder mittelalterliche Handschriften.

Trotzdem lassen sich solche Eigennamenerkennungen zum Beispiel nutzen, um automatisch Personen- und Ortsregister zu erstellen, prosopographische Netzwerke zu skizzieren oder Texte mit externen Ressourcen wie Prosopographien oder Ortsdatenbanken (beispielsweise Pleiades⁸) zu verknüpfen.

10.3.1 Video zu NER

 Schaue dir jetzt das [Video 10.3.1](#) zu NER an und bearbeite dann das Quiz in OLAT.

⁸Siehe <https://pleiades.stoa.org>.

10.4 Quiz zur Textanalyse historischer Daten

- ☑ Löse das [Quiz auf OLAT](#) und bearbeite dann das nächste Kapitel.

Kapitel

11

Literaturverzeichnis und
Glossar

Literaturverzeichnis

- Gaddis, J. L. (2002). *The Landscape of History: How Historians Map the Past*. Oxford University Press.
- Johnson, K. P., Burns, P. J., Stewart, J., Cook, T., Besnier, C., and Mattingly, W. J. B. (2021). The Classical Language Toolkit: An NLP Framework for Pre-Modern Languages. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, pages 20–29, Online. Association for Computational Linguistics.
- Jurafsky, D. and Martin, J. H. (2025). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd edition. Online manuscript released August 24, 2025.
- Kraft, K. (2001). “*Muster ohne Wert*”: zur Funktionalisierung und Marginalisierung des Musters. PhD thesis, Universität Dortmund.
- Lamar, K. A. (2024). *Formulaic Mechanics: A Computational Model of the Homeric Formula*. PhD thesis, Stanford University.
- Lockyear, K. (1997). *Multivariate Money. A Statistical Analysis of Roman Republican Coin Hoards with Special Reference to Material from Romania*. Ph.D., University of London, University College London (United Kingdom), England.
- Lévi-Strauss, C. (1973). *Das wilde Denken*. Suhrkamp-Taschenbuch Wissenschaft. Suhrkamp.
- Martin, M. (1977). Römische Schatzfunde aus Augst und Kaiseraugst. In *Augster Museumshefte*, volume 2. Max Muff AG, Pratteln.
- Moretti, F. (2013). *Distant Reading*. Verso Books.
- Seume, J. G. (1826). Von der Ordnung. In Schellenberg, J. P., editor, *Gesammelte Schriften*, volume 5. L. Schellenberg.

Bildquellen

- [Img1] WikimediaCommons (2024). File:olympic victors on papyrus 1185.jpg — wikimedia commons, the free media repository. [Online; accessed 7-July-2025].
- [Img2] WikimediaCommons (2025). File:abydos königsliste stitched 1.jpg — wikimedia commons, the free media repository. [Online; accessed 7-July-2025].

Glossar

abhängige Variable Die abhängige Variable ist die Grösse, deren Veränderung im Zusammenhang mit der unabhängigen Variable untersucht wird. Sie “reagiert” auf Veränderungen der unabhängigen Variable. In der historischen Analyse wäre dies zum Beispiel die Metallzusammensetzung der Münzen, die möglicherweise vom Fundort abhängt. 15

Argumente Argumente sind Werte, die an Funktionen übergeben werden, damit diese mit ihnen arbeiten können. In Python werden Argumente in Klammern hinter dem Funktionsnamen angegeben, z. B. `funktion(5, "Text")`. Eine Funktion kann mehrere Argumente entgegennehmen, und sie können unterschiedliche Datentypen haben. Historische Daten wie Jahreszahlen, Fundorte oder Beschreibungstexte können als Argumente an Analysefunktionen übergeben werden. 18

Attribut In der objekt-orientierten Programmierung ist ein Attribut eine Variable, die in einer Klasse definiert ist und Daten (Zustände) eines Objekts speichert. Jedes Objekt besitzt seine eigenen Attributwerte. Attribute werden meist innerhalb der `__init__`-Methode initialisiert und können mit Punktnotation (z. B. `objekt.name`) angesprochen werden.. 86

Automatentheorie Ein Teilgebiet der theoretischen Informatik, das sich mit abstrakten Maschinen (Automaten) zur Verarbeitung von Zeichenfolgen beschäftigt. In der Praxis liegt ihr ein Modell zugrunde, das reguläre Ausdrücke beschreibt: sogenannte endliche Automaten, die Zeichen Schritt für Schritt lesen und nach einem festen Regelsystem entscheiden, ob eine Eingabe akzeptiert wird. Obwohl die Automatentheorie in der Programmierung meist im Hintergrund bleibt, bildet sie die mathematische Grundlage für viele textverarbeitende Werkzeuge.. 44

Binärsystem Ein Zahlensystem, das nur die zwei Ziffern 0 und 1 kennt. Computer arbeiten intern mit dem Binärsystem, da elektrische Schaltungen nur zwei Zustände (an/aus) unterscheiden können. Zeichen und Daten werden als Kombinationen von Nullen und Einsen codiert. Beispielsweise steht 01000001 im ASCII-Zeichensatz für den Buchstaben A.. 74

Boolescher Wert Ein Boolescher Wert (engl. *Boolean*) ist ein Datentyp, der nur zwei mögliche Zustände annehmen kann: **True** (wahr) oder **False** (falsch).

Boolesche Werte werden in Python oft verwendet, um Bedingungen zu prüfen und den Programmfluss zu steuern.. 25

Built-in Namensraum Der Built-in Namensraum enthält alle Namen, die Python standardmässig zur Verfügung stellt zum Beispiel Funktionen wie `print()`, `len()` oder Konstanten wie `True` und `None`. Diese Namen stehen in jedem Python-Programm ohne zusätzliche Importe zur Verfügung und gehören zum fest eingebauten Funktionsumfang der Sprache.. 17

Debugging Debugging bezeichnet den Prozess des Findens und Behebens von Fehlern (Bugs) im Code. Dabei werden Programmfehler erkannt, analysiert und korrigiert. In Python kann Debugging durch das gezielte Einfügen von `print()`-Anweisungen, durch Debugger-Werkzeuge oder durch schrittweises Ausführen in Entwicklungsumgebungen (z. B. Jupyter oder PyCharm) unterstützt werden. Für die Arbeit mit historischen Daten ist Debugging wichtig, um unerwartete Ergebnisse in Analysen zu verstehen und zu verbessern.. 9

decodieren Die Umwandlung von codierten Daten in ein für Menschen oder Maschinen verständliches Format. Beim Lesen einer UTF-8-kodierten Textdatei bedeutet dies z. B., dass die Byte-Folgen in sichtbare Zeichen zurückverwandelt werden. In der digitalen Geschichtsforschung kann *Decodieren* auch das maschinelle Verstehen oder Interpretieren strukturierter Auszeichnung (z. B. TEI-XML) durch ein Programm oder eine Person bedeuten.. 74

Dictionary Ein Dictionary (Wörterbuch) ist ein veränderbarer, geordneter Datentyp in Python, der Daten als Paare von **Key** (Schlüssel) und **Value** (Wert) speichert. Jeder Key ist eindeutig und dient dazu, den zugehörigen Wert schnell abzurufen. Dictionaries eignen sich besonders für strukturierte Daten wie Fundbeschreibungen oder Metadaten historischer Objekte, z. B. `{"Fundort": "Aventicum", "Jahr": 305}`. 32

encodieren Im technischen Sinn: die Umwandlung von Informationen in ein bestimmtes Format, sodass sie von Computern effizient gespeichert, übertragen oder verarbeitet werden können. Im Kontext von Textdateien bedeutet das häufig die Umwandlung von Zeichen in eine Byte-Repräsentation, z. B. mittels UTF-8. In der digitalen Geschichtsforschung spricht man zudem oft vom *Auszeichnen* historischer Texte mit Metadaten – etwa mit `<placeName>` oder `<date>` in TEI-XML –, was man ebenfalls als *Encodierung* bezeichnet.. 74

Escape-Sequenz Eine Escape-Sequenz ist eine Zeichenkombination mit einem Backslash (`\`), die in Strings verwendet wird, um Steuerzeichen darzustellen. Zum Beispiel steht `\n` für einen Zeilenumbruch und `\t` für einen Tabulator. Escape-Sequenzen helfen, Textausgaben zu formatieren oder Sonderzeichen einzufügen.. 24

Float Ein Float (Gleitkommazahl) ist ein Datentyp für Zahlen mit Dezimalstellen, z. B. 3.14 oder -0.001. In Python wird dieser Typ verwendet, um mit historischen Grössen wie Gewichten, Masseinheiten oder Geldbeträgen zu rechnen, die nicht als ganze Zahlen dargestellt werden können. 16

Functional Programming Funktionale Programmierung ist ein Programmierparadigma, das Funktionen als zentrale Bausteine verwendet. Dabei wird darauf geachtet, dass Funktionen keine Nebeneffekte (z. B. veränderte globale Variablen) haben und immer denselben Rückgabewert für dieselben Eingaben liefern. Typische Merkmale sind die Nutzung von `map()`, `filter()`, `lambda`-Funktionen und rekursiven statt iterativen Strukturen. Das Paradigma stammt ursprünglich aus der Mathematik.. 84

globaler Namensraum Der globale Namensraum ist der Bereich, in dem Python Namen verwaltet, die ausserhalb von Funktionen oder Klassen definiert wurden also typischerweise auf Modulebene. Variablen im globalen Namensraum sind im gesamten Skript verfügbar, sofern sie nicht durch gleichnamige lokale Variablen überschrieben werden.. 17

Index Ein Index ist eine Positionsangabe für Elemente innerhalb einer geordneten Datenstruktur wie einer Liste oder einem String. In Python beginnt die Zählung bei 0, d. h. das erste Element hat den Index 0, das zweite den Index 1 usw. Der Index wird verwendet, um gezielt auf einzelne Elemente zuzugreifen oder sie zu verändern. In der Arbeit mit historischen Daten kann man so z. B. gezielt auf bestimmte Fundorte oder Jahreszahlen in einer Liste zugreifen.. 30

Integer Ein Integer (Ganzzahl) ist ein Datentyp, der ganze Zahlen ohne Dezimalstellen darstellt, z. B. -5, 0 oder 42. In Python gehören Integers zu den grundlegenden Datentypen und eignen sich für Zählungen, Indizes oder historische Jahreszahlen. 16

Interpreter Ein Interpreter ist ein Programm, das Quellcode zeilenweise analysiert und direkt ausführt, ohne ihn vorher in eine separate ausführbare Datei zu übersetzen. Python ist eine interpretierte Sprache: Der Python-Interpreter liest und verarbeitet den Code Schritt für Schritt zur Laufzeit. Dies erleichtert das Testen und Experimentieren, ist aber in der Ausführung meist langsamer als bei kompilierten Programmen. 16

Kernel Ein Kernel ist die zentrale Ausführungseinheit in interaktiven Programmumgebungen wie Jupyter Notebooks. Er führt den Code aus, verwaltet den aktuellen Zustand (z. B. Variablen im Speicher) und liefert Rückmeldungen an die Benutzeroberfläche. Wenn der Kernel neu gestartet wird, gehen alle gespeicherten Variablen und Ergebnisse verloren. 17

Key Ein Key (Schlüssel) ist ein Element in einem Dictionary (Wörterbuch), mit dem ein bestimmter Wert verknüpft ist. In Python dienen Keys dazu, Informationen gezielt und eindeutig abzurufen. Ein Dictionary besteht aus Key-Value-Paaren, z. B. `{"Fundort": "Aventicum"}`. Keys müssen unveränderlich (z. B. Strings oder Zahlen) und innerhalb eines Dictionaries eindeutig sein. In der historischen Datenverarbeitung können Keys etwa Inventarnummern oder Fundkategorien sein.. 32

Keyword Ein Keyword (dt. Schlüsselwort) ist ein reserviertes Wort in Python, das eine spezielle Bedeutung hat und Teil der Syntax ist. Keywords dürfen nicht als Variablennamen verwendet werden. Beispiele sind `if`, `for`, `while`, `class`, `def`, `True` oder `None`. Sie steuern den Aufbau und das Verhalten von Programmen. Eine vollständige Liste aller Keywords kann mit `import keyword; keyword.kwlist` angezeigt werden.. 38

Klasse Eine Klasse ist ein Bauplan für Objekte (Instanzen) in der objektorientierten Programmierung. Sie definiert Eigenschaften (Attribute) und Verhalten (Methoden), die ihre Objekte haben sollen. In der historischen Datenanalyse könnte man z. B. eine Klasse `Münze` erstellen, mit Attributen wie Fundort, Material oder Prägejahr. Jede einzelne Münze wäre dann ein Objekt (Instanz) dieser Klasse. Klassen helfen, Daten strukturiert und wiederverwendbar zu organisieren.. 22

Kleene-Stern Ein Operator in regulären Ausdrücken, benannt nach dem Mathematiker Stephen Kleene. Der Kleene-Stern (`*`) steht für null oder mehr Wiederholungen des unmittelbar vorangestellten Zeichens oder Ausdrucks. Beispielsweise steht der Ausdruck `a*` für die Zeichenfolgen "", "a", "aa", "aaa" usw. In der Praxis ermöglicht der Kleene-Stern, flexibel mit variierender Zeichenanzahl umzugehen – etwa bei unterschiedlich langen Namensformen oder wiederholten Satzzeichen.. 45

Konkatenation Konkatenation bezeichnet das Verketten (Zusammenfügen) von zwei oder mehreren Strings zu einem einzigen neuen String. In Python wird dafür der Operator `+` verwendet. Zum Beispiel ergibt `"Roma" + "num"` den String `"Romanum"`. In der historischen Datenanalyse kann man damit z. B. Fundortnamen mit Objektbezeichnungen kombinieren.. 24

Linked Open Data Ein Konzept der strukturierten Veröffentlichung von Daten im Web, das auf Offenheit, Interoperabilität und maschinenlesbare Verknüpfung setzt. Linked Open Data (LOD) basiert auf Standards wie URIs, RDF und SPARQL, damit Daten aus unterschiedlichen Quellen sinnvoll miteinander vernetzt und automatisiert weiterverwendet werden können. In den Geschichtswissenschaften erlaubt LOD etwa die Verbindung von Personen, Orten, Zeitpunkten und Ereignissen über verschiedene digitale Korpora hinweg (z. B. Prosopographien, Inschriften-Datenbanken, Wikidata).. 60

Liste Eine Liste ist ein geordneter, veränderbarer Datentyp in Python, der mehrere Elemente in einer festen Reihenfolge speichern kann. Listen können verschiedene Datentypen enthalten, zum Beispiel Zahlen, Strings oder sogar andere Listen. In der historischen Datenanalyse können Listen etwa verwendet werden, um Fundorte, Inventarnummern oder Jahreszahlen zu verwalten. Elemente in einer Liste werden über Indizes (Start bei 0) angesprochen.. 28

Logging Das **Logging** (Protokollieren) bezeichnet das automatische Aufzeichnen von Ereignissen, Fehlern und Statusmeldungen während der Ausführung eines Programms. Es dient der Fehlersuche (Debugging), Nachvollziehbarkeit

und Qualitätssicherung. In Python erfolgt Logging über das eingebaute Modul `logging`, das Nachrichten mit unterschiedlichem Schweregrad (z. B. `info`, `warning`, `error`) speichern kann – etwa in einer Datei oder auf der Konsole.. 94

lokaler Namensraum Der lokale Namensraum ist der Bereich, in dem Python Namen (z. B. Variablen) verwaltet, die innerhalb einer Funktion oder eines Blocks definiert wurden. Diese Namen sind nur während der Ausführung dieser Funktion sichtbar und verfügbar. Nach Beenden der Funktion wird der lokale Namensraum gelöscht.. 17

Loop Ein Loop (dt. Schleife) ist eine Kontrollstruktur, mit der ein bestimmter Codeblock mehrfach ausgeführt wird. In Python gibt es vor allem zwei Arten: den `for`-Loop, der über eine Sequenz (z. B. eine Liste) iteriert, und den `while`-Loop, der wiederholt wird, solange eine Bedingung erfüllt ist. Loops sind hilfreich, um wiederkehrende Aufgaben durchzuführen, zum Beispiel alle Münzen in einer Fundliste auszugeben oder statistische Berechnungen für mehrere Jahre durchzuführen.. 28, 37

Modul Ein Modul ist eine Datei, die Python-Code enthält – etwa Funktionen, Klassen oder Konstanten – und in anderen Programmen wiederverwendet werden kann. Python stellt viele nützliche Module zur Verfügung (z. B. `re` für reguläre Ausdrücke oder `math` für mathematische Funktionen). Eigene Module können ebenfalls erstellt werden, um den Code übersichtlich und modular zu gestalten.. 48

Mutierbarkeit Mutierbarkeit beschreibt, ob ein Objekt nach seiner Erstellung verändert (mutiert) werden kann. In Python gibt es mutierbare Objekte wie Listen oder Dictionaries—deren Inhalt kann nachträglich angepasst werden. Demgegenüber stehen nicht mutierbare (unveränderliche) Objekte wie Strings, Tupel oder Integers, die nach ihrer Erstellung nicht verändert werden können. Das Verständnis dieser Unterscheidung ist wichtig, um ungewollte Nebeneffekte beim Programmieren zu vermeiden.. 20

Namensraum Ein Namensraum (engl. *namespace*) ist ein Speicherbereich, in dem Python Namen wie Variablen, Funktionen oder Klassen verwaltet. Je nach Kontext gibt es unterschiedliche Namensräume, z. B. lokale innerhalb von Funktionen oder globale auf Modulebene. So wird sichergestellt, dass Namen nicht miteinander in Konflikt geraten. 17

objektorientierte Programmierung Ein Programmierparadigma, bei dem Programme in Form von sogenannten Objekten strukturiert werden. Jedes Objekt kombiniert Daten (Attribute) und Verhalten (Methoden). Objekte sind Instanzen von Klassen – abstrakten Bauplänen, die definieren, wie solche Objekte aufgebaut sind. Dieses Paradigma ermöglicht es, komplexe Programme modular, wiederverwendbar und wartbar zu gestalten. Zentral sind Konzepte wie *Vererbung*, *Kapselung* und *Polymorphie*.. 84

Reguläre Ausdrücke Ein regulärer Ausdruck (engl. *regular expression*, kurz: **RegEx**) ist ein Suchmuster, das verwendet wird, um bestimmte Zeichenfolgen in Texten zu erkennen, zu extrahieren oder zu ersetzen. In der Geschichtsforschung sind reguläre Ausdrücke besonders hilfreich beim automatisierten Durchsuchen und Strukturieren grosser Textmengen, etwa um Datumsangaben, Ortsnamen oder Zitate zu identifizieren.. 44

Schemata Strukturvorlagen oder Erwartungsmuster, die dabei helfen, Informationen zu ordnen, zu interpretieren oder systematisch zu erfassen. In der Geschichtsforschung können Schemata z. B. typische Aufbauformen von Quellen (wie Briefe, Inschriften oder Urkunden) sein. In der Datenverarbeitung bezeichnen Schemata zudem formale Definitionen von Datenstrukturen, etwa bei XML- oder JSON-Daten.. 57

Set Ein Set (Menge) ist ein veränderbarer, ungeordneter Datentyp in Python, der eine Sammlung einzigartiger (nicht doppelter) Elemente speichert. Sets eignen sich besonders, um Duplikate zu entfernen oder Mengenoperationen wie Vereinigung, Schnitt oder Differenz durchzuführen. In der historischen Datenverarbeitung kann ein Set z. B. verwendet werden, um die eindeutigen Fundorte in einer Liste zu ermitteln.. 34

Signatur Die **Signatur** einer Funktion beschreibt ihren Namen und die Anzahl sowie Reihenfolge der erwarteten Argumente. Sie sagt also aus, *wie* eine Funktion aufgerufen werden muss. Beispiel: Die Signatur von `len(s)` besteht aus dem Funktionsnamen `len` und einem Argument `s`. Die Signatur ist hilfreich, um eine Funktion korrekt zu verwenden – insbesondere bei mehreren oder optionalen Argumenten.. 78

Stand-off Annotation Eine Methode der Annotation, bei der die Anmerkungen nicht direkt im Text eingebettet, sondern in einer separaten Datei oder Struktur gespeichert werden. Statt den Text zu verändern, wird auf Textstellen über Positionsangaben oder eindeutige IDs verwiesen. Dies erlaubt parallele Annotationen, erhöht die Nachvollziehbarkeit und erleichtert die Verarbeitung durch Software – besonders bei digitalen Editionen, Korpora oder mehrschichtigen Interpretationen historischer Quellen.. 63

String Ein String (Zeichenkette) ist ein Datentyp, der eine Folge von Zeichen repräsentiert, z. B. Wörter, Sätze oder alphanumerische Codes. In Python werden Strings durch einfache oder doppelte Anführungszeichen dargestellt, z. B. `"Inschrift"` oder `'Provincia'` nützlich etwa für Ortsnamen, Fundbeschreibungen oder Quelltexte. 16

Token Ein **Token** ist eine kleinste Einheit, mit der ein Verfahren zur Text- oder Codeanalyse arbeitet. Beim sogenannten *Tokenising* wird ein Text in solche Einheiten zerlegt, meist Wörter, Satzzeichen oder Operatoren. In der natürlichen Sprachverarbeitung bezeichnet ein Token typischerweise ein einzelnes Wort oder Zeichen; in der Programmierung kann ein Token auch eine Zahl, ein Bezeichner oder ein Symbol wie `+` oder `=` sein.. 106

Topic Modeling **Topic Modeling** ist eine Methode des maschinellen Lernens, um in grossen Textsammlungen automatisch thematische Strukturen zu erkennen. Dabei werden wiederkehrende Wortmuster identifiziert und zu sogenannten “Topics” (Themen) zusammengefasst. Ein Text kann mehreren Topics zugeordnet werden. Das bekannteste Verfahren ist LDA (Latent Dirichlet Allocation). In der Geschichtsforschung hilft Topic Modeling, thematische Trends oder Diskursverschiebungen in grossen Textkorpora sichtbar zu machen.. 107

Tupel Ein Tupel ist eine geordnete, aber *nicht* veränderbare (nicht mutierbare) Datenstruktur in Python. Es funktioniert ähnlich wie eine Liste, verwendet jedoch runde Klammern () statt eckiger Klammern []. Tupel eignen sich gut für Datenpaare oder feste Werte, die nicht verändert werden sollen – etwa geographische Koordinaten, Zeitangaben oder feste Kombinationen wie (“Aventicum”, 211) für einen Fundort mit zugehörigem Jahr.. 36

Type In der Computerlinguistik ist ein **Type** eine eindeutige Wortform, die im Text mindestens einmal vorkommt. Er unterscheidet sich vom **Token**, der jede einzelne Wortvorkommnis zählt. Beispiel: Im Satz “alea iacta est” kommen drei Tokens, aber ebenfalls drei Types vor. Im Satz “A rose is a rose is a rose” sind es acht Tokens, aber nur drei Types (vier, wenn wir die Grossschreibung miteinberechnen).. 109

unabhängige Variable Eine unabhängige Variable ist eine Grösse, die in einem Experiment oder einer Datenanalyse gezielt verändert oder beobachtet wird, um ihre Auswirkungen auf eine andere Grösse zu untersuchen. In historischen Forschungsfragen könnte dies zum Beispiel der Fundort eines Münzhorts sein . 15

Value Ein Value (Wert) ist das Element in einem Dictionary (Wörterbuch), das mit einem bestimmten Key (Schlüssel) verknüpft ist. In Python wird ein Value durch seinen zugehörigen Key angesprochen, z. B. `daten["Fundort"]`. Values können beliebige Datentypen haben – Zahlen, Strings, Listen oder sogar weitere Dictionaries. In der historischen Analyse enthalten Values z. B. Informationen zu Fundorten, Materialien oder Datierungen.. 32

Wildcard Ein Platzhalterzeichen, das in regulären Ausdrücken oder Suchanfragen verwendet wird, um für ein oder mehrere beliebige Zeichen zu stehen. In vielen Suchsystemen steht das Sternchen * für beliebig viele Zeichen, das Fragezeichen ? für genau eines. In Python sind sogenannte Wildcards z. B. im Modul `re` (für reguläre Ausdrücke) oder bei Dateipfadsuchen mit dem Modul `glob` relevant.. 50

